

TFM script

Ana Martín Ramírez

02-01-2026

1. CONTROL DE CALIDAD y PROCESAMIENTO DE DATOS

```
# -----
# 0. Instalación y carga de paquetes
# -----
required_cran <- c("ggplot2", "ggrepel", "reshape2", "stats", "dplyr")
required_bioc <- c("GEOquery", "edgeR", "limma")

for (pkg in required_cran) {
  if (!requireNamespace(pkg, quietly = TRUE)) install.packages(pkg, dependencies =
TRUE)
  library(pkg, character.only = TRUE)
}

if (!requireNamespace("BiocManager", quietly = TRUE)) install.packages("BiocManager")
for (pkg in required_bioc) {
  if (!requireNamespace(pkg, quietly = TRUE)) BiocManager::install(pkg, update = FALSE)
  library(pkg, character.only = TRUE)
}

# =====
# CONTROL DE CALIDAD Y NORMALIZACIÓN DE LOS DATOS DE EXPRESIÓN
# =====

# 1. Definición de las muestras por dataset (Validado y Corregido)
GSE63420_groups <- list(NT =
c("GSM1548453", "GSM1548454", "GSM1548455", "GSM1548456", "GSM1548457", "GSM1548458", "GSM15484
59"), T =
c("GSM1548460", "GSM1548461", "GSM1548462", "GSM1548463", "GSM1548464", "GSM1548465", "GSM15484
66"))

GSE119336_groups <- list(NT =
c("GSM3370133", "GSM3370135", "GSM3370137", "GSM3370139", "GSM3370141", "GSM3370143", "GSM33701
45", "GSM3370147", "GSM3370149", "GSM3370151", "GSM3370153", "GSM3370155", "GSM3370157", "GSM337
0159", "GSM3370161"), T =
c("GSM3370134", "GSM3370136", "GSM3370138", "GSM3370140", "GSM3370142", "GSM3370144", "GSM33701
46", "GSM3370148", "GSM3370150", "GSM3370152", "GSM3370154", "GSM3370156", "GSM3370158", "GSM337
0160", "GSM3370162"))
)

GSE107943_groups <- list(NT =
c("GSM2883958", "GSM2883959", "GSM2883960", "GSM2883961", "GSM2883962", "GSM2883963", "GSM28839
64", "GSM2883965", "GSM2883966", "GSM2883967", "GSM2883968", "GSM2883969", "GSM2883970", "GSM288
3971", "GSM2883972", "GSM2883973", "GSM2883974", "GSM2883975", "GSM2883976", "GSM2883977", "GSM2
883978", "GSM2883979", "GSM2883980", "GSM2883981", "GSM2883982", "GSM2883983", "GSM2883984"),
T =
c("GSM2883928", "GSM2883929", "GSM2883930", "GSM2883931", "GSM2883932", "GSM2883933", "GSM28839
34", "GSM2883935", "GSM2883936", "GSM2883937", "GSM2883938", "GSM2883939", "GSM2883940", "GSM288
3941", "GSM2883942", "GSM2883943", "GSM2883944", "GSM2883945", "GSM2883946", "GSM2883947", "GSM2
883948", "GSM2883949", "GSM2883950", "GSM2883951", "GSM2883952", "GSM2883953", "GSM2883954", "GS
M2883955", "GSM2883956", "GSM2883957"))
)
```



```

"GSM7830165", "GSM7830166", "GSM7830167")),

groups=list(T=c("GSM7829922", "GSM7829923", "GSM7829924", "GSM7829925", "GSM7829926", "GSM7829
927", "GSM7829928", "GSM7829929", "GSM7829930", "GSM7829931", "GSM7829932", "GSM7829933", "GSM78
29934", "GSM7829935", "GSM7829936", "GSM7829937", "GSM7829938", "GSM7829939", "GSM7829940", "GSM
7829941", "GSM7829942", "GSM7829943", "GSM7829944", "GSM7829945", "GSM7829946", "GSM7829947", "G
SM7829948", "GSM7829949", "GSM7829950", "GSM7829951", "GSM7829952", "GSM7829953", "GSM7829954",
"GSM7829955", "GSM7829956", "GSM7829957", "GSM7829958", "GSM7829959", "GSM7829960", "GSM7829961
", "GSM7829962", "GSM7829963", "GSM7829964", "GSM7829965", "GSM7829966", "GSM7829967", "GSM78299
68", "GSM7829969", "GSM7829970", "GSM7829971", "GSM7829972", "GSM7829973", "GSM7829974", "GSM782
9975", "GSM7829976", "GSM7829977", "GSM7829978", "GSM7829979", "GSM7829980", "GSM7829981", "GSM7
829982", "GSM7829983", "GSM7829984", "GSM7829985", "GSM7829986", "GSM7829987", "GSM7829988", "GSM
7829989", "GSM7829990", "GSM7829991", "GSM7829992", "GSM7829993", "GSM7829994", "GSM7829995", "G
SM7829996", "GSM7829997", "GSM7829998", "GSM7829999", "GSM7830000", "GSM7830001", "GSM7830002",
"GSM7830003", "GSM7830004", "GSM7830005", "GSM7830006", "GSM7830007", "GSM7830008", "GSM7830009",
"GSM7830110", "GSM7830111", "GSM7830112", "GSM7830113", "GSM7830114", "GSM7830115", "GSM78301
16", "GSM7830117", "GSM7830118", "GSM7830119", "GSM7830120", "GSM7830121", "GSM7830122", "GSM78
30123", "GSM7830124", "GSM7830125", "GSM7830126", "GSM7830127", "GSM7830128", "GSM7830129", "GSM
7830130", "GSM7830131", "GSM7830132", "GSM7830133", "GSM7830134", "GSM7830135", "GSM7830136", "G
SM7830137", "GSM7830138", "GSM7830139", "GSM7830140", "GSM7830141", "GSM7830142", "GSM7830143",
"GSM7830144", "GSM7830145", "GSM7830146", "GSM7830147", "GSM7830148", "GSM7830149", "GSM7830150",
"GSM7830151", "GSM7830152", "GSM7830153", "GSM7830154", "GSM7830155", "GSM7830156", "GSM78301
57", "GSM7830158", "GSM7830159", "GSM7830160", "GSM7830161", "GSM7830162", "GSM7830163", "GSM783
0164", "GSM7830165", "GSM7830166", "GSM7830167"))))

# 2. Función para Leer TSV
read_archs4_tsv <- function(path) {
  df <- tryCatch(read.delim(path, header = TRUE, stringsAsFactors = FALSE, check.names
= FALSE),
    error = function(e) read.delim(path, header = FALSE, stringsAsFactors
= FALSE, check.names = FALSE))
  if (names(df)[1] == "" || names(df)[1] %in% c("V1", "gene_id", "Gene")) names(df)[1]
<- "gene"
  return(df)
}

# 3. Función principal para QC, PCA, y Eliminación de Outliers
run_qc_and_pca <- function(path, dataset_name, keep_samples, groups_list,
min_expr_fraction = 0.2, count_threshold = 1, sd_threshold = 3) {
  message("Procesando: ", dataset_name)
  out_dir <- "QC_filtered"
  if(!dir.exists(out_dir)) dir.create(out_dir)

  tsv <- read_archs4_tsv(path)
  genes <- as.character(tsv[[1]])
  mat_counts_raw <- tsv[, -1]
  rownames(mat_counts_raw) <- make.unique(genes)
  colnames(mat_counts_raw) <- gsub("^X", "", colnames(mat_counts_raw))

  samples_present <- intersect(keep_samples, colnames(mat_counts_raw))
  if (length(samples_present) == 0) stop("No se encontraron muestras en ",
dataset_name)
  mat_sub <- mat_counts_raw[, samples_present, drop = FALSE]
  mat_sub[] <- lapply(mat_sub, as.numeric)
  mat_sub[is.na(mat_sub)] <- 0

  keep_genes <- rowSums(mat_sub > count_threshold) >= (min_expr_fraction *
ncol(mat_sub))
  mat_filtered <- mat_sub[keep_genes, , drop = FALSE]
  write.csv(mat_filtered, file.path(out_dir, paste0(dataset_name,
"_counts_filtered.csv")))
}

```

```

dge <- edgeR::DGEList(counts = mat_filtered)
dge <- edgeR::calcNormFactors(dge, method = "TMM")
mat_cpm_log <- edgeR::cpm(dge, log = TRUE, prior.count = 1)
write.csv(mat_cpm_log, file.path(out_dir, paste0(dataset_name,
"_cpm_log2_filtered.csv")))

pca <- prcomp(t(mat_cpm_log), center = TRUE, scale. = TRUE)
pca_df <- data.frame(Sample = rownames(pca$x), PC1 = pca$x[,1], PC2 = pca$x[,2])

group_vector <- character(length(pca_df$Sample))
names(group_vector) <- pca_df$Sample
for (group_name in names(groups_list)) {
  samples_in_group <- intersect(groups_list[[group_name]], pca_df$Sample)
  group_vector[samples_in_group] <- ifelse(group_name == "NT", "No Tumoral",
ifelse(group_name == "T", "Tumoral", group_name))
}
pca_df$Group <- factor(group_vector, levels = unique(c("No Tumoral", "Tumoral",
unique(group_vector))))
pca_df$Group_Code <- sapply(group_vector, function(x) ifelse(x == "No Tumoral", "NT",
"T"))

pca_df <- pca_df %>% group_by(Group_Code) %>%
  mutate(Dist_Centroide = sqrt((PC1 - mean(PC1))^2 + (PC2 - mean(PC2))^2),
    Threshold = mean(Dist_Centroide) + sd_threshold * sd(Dist_Centroide),
    Outlier = Dist_Centroide > Threshold) %>% ungroup()

color_map_final <- c("No Tumoral" = "green4", "Tumoral" = "red3")[names(c("No
Tumoral" = "green4", "Tumoral" = "red3")) %in% unique(pca_df$Group)]

p_pca <- ggplot(pca_df, aes(PC1, PC2, color = Group)) + geom_point(size = 3) +
  geom_point(data = subset(pca_df, Outlier == TRUE), shape = 21, size = 5, color =
"black", fill = NA, stroke = 1.5) +
  geom_text_repel(data = subset(pca_df, Outlier == TRUE), aes(label = Sample),
color = "red", size = 3, force = 2) +
  labs(title = paste0("PCA: ", dataset_name), subtitle = paste("Outliers
detectados:", sum(pca_df$Outlier)),
x = paste0("PC1 (", round(summary(pca)$importance[2,1]*100, 1), "%)"),
y = paste0("PC2 (", round(summary(pca)$importance[2,2]*100, 1), "%)")) +
  scale_color_manual(values = color_map_final) + theme_minimal()
print(p_pca)

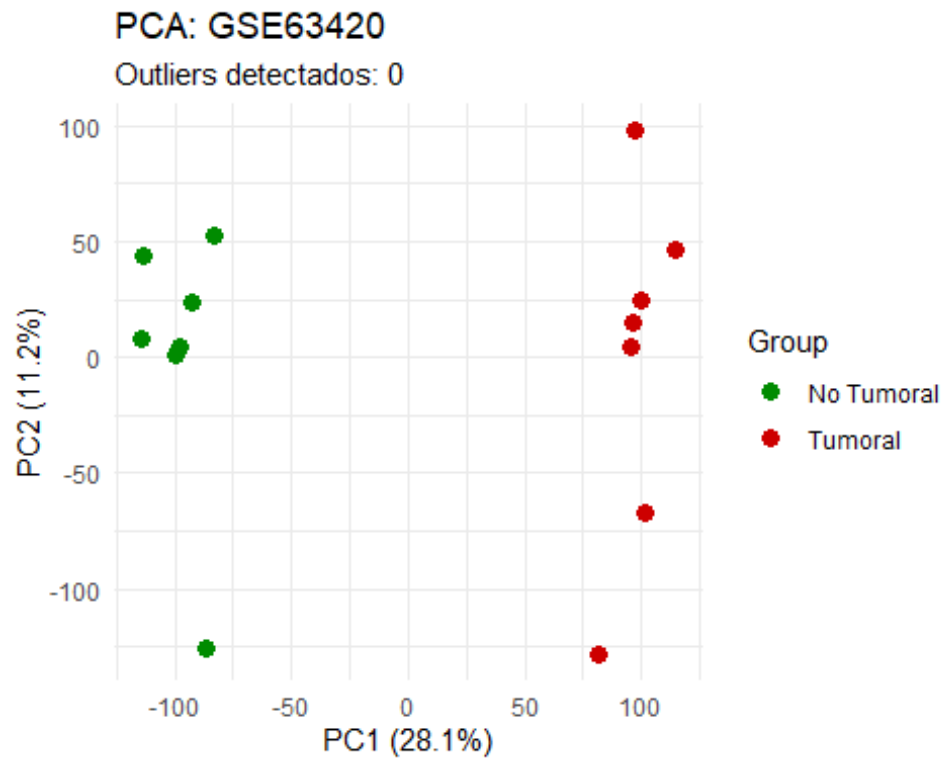
samples_to_keep <- pca_df$Sample[!pca_df$Outlier]
mat_cpm_log_definitivo <- mat_cpm_log[, samples_to_keep, drop = FALSE]
write.csv(mat_cpm_log_definitivo, file.path(out_dir, paste0(dataset_name,
"_cpm_log2_filtered_definitivos.csv")))

return(list(qc_row = data.frame(Dataset = dataset_name, Muestras_finales =
ncol(mat_cpm_log_definitivo), Outliers = sum(pca_df$Outlier), mat_definitiva =
mat_cpm_log_definitivo))
}

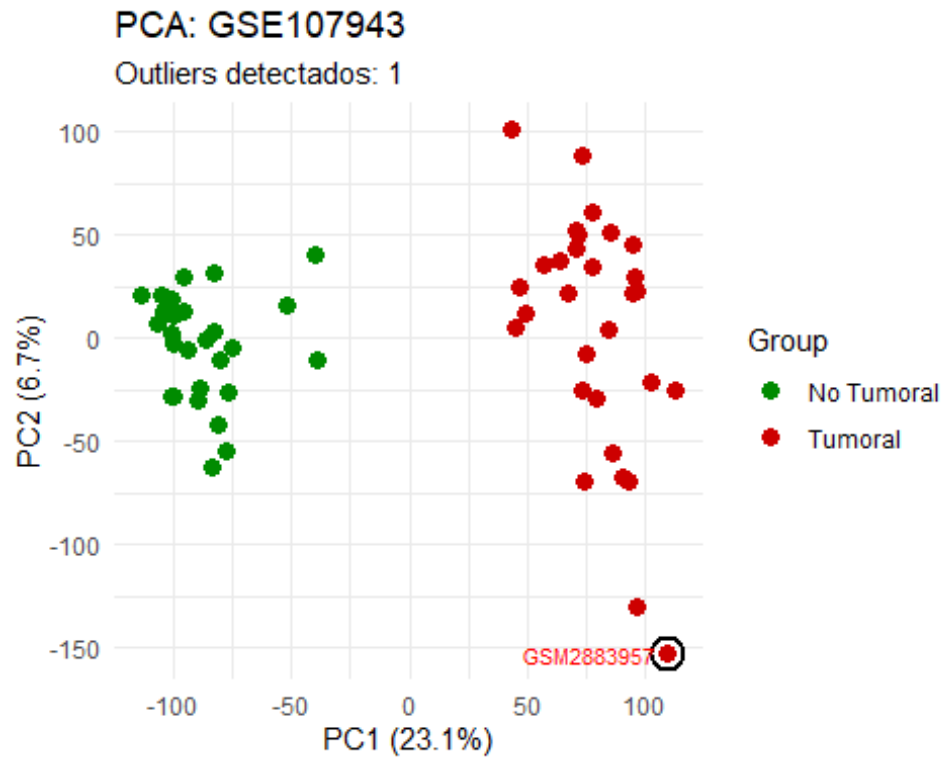
# 4. Ejecutar para todos los datasets
qc_results_list <- lapply(names(datasets_info), function(ds) {
  info <- datasets_info[[ds]]
  if (!file.exists(info$file)) return(NULL)
  run_qc_and_pca(info$file, ds, info$samples, groups_list = info$groups, sd_threshold =
3)
})

```

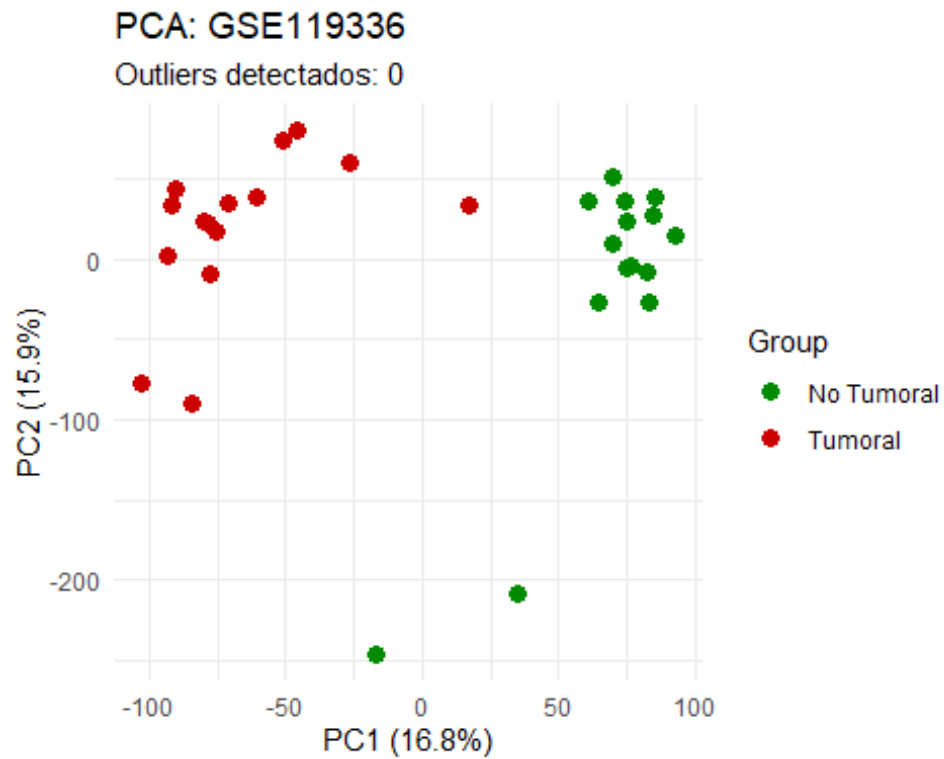
```
## Procesando: GSE63420
```



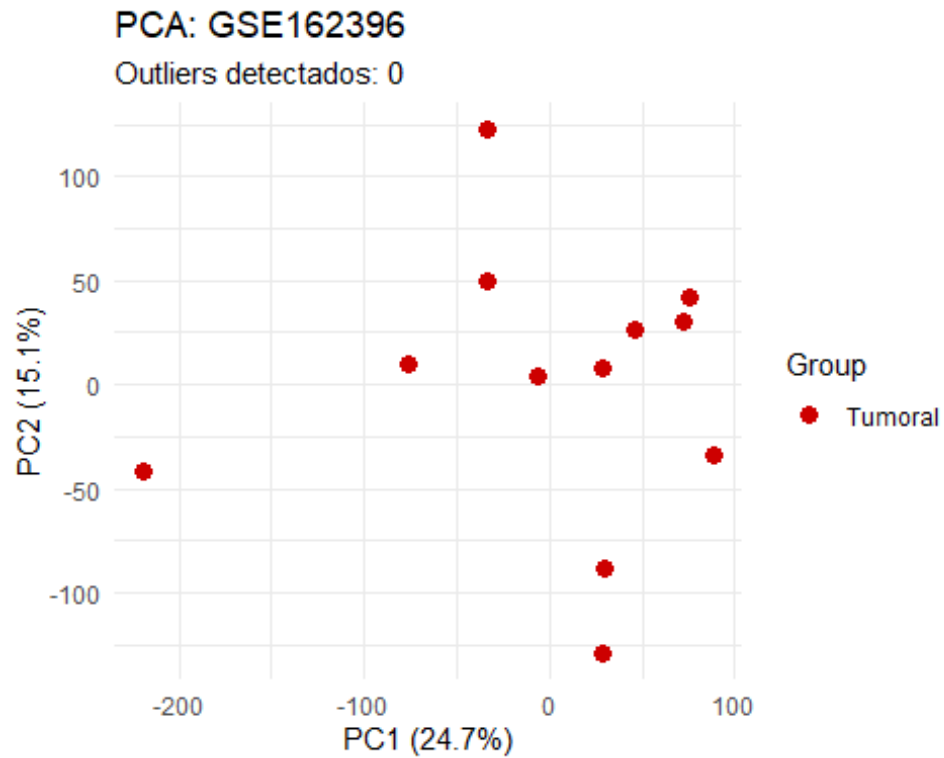
Procesando: GSE107943



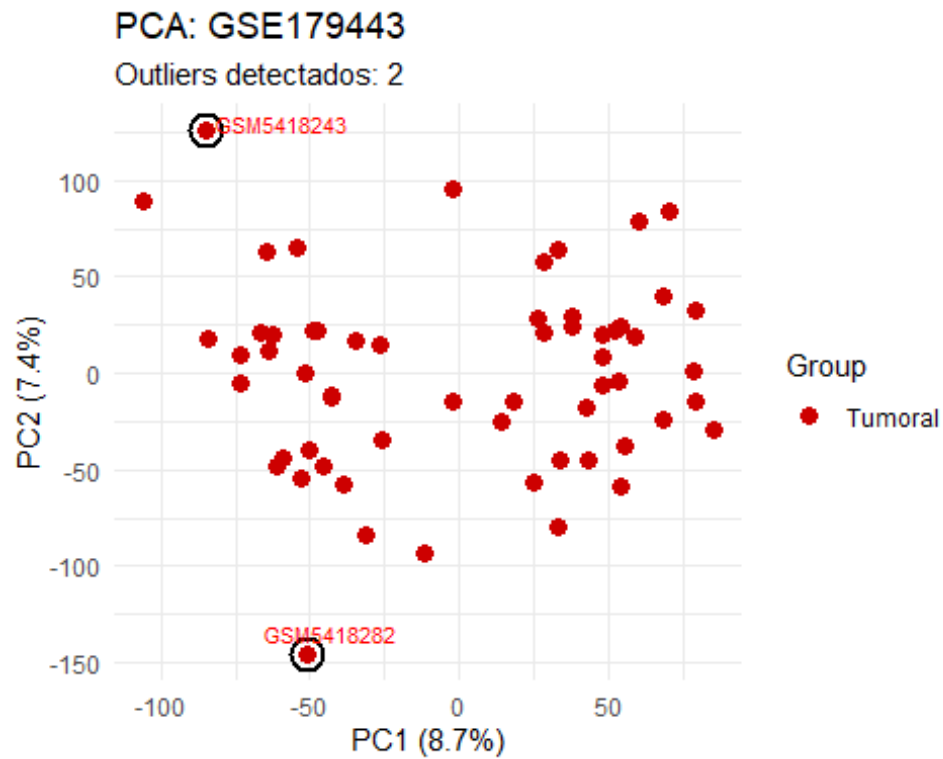
Procesando: GSE119336



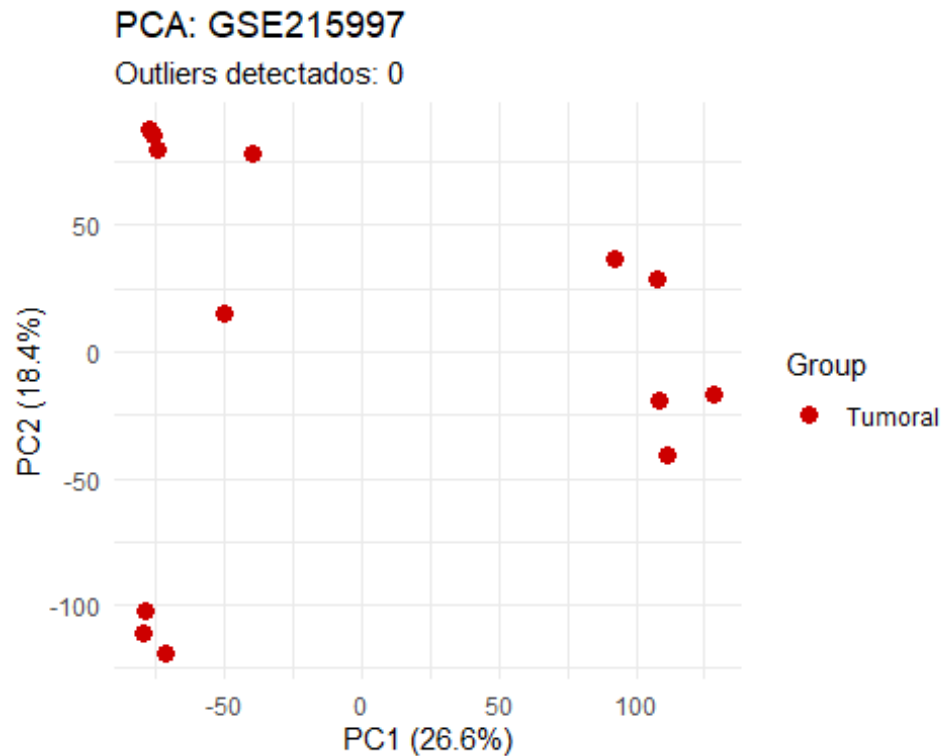
Procesando: GSE162396



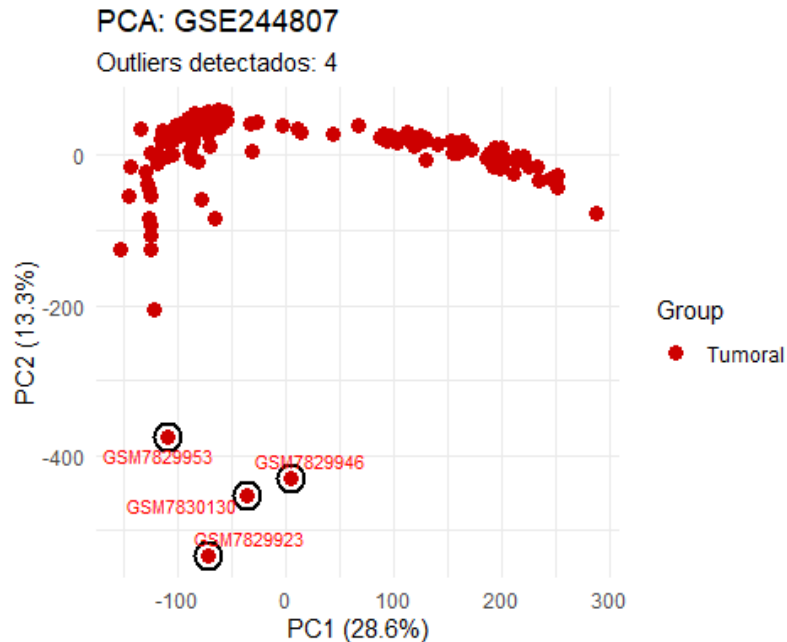
Procesando: GSE179443



Procesando: GSE215997



Procesando: GSE244807



5. Mostrar resumen final

```
valid_qc_results <- qc_results_list[!sapply(qc_results_list, is.null)]
qc_summary <- do.call(rbind, lapply(valid_qc_results, function(x) x$qc_row))
print(qc_summary)
```

```
##      Dataset Muestras_finales Outliers
## 1 GSE63420          14          0
## 2 GSE107943         56          1
## 3 GSE119336         30          0
## 4 GSE162396         12          0
## 5 GSE179443         57          2
## 6 GSE215997         13          0
## 7 GSE244807        142          4
```

2. INTEGRACIÓN DE DATOS DE EXPRESIÓN GÉNICA

```
library(dplyr); library(sva); library(ggplot2); library(factoextra)
```

--- 1. Definir Rutas y Archivos ---

```
input_dir <- "QC_filtered"
output_dir <- "Datos_Integrados"
output_file <- "matriz_con_batch.csv"
if (!dir.exists(output_dir)) dir.create(output_dir)
```

--- 2. Carga y Consolidación de Datasets ---

```
dataset_files <- list(
  GSE63420 = file.path(input_dir, "GSE63420_cpm_log2_filtered_definitivos.csv"),
  GSE107943 = file.path(input_dir, "GSE107943_cpm_log2_filtered_definitivos.csv"),
  GSE162396 = file.path(input_dir, "GSE162396_cpm_log2_filtered_definitivos.csv"),
  GSE179443 = file.path(input_dir, "GSE179443_cpm_log2_filtered_definitivos.csv"),
  GSE119336 = file.path(input_dir, "GSE119336_cpm_log2_filtered_definitivos.csv"),
  GSE215997 = file.path(input_dir, "GSE215997_cpm_log2_filtered_definitivos.csv"),
  GSE244807 = file.path(input_dir, "GSE244807_cpm_log2_filtered_definitivos.csv")
)
```



```

dataset_list <- list()
for (gse in names(dataset_files)) {
  path <- dataset_files[[gse]]
  if (file.exists(path)) {
    mat <- read.csv(path, row.names = 1, check.names = FALSE)
    dataset_list[[gse]] <- as.matrix(mat)
  } else { warning(paste("ADVERTENCIA: Archivo no encontrado para", gse)) }
}

if (length(dataset_list) == 0) stop("ERROR: Ningún dataset cargado.")
cat(paste("Total datasets cargados:", length(dataset_list), "\n"))

## Total datasets cargados: 7

common_genes <- Reduce(intersect, lapply(dataset_list, rownames))
cat(paste("Genes comunes:", length(common_genes), "\n"))

## Genes comunes: 24141

integrated_list <- lapply(dataset_list, function(mat) mat[common_genes, ])
matriz_con_batch <- do.call(cbind, integrated_list)
cat(paste("Dimensiones finales:", paste(dim(matriz_con_batch), collapse = " x "), "\n"))

## Dimensiones finales: 24141 x 324

# --- 3. Creación de La Variable Lote (Batch) ---

sample_names <- colnames(matriz_con_batch)
batch_vector <- character(length(sample_names))
start_index <- 1
for (gse_id in names(dataset_list)) {
  num_samples <- ncol(dataset_list[[gse_id]])
  end_index <- start_index + num_samples - 1
  batch_vector[start_index:end_index] <- gse_id
  start_index <- end_index + 1
}

pheno <- data.frame(sample = sample_names, batch = as.factor(batch_vector), row.names =
sample_names)

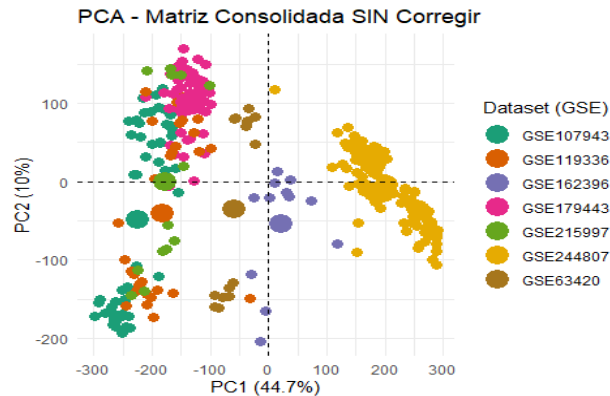
# --- 4. Guardado de La Matriz Consolidada ---
full_output_path <- file.path(output_dir, output_file)
write.csv(matriz_con_batch, full_output_path, quote = FALSE)

# =====
# 5. COMPROBACIÓN DEL EFECTO LOTE (PCA)
# =====

run_pca_and_plot <- function(data_matrix, pheno_df, title_suffix) {
  data_pca <- t(data_matrix)
  if (ncol(data_pca) < 2 || nrow(data_pca) < 2) {
    warning("Datos insuficientes para PCA."); return(NULL)
  }
  pca_res <- prcomp(data_pca, center = TRUE, scale. = FALSE)
  p <- fviz_pca_ind(pca_res, geom.ind = "point", col.ind = pheno_df$batch,
    pointshape = 19, palette = "Dark2", legend.title = "Dataset (GSE)",
    pointsize = 3, title = paste("PCA -", title_suffix),
    xlab = paste0("PC1 (", round(get_eigenvalue(pca_res)[1,
"variance.percent"], 1), "%)"),
    ylab = paste0("PC2 (", round(get_eigenvalue(pca_res)[2,
"variance.percent"], 1), "%)")) + theme_minimal()
  print(p)
}

```

```
run_pca_and_plot(matriz_con_batch, pheno, "Matriz Consolidada SIN Corregir")
```



3. FILTRADO DE METADATOS

```
# 1. INSTALACIÓN Y CARGA DE LIBRERÍAS (Sin cambios)
# -----
options(repos = c(CRAN = "https://cran.rstudio.com/"))
if (!requireNamespace("tidyverse", quietly = TRUE)) install.packages("tidyverse")
if (!requireNamespace("janitor", quietly = TRUE)) install.packages("janitor")
library(tidyverse); library(janitor)

# -----
# 2. DEFINICIÓN DE DATOS (MUESTRAS Y ARCHIVOS)
# -----

muestras_interes <- list(
  GSE63420 =
c("GSM1548453", "GSM1548454", "GSM1548455", "GSM1548456", "GSM1548457", "GSM1548458", "GSM1548459", "GSM1548460", "GSM1548461", "GSM1548462", "GSM1548463", "GSM1548464", "GSM1548465", "GSM1548466"),
  GSE119336 =
c("GSM3370133", "GSM3370135", "GSM3370137", "GSM3370139", "GSM3370141", "GSM3370143", "GSM3370145", "GSM3370147", "GSM3370149", "GSM3370151", "GSM3370153", "GSM3370155", "GSM3370157", "GSM3370159", "GSM3370161", "GSM3370163", "GSM3370165", "GSM3370167", "GSM3370169", "GSM3370171", "GSM3370173", "GSM3370175", "GSM3370177", "GSM3370179", "GSM3370181", "GSM3370183", "GSM3370185", "GSM3370187", "GSM3370189", "GSM3370191", "GSM3370193", "GSM3370195", "GSM3370197", "GSM3370199", "GSM3370201", "GSM3370203", "GSM3370205", "GSM3370207", "GSM3370209", "GSM3370211", "GSM3370213", "GSM3370215", "GSM3370217", "GSM3370219", "GSM3370221", "GSM3370223", "GSM3370225", "GSM3370227", "GSM3370229", "GSM3370231", "GSM3370233", "GSM3370235", "GSM3370237", "GSM3370239", "GSM3370241", "GSM3370243", "GSM3370245", "GSM3370247", "GSM3370249", "GSM3370251", "GSM3370253", "GSM3370255", "GSM3370257", "GSM3370259", "GSM3370261", "GSM3370263", "GSM3370265", "GSM3370267", "GSM3370269", "GSM3370271", "GSM3370273", "GSM3370275", "GSM3370277", "GSM3370279", "GSM3370281", "GSM3370283", "GSM3370285", "GSM3370287", "GSM3370289", "GSM3370291", "GSM3370293", "GSM3370295", "GSM3370297", "GSM3370299", "GSM3370301", "GSM3370303", "GSM3370305", "GSM3370307", "GSM3370309", "GSM3370311", "GSM3370313", "GSM3370315", "GSM3370317", "GSM3370319", "GSM3370321", "GSM3370323", "GSM3370325", "GSM3370327", "GSM3370329", "GSM3370331", "GSM3370333", "GSM3370335", "GSM3370337", "GSM3370339", "GSM3370341", "GSM3370343", "GSM3370345", "GSM3370347", "GSM3370349", "GSM3370351", "GSM3370353", "GSM3370355", "GSM3370357", "GSM3370359", "GSM3370361", "GSM3370363", "GSM3370365", "GSM3370367", "GSM3370369", "GSM3370371", "GSM3370373", "GSM3370375", "GSM3370377", "GSM3370379", "GSM3370381", "GSM3370383", "GSM3370385", "GSM3370387", "GSM3370389", "GSM3370391", "GSM3370393", "GSM3370395", "GSM3370397", "GSM3370399", "GSM3370401", "GSM3370403", "GSM3370405", "GSM3370407", "GSM3370409", "GSM3370411", "GSM3370413", "GSM3370415", "GSM3370417", "GSM3370419", "GSM3370421", "GSM3370423", "GSM3370425", "GSM3370427", "GSM3370429", "GSM3370431", "GSM3370433", "GSM3370435", "GSM3370437", "GSM3370439", "GSM3370441", "GSM3370443", "GSM3370445", "GSM3370447", "GSM3370449", "GSM3370451", "GSM3370453", "GSM3370455", "GSM3370457", "GSM3370459", "GSM3370461", "GSM3370463", "GSM3370465", "GSM3370467", "GSM3370469", "GSM3370471", "GSM3370473", "GSM3370475", "GSM3370477", "GSM3370479", "GSM3370481", "GSM3370483", "GSM3370485", "GSM3370487", "GSM3370489", "GSM3370491", "GSM3370493", "GSM3370495", "GSM3370497", "GSM3370499", "GSM3370501", "GSM3370503", "GSM3370505", "GSM3370507", "GSM3370509", "GSM3370511", "GSM3370513", "GSM3370515", "GSM3370517", "GSM3370519", "GSM3370521", "GSM3370523", "GSM3370525", "GSM3370527", "GSM3370529", "GSM3370531", "GSM3370533", "GSM3370535", "GSM3370537", "GSM3370539", "GSM3370541", "GSM3370543", "GSM3370545", "GSM3370547", "GSM3370549", "GSM3370551", "GSM3370553", "GSM3370555", "GSM3370557", "GSM3370559", "GSM3370561", "GSM3370563", "GSM3370565", "GSM3370567", "GSM3370569", "GSM3370571", "GSM3370573", "GSM3370575", "GSM3370577", "GSM3370579", "GSM3370581", "GSM3370583", "GSM3370585", "GSM3370587", "GSM3370589", "GSM3370591", "GSM3370593", "GSM3370595", "GSM3370597", "GSM3370599", "GSM3370601", "GSM3370603", "GSM3370605", "GSM3370607", "GSM3370609", "GSM3370611", "GSM3370613", "GSM3370615", "GSM3370617", "GSM3370619", "GSM3370621", "GSM3370623", "GSM3370625", "GSM3370627", "GSM3370629", "GSM3370631", "GSM3370633", "GSM3370635", "GSM3370637", "GSM3370639", "GSM3370641", "GSM3370643", "GSM3370645", "GSM3370647", "GSM3370649", "GSM3370651", "GSM3370653", "GSM3370655", "GSM3370657", "GSM3370659", "GSM3370661", "GSM3370663", "GSM3370665", "GSM3370667", "GSM3370669", "GSM3370671", "GSM3370673", "GSM3370675", "GSM3370677", "GSM3370679", "GSM3370681", "GSM3370683", "GSM3370685", "GSM3370687", "GSM3370689", "GSM3370691", "GSM3370693", "GSM3370695", "GSM3370697", "GSM3370699", "GSM3370701", "GSM3370703", "GSM3370705", "GSM3370707", "GSM3370709", "GSM3370711", "GSM3370713", "GSM3370715", "GSM3370717", "GSM3370719", "GSM3370721", "GSM3370723", "GSM3370725", "GSM3370727", "GSM3370729", "GSM3370731", "GSM3370733", "GSM3370735", "GSM3370737", "GSM3370739", "GSM3370741", "GSM3370743", "GSM3370745", "GSM3370747", "GSM3370749", "GSM3370751", "GSM3370753", "GSM3370755", "GSM3370757", "GSM3370759", "GSM3370761", "GSM3370763", "GSM3370765", "GSM3370767", "GSM3370769", "GSM3370771", "GSM3370773", "GSM3370775", "GSM3370777", "GSM3370779", "GSM3370781", "GSM3370783", "GSM3370785", "GSM3370787", "GSM3370789", "GSM3370791", "GSM3370793", "GSM3370795", "GSM3370797", "GSM3370799", "GSM3370801", "GSM3370803", "GSM3370805", "GSM3370807", "GSM3370809", "GSM3370811", "GSM3370813", "GSM3370815", "GSM3370817", "GSM3370819",
```

```

M5418254", "GSM5418255", "GSM5418256", "GSM5418257", "GSM5418258", "GSM5418259", "GSM5418260", "
GSM5418261", "GSM5418262", "GSM5418263", "GSM5418264", "GSM5418265", "GSM5418266", "GSM5418267"
, "GSM5418268", "GSM5418269", "GSM5418270", "GSM5418271", "GSM5418272", "GSM5418273", "GSM541827
4", "GSM5418275", "GSM5418276", "GSM5418277", "GSM5418278", "GSM5418279", "GSM5418280", "GSM5418
281", "GSM5418283", "GSM5418284"),
  GSE215997 =
  c("GSM6654037", "GSM6654038", "GSM6654039", "GSM6654040", "GSM6654041", "GSM6654042", "GSM66540
43", "GSM6654044", "GSM6654045", "GSM6654046", "GSM6654047", "GSM6654048", "GSM6654049"),
  GSE244807 =
  c("GSM7829922", "GSM7829924", "GSM7829925", "GSM7829926", "GSM7829927", "GSM7829928", "GSM78299
29", "GSM7829930", "GSM7829931", "GSM7829932", "GSM7829933", "GSM7829934", "GSM7829935", "GSM782
9936", "GSM7829937", "GSM7829938", "GSM7829939", "GSM7829940", "GSM7829941", "GSM7829942", "GSM7
829943", "GSM7829944", "GSM7829945", "GSM7829947", "GSM7829948", "GSM7829949", "GSM7829950", "GS
M7829951", "GSM7829952", "GSM7829954", "GSM7829955", "GSM7829956", "GSM7829957", "GSM7829958", "
GSM7829959", "GSM7829960", "GSM7829961", "GSM7829962", "GSM7829963", "GSM7829964", "GSM7829965"
, "GSM7829966", "GSM7829967", "GSM7829968", "GSM7829969", "GSM7829970", "GSM7829971", "GSM782997
2", "GSM7829973", "GSM7829974", "GSM7829975", "GSM7829976", "GSM7829977", "GSM7829978", "GSM7829
979", "GSM7829980", "GSM7829981", "GSM7829982", "GSM7829983", "GSM7829984", "GSM7829985", "GSM78
29986", "GSM7829987", "GSM7829988", "GSM7829989", "GSM7829990", "GSM7829991", "GSM7829992", "GSM
7829993", "GSM7829994", "GSM7829995", "GSM7829996", "GSM7829997", "GSM7829998", "GSM7829999", "G
SM7830000", "GSM7830001", "GSM7830002", "GSM7830003", "GSM7830004", "GSM7830005", "GSM7830006", "
GSM7830007", "GSM7830008", "GSM7830009", "GSM7830010", "GSM7830011", "GSM7830012", "GSM7830013"
, "GSM7830014", "GSM7830015", "GSM7830016", "GSM7830017", "GSM7830018", "GSM7830019", "GSM78300
20", "GSM7830021", "GSM7830022", "GSM7830023", "GSM7830024", "GSM7830025", "GSM7830026", "GSM783
0027", "GSM7830028", "GSM7830029", "GSM7830030", "GSM7830031", "GSM7830032", "GSM7830033", "GSM7
830034", "GSM7830035", "GSM7830036", "GSM7830037", "GSM7830038", "GSM7830039", "GSM7830040", "GS
M7830041", "GSM7830042", "GSM7830043", "GSM7830044", "GSM7830045", "GSM7830046", "GSM7830047", "
GSM7830048", "GSM7830049", "GSM7830050", "GSM7830051", "GSM7830052", "GSM7830053", "GSM7830054"
, "GSM7830055", "GSM7830056", "GSM7830057", "GSM7830058", "GSM7830059", "GSM7830060", "GSM783006
1", "GSM7830062", "GSM7830063", "GSM7830064", "GSM7830065", "GSM7830066", "GSM7830067", "GSM7830
068", "GSM7830069", "GSM7830070", "GSM7830071", "GSM7830072", "GSM7830073", "GSM7830074", "GSM78
30075", "GSM7830076", "GSM7830077", "GSM7830078", "GSM7830079", "GSM7830080", "GSM7830081", "GSM
7830082", "GSM7830083", "GSM7830084", "GSM7830085", "GSM7830087", "GSM7830088", "GSM7830089", "G
SM7830090", "GSM7830091", "GSM7830092", "GSM7830093", "GSM7830094", "GSM7830095", "GSM7830096",
"GSM7830097", "GSM7830098", "GSM7830099", "GSM7830100", "GSM7830101", "GSM7830102", "GSM7830103
", "GSM7830104", "GSM7830105", "GSM7830106", "GSM7830107", "GSM7830108", "GSM7830109", "GSM78301
0", "GSM7830111", "GSM7830112", "GSM7830113", "GSM7830114", "GSM7830115", "GSM7830116", "GSM783
0117", "GSM7830118", "GSM7830119", "GSM7830120", "GSM7830121", "GSM7830122", "GSM7830123", "GSM7
830124", "GSM7830125", "GSM7830126", "GSM7830127", "GSM7830128", "GSM7830129", "GSM7830131", "GS
M7830132", "GSM7830133", "GSM7830134", "GSM7830135", "GSM7830136", "GSM7830137", "GSM7830138", "
GSM7830139", "GSM7830140", "GSM7830141", "GSM7830142", "GSM7830143", "GSM7830144", "GSM7830145"
, "GSM7830146", "GSM7830147", "GSM7830148", "GSM7830149", "GSM7830150", "GSM7830151", "GSM783015
2", "GSM7830153", "GSM7830154", "GSM7830155", "GSM7830156", "GSM7830157", "GSM7830158", "GSM7830
159", "GSM7830160", "GSM7830161", "GSM7830162", "GSM7830163", "GSM7830164", "GSM7830165", "GSM78
30166", "GSM7830167")
)

metadata_files <- c(GSE63420="GSE63420_metadata.csv", GSE119336="GSE119336_metadata.csv",
GSE107943="GSE107943_metadata.csv", GSE162396="GSE162396_metadata.csv",
GSE179443="GSE179443_metadata.csv", GSE215997="GSE215997_metadata.csv",
GSE244807="GSE244807_metadata.csv")

# -----
# 3. FUNCIÓN DE LIMPIEZA Y FILTRADO (DEFINITIVA)
# -----
clean_and_filter_metadata <- function(gse_id, file_path, gsm_ids) {
  cat(paste0("\n 🔄 Procesando ", gse_id, "... \n"))
  df <- read_csv(file_path, show_col_types = FALSE)
  if (!"geo_accession" %in% names(df)) df <- df %>% rename(geo_accession = 1)
  df_filtered <- df %>% filter(geo_accession %in% gsm_ids) %>% slice(match(gsm_ids,
geo_accession))
}

```

```

    if (nrow(df_filtered) != length(gsm_ids)) {
      warning(paste("Error en", gse_id, ": esperadas", length(gsm_ids), "obtenidas",
nrow(df_filtered)))
      return(NULL)
    }
    df_processed <- df_filtered %>% janitor::clean_names() %>%
mutate(across(starts_with("characteristics_ch1"), as.character)) %>%
      mutate(across(where(is.character) | where(is.factor), ~ na_if(., "NA") %>%
na_if("") %>% na_if("null") %>% na_if("N/A")))
    char_cols <- names(df_processed %>% select(where(is.character)))
    for (col_name in setdiff(char_cols, "geo_accession")) {
      first_val <- df_processed %>% pull(!sym(col_name)) %>% na.omit() %>%
as.character() %>% head(1)
      if (length(first_val) > 0) {
        extracted <- str_extract(first_val, "^[^:]+:")
        if (!is.na(extracted)) {
          new_name <- extracted %>% str_replace_all(":", "") %>% str_trim() %>%
janitor::make_clean_names()
          if (nchar(new_name) > 1 && new_name != col_name) {
            orig <- new_name; counter <- 2
            while (new_name %in% names(df_processed)) { new_name <- paste0(orig,
"_", counter); counter <- counter + 1 }
            df_processed <- df_processed %>% rename(!new_name :=
!!sym(col_name)) %>% mutate(!new_name := str_replace(!sym(new_name), "^[^:]+:\\s*",
""))
          }
        }
      }
    }
    df_final <- df_processed %>% rename_with(~ str_replace(., "_mo$", ""),
ends_with("_mo")) %>% rename_with(~ str_replace(., "_cm$", ""), ends_with("_cm"))
    write_csv(df_final, paste0(gse_id, "_metadata_filtered.csv"))
    cat(paste0(" Exito: ", gse_id, " | Filas: ", nrow(df_final), " | Cols: ",
ncol(df_final), "\n"))
    return(df_final)
  }

# -----
# 4. EJECUCIÓN DEL FILTRADO
# -----
metadata_filtrados <- list()
for (gse in names(muestras_interes)) {
  metadata_filtrados[[gse]] <- clean_and_filter_metadata(gse, metadata_files[[gse]],
muestras_interes[[gse]])
}
## Procesando GSE63420...
## Exito: GSE63420 | Filas: 14 | Cols: 43
## Procesando GSE119336...
## Exito: GSE119336 | Filas: 30 | Cols: 42
## Procesando GSE107943...
## Exito: GSE107943 | Filas: 56 | Cols: 70
## Procesando GSE162396...
## Exito: GSE162396 | Filas: 12 | Cols: 45
## Procesando GSE179443...
## Exito: GSE179443 | Filas: 57 | Cols: 42
## Procesando GSE215997...
## Exito: GSE215997 | Filas: 13 | Cols: 38
## Procesando GSE244807...
## Exito: GSE244807 | Filas: 241 | Cols: 46

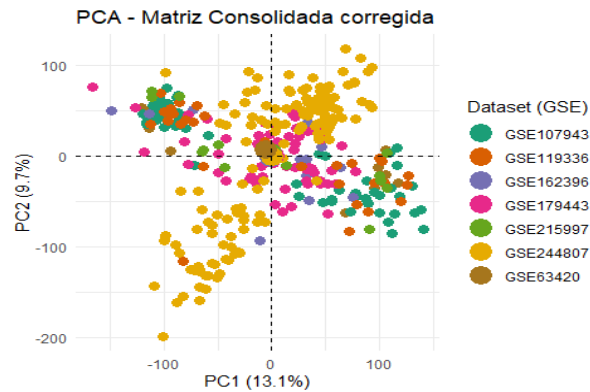
```

```
names(metadata_filtrados)
```

```
## [1] "GSE63420" "GSE119336" "GSE107943" "GSE162396" "GSE179443" "GSE215997"  
## [7] "GSE244807"
```

4. CORRECCIÓN DE EFECTO DE LOTE

```
library(sva)  
library(dplyr)  
library(stringr)  
# -----  
# 1. Crear el vector de Lotes (batch)  
# -----  
nombres_datasets <- names(integrated_list)  
batch <- factor(rep(nombres_datasets, times = sapply(integrated_list, ncol)))  
  
cat(paste("Longitud vector batch:", length(batch), "\n"))  
## Longitud vector batch: 324  
  
cat(paste("Muestras en matriz_con_batch:", ncol(matriz_con_batch), "\n"))  
## Muestras en matriz_con_batch: 324  
  
# --- Paso Crítico: Aplicar ComBat ---  
matriz_corregida_ComBat <- sva::ComBat(dat = matriz_con_batch, batch = batch, par.prior =  
TRUE)  
  
cat(paste("\nDimensiones matriz corregida:", paste(dim(matriz_corregida_ComBat), collapse  
= " x "), "\n"))  
  
## Dimensiones matriz corregida: 24141 x 324  
  
# -----  
# 4. Asegurar Nombres y Guardar  
# -----  
colnames(matriz_corregida_ComBat) <- colnames(matriz_con_batch)  
rownames(matriz_corregida_ComBat) <- rownames(matriz_con_batch)  
  
output_dir <- "Datos_Integrados"  
if (!dir.exists(output_dir)) dir.create(output_dir)  
  
output_filepath <- file.path(output_dir, "matriz_corregida_ComBat.csv")  
write.csv(matriz_corregida_ComBat, file = output_filepath, row.names = TRUE)  
  
cat(paste0("\nÉxito: Matriz guardada en: '", output_filepath, "'\n"))  
  
## Éxito: Matriz guardada en: 'Datos_Integrados/matriz_corregida_ComBat.csv'  
  
# -----  
# 5. VALIDACIÓN POST-CORRECCIÓN (PCA)  
# -----  
  
run_pca_and_plot(matriz_corregida_ComBat, pheno, "Matriz Consolidada corregida")
```



5. IDENTIFICACIÓN DE MUESTRAS TUMORALES VS NO TUMORALES

```
# -----
# 1. Definición de Las muestras por dataset (Validado y Corregido)
# -----

# --- Datasets con dos condiciones (T vs NT) ---
GSE63420_groups <- list(
  NT =
c("GSM1548453", "GSM1548454", "GSM1548455", "GSM1548456", "GSM1548457", "GSM1548458", "GSM1548459"),
  T =
c("GSM1548460", "GSM1548461", "GSM1548462", "GSM1548463", "GSM1548464", "GSM1548465", "GSM1548466")
)

GSE119336_groups <- list(
  NT = c("GSM3370133", "GSM3370135", "GSM3370137", "GSM3370139", "GSM3370141",
"GSM3370143", "GSM3370145", "GSM3370147", "GSM3370149", "GSM3370151", "GSM3370153",
"GSM3370155", "GSM3370157", "GSM3370159", "GSM3370161"),
  T = c("GSM3370134", "GSM3370136", "GSM3370138", "GSM3370140", "GSM3370142",
"GSM3370144", "GSM3370146", "GSM3370148", "GSM3370150", "GSM3370152", "GSM3370154",
"GSM3370156", "GSM3370158", "GSM3370160", "GSM3370162")
)

GSE107943_groups <- list(
  NT =
c("GSM2883958", "GSM2883959", "GSM2883960", "GSM2883961", "GSM2883962", "GSM2883963", "GSM2883964",
"GSM2883965", "GSM2883966", "GSM2883967", "GSM2883968", "GSM2883969", "GSM2883970", "GSM2883971",
"GSM2883972", "GSM2883973", "GSM2883974", "GSM2883975", "GSM2883976", "GSM2883977",
"GSM2883978", "GSM2883979", "GSM2883980", "GSM2883981", "GSM2883982", "GSM2883983",
"GSM2883984"),
  T =
c("GSM2883928", "GSM2883929", "GSM2883930", "GSM2883931", "GSM2883932", "GSM2883933", "GSM2883934",
"GSM2883935", "GSM2883936", "GSM2883937", "GSM2883938", "GSM2883939", "GSM2883940", "GSM2883941",
"GSM2883942", "GSM2883943", "GSM2883944", "GSM2883945", "GSM2883946", "GSM2883947", "GSM2883948",
"GSM2883949", "GSM2883950", "GSM2883951", "GSM2883952", "GSM2883953", "GSM2883954", "GSM2883955",
"GSM2883956")
)

# --- Datasets que se asumen como solo Tumorales (T) ---
datasets_solo_T <- list(
  GSE162396 = c("GSM4950980", "GSM4950981", "GSM4950982", "GSM4950983", "GSM4950984",
"GSM4950985", "GSM4950986", "GSM4950987", "GSM4950988", "GSM4950989", "GSM4950990",
"GSM4950991"),

```



```

# -----
# 2. ALINEACIÓN Y MAPEO DE MUESTRAS (TUMORAL vs NO_TUMORAL)
# -----
suppressPackageStartupMessages({ library(tidyverse) })

# 2. Consolidación de Grupos
ids_NT <- c(GSE63420_groups$NT, GSE119336_groups$NT, GSE107943_groups$NT)
ids_T_con_control <- c(GSE63420_groups$T, GSE119336_groups$T, GSE107943_groups$T)
ids_T_solo_T <- unlist(datasets_solo_T)
ids_T <- c(ids_T_con_control, ids_T_solo_T)

# 2.3. Tabla maestra
metadata_completo <- data.frame(
  geo_accession = c(ids_NT, ids_T),
  Condition = c(rep("No_Tumoral", length(ids_NT)), rep("Tumoral", length(ids_T))),
  stringsAsFactors = FALSE
)

# 3. Alineación
muestras_en_matriz <- colnames(matriz_corregida_ComBat)
datos_fenotipicos <- metadata_completo %>%
  filter(geo_accession %in% muestras_en_matriz) %>%
  slice(match(muestras_en_matriz, geo_accession)) %>%
  mutate(Condition = factor(Condition, levels = c("No_Tumoral", "Tumoral")))

rownames(datos_fenotipicos) <- datos_fenotipicos$geo_accession

# 4. Verificación

print(table(datos_fenotipicos$Condition))

##
## No_Tumoral    Tumoral
##          49         275

# =====
# VISUALIZACIÓN DE LA DISTRIBUCIÓN
# =====
library(ggplot2)

datos_muestras <- data.frame(
  Cohorte_GSE = factor(c(rep("GSE107943", 1), rep("GSE119336", 1), rep("GSE63420", 1),
    rep("GSE107943", 1), rep("GSE119336", 1), rep("GSE63420", 1),
    rep("GSE162396", 1), rep("GSE179443", 1), rep("GSE215997", 1),
    rep("GSE244807", 1)),
    levels = c("GSE63420", "GSE107943", "GSE119336", "GSE162396",
"GSE179443", "GSE215997", "GSE244807")),
  Condicion_Biologica = factor(c(rep("No Tumoral (NT)", 3), rep("Tumoral (T)", 7)),
    levels = c("No Tumoral (NT)", "Tumoral (T)")),
  N_Muestras = c(27, 15, 7, 29, 15, 7, 12, 57, 13, 241)
)

colores_gse <- c("GSE107943" = "#FF6347", "GSE119336" = "#FFA07A", "GSE162396" =
"#008000",
  "GSE179443" = "#3CB371", "GSE215997" = "#1E90FF", "GSE244807" =
"#9370DB", "GSE63420" = "#FFC0CB")

ggplot(datos_muestras, aes(x = Condicion_Biologica, y = N_Muestras, fill = Cohorte_GSE))
+
  geom_bar(stat = "identity") +

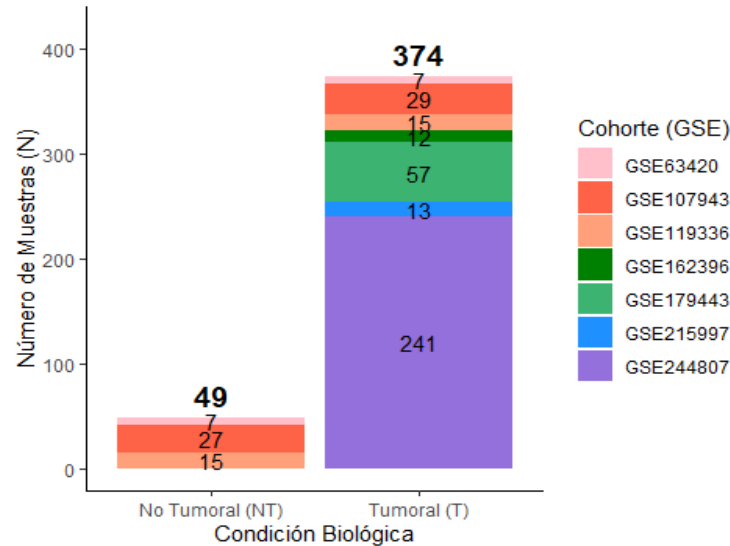
```



```

geom_text(aes(label = N_Muestras), position = position_stack(vjust = 0.5), size = 4) +
geom_text(data = datos_muestras %>% group_by(Condicion_Biologica) %>% summarise(Total =
sum(N_Muestras)),
aes(x = Condicion_Biologica, y = Total, label = Total, fill = NULL), vjust =
-0.5, size = 5, fontface = "bold") +
scale_fill_manual(values = colores_gse) +
scale_y_continuous(limits = c(0, 420)) +
labs(y = "Número de Muestras (N)", x = "Condición Biológica", fill = "Cohorte (GSE)") +
theme_classic()

```



6. ANÁLISIS DE EXPRESIÓN GÉNICA DIFERENCIAL MEDIANTE LIMMA + VOOM

```

exp_file <- "matriz_corregida_ComBat.csv"
if (file.exists(exp_file)) {
  # Cargar la matriz de expresión
  matriz_corregida_ComBat <- read.csv(exp_file, row.names = 1, check.names = FALSE)
  cat(paste0("Matriz 'matriz_corregida_ComBat' cargada con ",
nrow(matriz_corregida_ComBat), " genes y ", ncol(matriz_corregida_ComBat), "
muestras.\n"))
} else {
  stop(paste0("ERROR CRÍTICO: No se encuentra el fichero de expresión requerido: '",
exp_file, "'. Asegúrate de que está en el directorio de trabajo.))
}

## Matriz 'matriz_corregida_ComBat' cargada con 24144 genes y 423 muestras.

# =====
# 1. INSTALACIÓN Y CARGA DE LIBRERÍAS
# =====
if (!requireNamespace("BiocManager", quietly = TRUE)) {
  install.packages("BiocManager")
}
if (!requireNamespace("limma", quietly = TRUE)) {
  BiocManager::install("limma")
}
if (!requireNamespace("EnhancedVolcano", quietly = TRUE)) {
  BiocManager::install("EnhancedVolcano")
}
if (!requireNamespace("tidyverse", quietly = TRUE)) {
  install.packages("tidyverse")
}

```

```

if (!requireNamespace("pheatmap", quietly = TRUE)) {
  install.packages("pheatmap")
}
# LIBRERÍAS DE SUPERVIVENCIA
if (!requireNamespace("survival", quietly = TRUE)) {
  install.packages("survival")
}
if (!requireNamespace("survminer", quietly = TRUE)) {
  install.packages("survminer")
}

library(limma)
library(EnhancedVolcano)
library(tidyverse)
library(pheatmap)
library(survival)
library(survminer)
library(pheatmap)

# =====
# 1.5. Definición de Las muestras por dataset
# =====
# T: Tumoral, NT: No Tumoral

GSE63420_groups <- list(
  NT =
c("GSM1548453", "GSM1548454", "GSM1548455", "GSM1548456", "GSM1548457", "GSM1548458", "GSM1548459"),
  T =
c("GSM1548460", "GSM1548461", "GSM1548462", "GSM1548463", "GSM1548464", "GSM1548465", "GSM1548466")
)
GSE119336_groups <- list(
  NT = c("GSM3370133", "GSM3370135", "GSM3370137", "GSM3370139", "GSM3370141",
"GSM3370143", "GSM3370145", "GSM3370147", "GSM3370149", "GSM3370151", "GSM3370153",
"GSM3370155", "GSM3370157", "GSM3370159", "GSM3370161"),
  T = c("GSM3370134", "GSM3370136", "GSM3370138", "GSM3370140", "GSM3370142",
"GSM3370144", "GSM3370146", "GSM3370148", "GSM3370150", "GSM3370152", "GSM3370154",
"GSM3370156", "GSM3370158", "GSM3370160", "GSM3370162")
)
GSE107943_groups <- list(
  NT =
c("GSM2883958", "GSM2883959", "GSM2883960", "GSM2883961", "GSM2883962", "GSM2883963", "GSM2883964",
"GSM2883965", "GSM2883966", "GSM2883967", "GSM2883968", "GSM2883969", "GSM2883970", "GSM2883971",
"GSM2883972", "GSM2883973", "GSM2883974", "GSM2883975", "GSM2883976", "GSM2883977",
"GSM2883978", "GSM2883979", "GSM2883980", "GSM2883981", "GSM2883982", "GSM2883983",
"GSM2883984"),
  T =
c("GSM2883928", "GSM2883929", "GSM2883930", "GSM2883931", "GSM2883932", "GSM2883933", "GSM2883934",
"GSM2883935", "GSM2883936", "GSM2883937", "GSM2883938", "GSM2883939", "GSM2883940", "GSM2883941",
"GSM2883942", "GSM2883943", "GSM2883944", "GSM2883945", "GSM2883946", "GSM2883947", "GSM2883948",
"GSM2883949", "GSM2883950", "GSM2883951", "GSM2883952", "GSM2883953", "GSM2883954", "GSM2883955",
"GSM2883956")
)

```

```

# =====
# 2. PROCESAMIENTO Y ALINEACIÓN DE METADATOS (DGE)
# =====

# 2.1. Consolidación de grupos
muestras_interes <- list(
  NT = c(GSE63420_groups$NT, GSE119336_groups$NT, GSE107943_groups$NT),
  T = c(GSE63420_groups$T, GSE119336_groups$T, GSE107943_groups$T)
)

ids_NT <- muestras_interes$NT
ids_T <- muestras_interes$T

# 2.2. Crear la tabla maestra de metadatos (GEO Accession vs Condición)
metadata_completo <- data.frame(
  geo_accession = c(ids_NT, ids_T),
  Condicion = c(rep("No_Tumoral", length(ids_NT)),
                rep("Tumoral", length(ids_T))),
  stringsAsFactors = FALSE
)

# 2.3. ALINEACIÓN CRÍTICA CON LA MATRIZ CORREGIDA
muestras_seleccionadas <- metadata_completo$geo_accession

# Filtrar la matriz de expresión para incluir solo las muestras seleccionadas
data_matrix <- as.matrix(matriz_corregida_ComBat[, muestras_seleccionadas, drop = FALSE])

# Obtener los nombres de las muestras que realmente están en la matriz filtrada
muestras_en_matriz <- colnames(data_matrix)

# 2.3a. Filtrar el metadata para mantener solo las muestras de la matriz
datos_fenotipicos <- metadata_completo %>%
  filter(geo_accession %in% muestras_en_matriz)

# 2.3b. REORDENAR CRÍTICAMENTE: Alinear las filas del metadata al orden EXACTO de las
# columnas de la matriz
datos_fenotipicos <- datos_fenotipicos %>%
  slice(match(muestras_en_matriz, geo_accession)) %>%
  # Convertir 'Condicion' a factor con el nivel de referencia (No_Tumoral) primero
  mutate(Condicion = factor(Condicion, levels = c("No_Tumoral", "Tumoral")))

# 2.3c. Asignar los nombres de fila (muestras)
rownames(datos_fenotipicos) <- datos_fenotipicos$geo_accession

# 2.4. RESULTADO Y VERIFICACIÓN FINAL
num_muestras_final <- ncol(data_matrix)
cat(paste("Número total de muestras en la matriz corregida (SOLO T vs NT):",
  num_muestras_final,
  " muestras.\n"))

## Número total de muestras en la matriz corregida (SOLO T vs NT): 100 muestras.

print(table(datos_fenotipicos$Condicion))

##
## No_Tumoral    Tumoral
##          49          51

```

```

# =====
# 3. y 4. DEFINICIÓN DEL DISEÑO Y MODELO
# =====

diseño <- model.matrix(~Condicion, data=datos_fenotipicos)
cat("\nDiseño del modelo:\n")

# =====
# 5. AJUSTAR MODELO LINEAL CON LIMMA
# =====

fit <- lmFit(data_matrix, diseño)
fit <- eBayes(fit)

# =====
# 6. EXTRAER RESULTADOS DE EXPRESIÓN DIFERENCIAL
# =====

padj_cutoff <- 0.05
logFC_cutoff <- 1

results_all <- topTable(fit, coef="CondicionTumoral", number=Inf, adjust.method="fdr")

DGE_genes <- results_all %>%
  filter(adj.P.Val < padj_cutoff, abs(logFC) > logFC_cutoff)

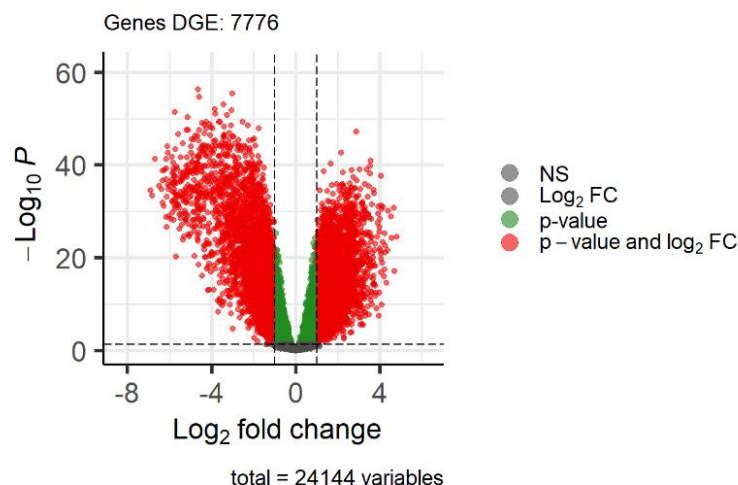
cat(paste0("\nSe encontraron ", nrow(DGE_genes), " genes diferencialmente expresados
(DGE) después de la corrección de ComBat.\n"))
## Se encontraron 7776 genes diferencialmente expresados (DGE) después de la corrección
de ComBat.

# =====
# 7. GENERAR VOLCANO PLOT
# =====

if (exists("results_all") && is.data.frame(results_all) && nrow(results_all) > 0) {
  volcano_plot <- EnhancedVolcano(
    results_all,
    lab = rownames(results_all),
    x = 'logFC',
    y = 'P.Value',
    title = 'Volcano Plot: Tumoral vs. No Tumoral - Cohortes con Control',
    subtitle = paste0('Genes DGE: ', nrow(DGE_genes)),
    pCutoff = padj_cutoff,
    FCcutoff = logFC_cutoff,
    pointSize = 1.5,
    labSize = 0.0,
    col = c('grey30', 'grey30', 'forestgreen', 'red2'),
    colAlpha = 0.6,
    legendPosition = 'right',
    selectLab = character(0) # Ocultar etiquetas de genes
  )
  print(volcano_plot)
  cat("\n Volcano Plot generado SIN etiquetas de genes.\n")
}

```

Volcano Plot: Tumoral vs. No Tumoral - Cohortes



Volcano Plot generado SIN etiquetas de genes.

```
# =====
# 8. SEPARAR Y GUARDAR GENES SOBREEXPRESADOS E INFRAEXPRESADOS
# =====
if (exists("DGE_genes") && is.data.frame(DGE_genes) && nrow(DGE_genes) > 0) {
  genes_sobreexpresados <- DGE_genes %>%
    filter(logFC > 0) %>%
    arrange(desc(logFC))

  genes_infraexpresados <- DGE_genes %>%
    filter(logFC < 0) %>%
    arrange(logFC)

  write.csv(genes_sobreexpresados, "Genes_Sobreexpresados_T_vs_NT.csv", row.names =
TRUE)
  write.csv(genes_infraexpresados, "Genes_Infraexpresados_T_vs_NT.csv", row.names =
TRUE)

  cat(paste0("\nResumen de genes DGE T vs NT:\n"))
  cat(paste0("  Genes sobreexpresados: ", nrow(genes_sobreexpresados), "\n"))
  cat(paste0("  Genes infraexpresados: ", nrow(genes_infraexpresados), "\n"))
} else {
  cat("\nNo se encontraron genes diferencialmente expresados. Omitiendo guardar
CSVs.\n")
  # Crear dataframes vacíos si no hay DGE para evitar errores en la siguiente sección
  genes_sobreexpresados <- data.frame(logFC=numeric(), adj.P.Val=numeric())
  genes_infraexpresados <- data.frame(logFC=numeric(), adj.P.Val=numeric())
}

##
## Resumen de genes DGE T vs NT:
## Genes sobreexpresados: 4339
## Genes infraexpresados: 3437
```

7. ANÁLISIS UNIVARIANTE MEDIANTE EL MODELO DE COX- TODOS LOS GENES DE LA FIRMA DE TGF-B

```
# =====
# 1. CARGA DE LIBRERÍAS NECESARIAS
# =====
if (!requireNamespace("tidyverse", quietly = TRUE)) install.packages("tidyverse")
if (!requireNamespace("survival", quietly = TRUE)) install.packages("survival")
if (!requireNamespace("survminer", quietly = TRUE)) install.packages("survminer")
library(tidyverse)
library(survival)
library(survminer)

# =====
# 2. DEFINICIÓN DE MUESTRAS TUMORALES Y FIRMA GÉNICA
# =====

# --- 2.1 Muestras Tumorales (Cohorte Consolidada) ---
muestras_tumorales <- c(

"GSM1548460", "GSM1548461", "GSM1548462", "GSM1548463", "GSM1548464", "GSM1548465", "GSM1548466",
",
"GSM3370134", "GSM3370136", "GSM3370138", "GSM3370140", "GSM3370142", "GSM3370144",
"GSM3370146", "GSM3370148", "GSM3370150", "GSM3370152", "GSM3370154", "GSM3370156",
"GSM3370158", "GSM3370160", "GSM3370162",

"GSM2883928", "GSM2883929", "GSM2883930", "GSM2883931", "GSM2883932", "GSM2883933", "GSM2883934",
"GSM2883935", "GSM2883936", "GSM2883937", "GSM2883938", "GSM2883939", "GSM2883940", "GSM2883941",
"GSM2883942", "GSM2883943", "GSM2883944", "GSM2883945", "GSM2883946", "GSM2883947", "GSM2883948",
"GSM2883949", "GSM2883950", "GSM2883951", "GSM2883952", "GSM2883953", "GSM2883954", "GSM2883955",
"GSM2883956",
"GSM4950980", "GSM4950981", "GSM4950982", "GSM4950983", "GSM4950984", "GSM4950985",
"GSM4950986", "GSM4950987", "GSM4950988", "GSM4950989", "GSM4950990", "GSM4950991",
"GSM5418226", "GSM5418227", "GSM5418228", "GSM5418229", "GSM5418230", "GSM5418231",
"GSM5418232", "GSM5418233", "GSM5418234", "GSM5418235", "GSM5418236", "GSM5418237",
"GSM5418238", "GSM5418239", "GSM5418240", "GSM5418241", "GSM5418242", "GSM5418243",
"GSM5418244", "GSM5418245", "GSM5418246", "GSM5418247", "GSM5418248", "GSM5418249", "GSM5418250",
"GSM5418251", "GSM5418252", "GSM5418253", "GSM5418254", "GSM5418255", "GSM5418256",
"GSM5418257", "GSM5418258", "GSM5418259", "GSM5418260", "GSM5418261", "GSM5418262",
"GSM5418263", "GSM5418264", "GSM5418265", "GSM5418266", "GSM5418267", "GSM5418268",
"GSM5418269", "GSM5418270", "GSM5418271", "GSM5418272", "GSM5418273", "GSM5418274",
"GSM5418275", "GSM5418276", "GSM5418277", "GSM5418278", "GSM5418279", "GSM5418280",
"GSM5418281", "GSM5418283", "GSM5418284",
"GSM6654037", "GSM6654038", "GSM6654039", "GSM6654040", "GSM6654041", "GSM6654042",
"GSM6654043", "GSM6654044", "GSM6654045", "GSM6654046", "GSM6654047", "GSM6654048",
"GSM6654049",
"GSM7829922", "GSM7829924", "GSM7829925", "GSM7829926", "GSM7829927", "GSM7829928",
"GSM7829929", "GSM7829930", "GSM7829931", "GSM7829932", "GSM7829933", "GSM7829934",
"GSM7829935", "GSM7829936", "GSM7829937", "GSM7829938", "GSM7829939", "GSM7829940",
"GSM7829941", "GSM7829942", "GSM7829943", "GSM7829944", "GSM7829945", "GSM7829947",
"GSM7829948", "GSM7829949", "GSM7829950", "GSM7829951", "GSM7829952", "GSM7829954",
"GSM7829955", "GSM7829956", "GSM7829957", "GSM7829958", "GSM7829959", "GSM7829960",
"GSM7829961", "GSM7829962", "GSM7829963", "GSM7829964", "GSM7829965", "GSM7829966",
"GSM7829967", "GSM7829968", "GSM7829969", "GSM7829970", "GSM7829971", "GSM7829972",
"GSM7829973", "GSM7829974", "GSM7829975", "GSM7829976", "GSM7829977", "GSM7829978",
"GSM7829979", "GSM7829980", "GSM7829981", "GSM7829982", "GSM7829983", "GSM7829984",
"GSM7829985", "GSM7829986", "GSM7829987", "GSM7829988", "GSM7829989", "GSM7829990",
"GSM7829991", "GSM7829992", "GSM7829993", "GSM7829994", "GSM7829995", "GSM7829996",
"GSM7829997", "GSM7829998", "GSM7829999", "GSM7830000", "GSM7830001", "GSM7830002",
"GSM7830003", "GSM7830004", "GSM7830005", "GSM7830006", "GSM7830007", "GSM7830008",
"GSM7830009", "GSM7830010", "GSM7830011", "GSM7830012", "GSM7830013", "GSM7830014",
```

```

"GSM7830015", "GSM7830016", "GSM7830017", "GSM7830018", "GSM7830019", "GSM7830020",
"GSM7830021", "GSM7830022", "GSM7830023", "GSM7830024", "GSM7830025", "GSM7830026",
"GSM7830027", "GSM7830028", "GSM7830029", "GSM7830030", "GSM7830031", "GSM7830032",
"GSM7830033", "GSM7830034", "GSM7830035", "GSM7830036", "GSM7830037", "GSM7830038",
"GSM7830039", "GSM7830040", "GSM7830041", "GSM7830042", "GSM7830043", "GSM7830044",
"GSM7830045", "GSM7830046", "GSM7830047", "GSM7830048", "GSM7830049", "GSM7830050",
"GSM7830051", "GSM7830052", "GSM7830053", "GSM7830054", "GSM7830055", "GSM7830056",
"GSM7830057", "GSM7830058", "GSM7830059", "GSM7830060", "GSM7830061", "GSM7830062",
"GSM7830063", "GSM7830064", "GSM7830065", "GSM7830066", "GSM7830067", "GSM7830068",
"GSM7830069", "GSM7830070", "GSM7830071", "GSM7830072", "GSM7830073", "GSM7830074",
"GSM7830075", "GSM7830076", "GSM7830077", "GSM7830078", "GSM7830079", "GSM7830080",
"GSM7830081", "GSM7830082", "GSM7830083", "GSM7830084", "GSM7830085", "GSM7830087",
"GSM7830088", "GSM7830089", "GSM7830090", "GSM7830091", "GSM7830092", "GSM7830093",
"GSM7830094", "GSM7830095", "GSM7830096", "GSM7830097", "GSM7830098", "GSM7830099",
"GSM7830100", "GSM7830101", "GSM7830102", "GSM7830103", "GSM7830104", "GSM7830105",
"GSM7830106", "GSM7830107", "GSM7830108", "GSM7830109", "GSM7830110", "GSM7830111",
"GSM7830112", "GSM7830113", "GSM7830114", "GSM7830115", "GSM7830116", "GSM7830117",
"GSM7830118", "GSM7830119", "GSM7830120", "GSM7830121", "GSM7830122", "GSM7830123",
"GSM7830124", "GSM7830125", "GSM7830126", "GSM7830127", "GSM7830128", "GSM7830129",
"GSM7830131", "GSM7830132", "GSM7830133", "GSM7830134", "GSM7830135", "GSM7830136",
"GSM7830137", "GSM7830138", "GSM7830139", "GSM7830140", "GSM7830141", "GSM7830142",
"GSM7830143", "GSM7830144", "GSM7830145", "GSM7830146", "GSM7830147", "GSM7830148",
"GSM7830149", "GSM7830150", "GSM7830151", "GSM7830152", "GSM7830153", "GSM7830154",
"GSM7830155", "GSM7830156", "GSM7830157", "GSM7830158", "GSM7830159", "GSM7830160",
"GSM7830161", "GSM7830162", "GSM7830163", "GSM7830164", "GSM7830165", "GSM7830166",
"GSM7830167"
)
cat("Muestras GSE244807 para análisis de supervivencia:", length(muestras_tumorales),
"\n")

```

Muestras GSE244807 para análisis de supervivencia: 374

```

# --- 2.2 Muestras de Interés para Supervivencia (GSE244807) ---
GSE244807_samples <- c("GSM7829922", "GSM7829924", "GSM7829925", "GSM7829926",
"GSM7829927", "GSM7829928", "GSM7829929", "GSM7829930", "GSM7829931", "GSM7829932",
"GSM7829933", "GSM7829934", "GSM7829935", "GSM7829936", "GSM7829937", "GSM7829938",
"GSM7829939", "GSM7829940", "GSM7829941", "GSM7829942", "GSM7829943", "GSM7829944",
"GSM7829945", "GSM7829947", "GSM7829948", "GSM7829949", "GSM7829950", "GSM7829951",
"GSM7829952", "GSM7829954", "GSM7829955", "GSM7829956", "GSM7829957", "GSM7829958",
"GSM7829959", "GSM7829960", "GSM7829961", "GSM7829962", "GSM7829963", "GSM7829964",
"GSM7829965", "GSM7829966", "GSM7829967", "GSM7829968", "GSM7829969", "GSM7829970",
"GSM7829971", "GSM7829972", "GSM7829973", "GSM7829974", "GSM7829975", "GSM7829976",
"GSM7829977", "GSM7829978", "GSM7829979", "GSM7829980", "GSM7829981", "GSM7829982",
"GSM7829983", "GSM7829984", "GSM7829985", "GSM7829986", "GSM7829987", "GSM7829988",
"GSM7829989", "GSM7829990", "GSM7829991", "GSM7829992", "GSM7829993", "GSM7829994",
"GSM7829995", "GSM7829996", "GSM7829997", "GSM7829998", "GSM7829999", "GSM7830000",
"GSM7830001", "GSM7830002", "GSM7830003", "GSM7830004", "GSM7830005", "GSM7830006",
"GSM7830007", "GSM7830008", "GSM7830009", "GSM7830010", "GSM7830011", "GSM7830012",
"GSM7830013", "GSM7830014", "GSM7830015", "GSM7830016", "GSM7830017", "GSM7830018",
"GSM7830019", "GSM7830020", "GSM7830021", "GSM7830022", "GSM7830023", "GSM7830024",
"GSM7830025", "GSM7830026", "GSM7830027", "GSM7830028", "GSM7830029", "GSM7830030",
"GSM7830031", "GSM7830032", "GSM7830033", "GSM7830034", "GSM7830035", "GSM7830036",
"GSM7830037", "GSM7830038", "GSM7830039", "GSM7830040", "GSM7830041", "GSM7830042",
"GSM7830043", "GSM7830044", "GSM7830045", "GSM7830046", "GSM7830047", "GSM7830048",
"GSM7830049", "GSM7830050", "GSM7830051", "GSM7830052", "GSM7830053", "GSM7830054",
"GSM7830055", "GSM7830056", "GSM7830057", "GSM7830058", "GSM7830059", "GSM7830060",
"GSM7830061", "GSM7830062", "GSM7830063", "GSM7830064", "GSM7830065", "GSM7830066",
"GSM7830067", "GSM7830068", "GSM7830069", "GSM7830070", "GSM7830071", "GSM7830072",
"GSM7830073", "GSM7830074", "GSM7830075", "GSM7830076", "GSM7830077", "GSM7830078",
"GSM7830079", "GSM7830080", "GSM7830081", "GSM7830082", "GSM7830083", "GSM7830084",
"GSM7830085", "GSM7830087", "GSM7830088", "GSM7830089", "GSM7830090", "GSM7830091",

```



```

"GSM7830092", "GSM7830093", "GSM7830094", "GSM7830095", "GSM7830096", "GSM7830097",
"GSM7830098", "GSM7830099", "GSM7830100", "GSM7830101", "GSM7830102", "GSM7830103",
"GSM7830104", "GSM7830105", "GSM7830106", "GSM7830107", "GSM7830108", "GSM7830109",
"GSM7830110", "GSM7830111", "GSM7830112", "GSM7830113", "GSM7830114", "GSM7830115",
"GSM7830116", "GSM7830117", "GSM7830118", "GSM7830119", "GSM7830120", "GSM7830121",
"GSM7830122", "GSM7830123", "GSM7830124", "GSM7830125", "GSM7830126", "GSM7830127",
"GSM7830128", "GSM7830129", "GSM7830131", "GSM7830132", "GSM7830133", "GSM7830134",
"GSM7830135", "GSM7830136", "GSM7830137", "GSM7830138", "GSM7830139", "GSM7830140",
"GSM7830141", "GSM7830142", "GSM7830143", "GSM7830144", "GSM7830145", "GSM7830146",
"GSM7830147", "GSM7830148", "GSM7830149", "GSM7830150", "GSM7830151", "GSM7830152",
"GSM7830153", "GSM7830154", "GSM7830155", "GSM7830156", "GSM7830157", "GSM7830158",
"GSM7830159", "GSM7830160", "GSM7830161", "GSM7830162", "GSM7830163", "GSM7830164",
"GSM7830165", "GSM7830166", "GSM7830167")
cat("Muestras GSE244807 para análisis de supervivencia:", length(GSE244807_samples),
"\n")

## Muestras GSE244807 para análisis de supervivencia: 241

# --- 2.3 Definición de la Firma Génica TGF-β Pro-Tumorigénica ---
genes_sobreexp <-
c("PMEPA1", "BHLHE40", "ADAM19", "SERPINE1", "EPHB2", "TGFB1", "JUNB", "LTBP2", "SEMA7A", "COL7A1",
"AMIGO2", "SH3KBP1", "SMAD7", "SPHK1", "NUAK1", "LIPG", "VDR", "ZSWIM4", "SKIL", "MLLT11", "DCBLD1",
"FOXP1", "COL1A1", "PDLIM7", "PNMA1", "ITGA5", "CDH2", "NAV1", "SLC19A2", "DSE", "SNAI1", "LPXN",
"CSRP1", "PORCN", "SGK1", "TUFT1", "ANKLE2", "ZNF697", "BMP1", "MYORG", "PTPRK", "ENSG00000290569",
"GRB10", "ENSG00000274265", "MLXIP", "PODXL", "TPM1", "IL11", "TMCC1", "PID1", "NEDD9", "COL16A1",
"ENSG00000275993", "PCDH1", "MMP14", "GALNT10", "DLC1", "CHST11", "NOL4L", "PTPRE", "ENSG00000285517",
"CCN2", "CLTCL1", "SPSB1", "GADD45B", "MIR193BHG", "PHLDB1", "ITGB5", "SPECC1", "MICAL2", "ETV6",
"SIK1", "SIK1", "RALA", "VEGFA", "FHL3", "PPARD", "NT5DC2", "MARCKSL1", "PFKFB4", "ITGA2", "HIVEP1",
"INHBA", "ADAMTS6", "SRRD", "PACS1", "MT1X", "CDC42SE1", "TSPAN14", "PGRMC2", "MSANTD3", "PSMD2", "POMT2",
"LIIF", "ANKRD37", "SLC29A1", "PLEK2", "HAPLN3", "ADAM12", "ABCA1", "FSTL3", "MRC2", "UBASH3B",
"PLPP4", "IGF1R", "HHAT", "GLI2", "LIMS1", "SGPL1", "METRNL", "IFFO2", "MRAS", "KDM5B", "MARCHF4",
"KRT7", "KCTD11", "P4HA2", "HSF2BP", "DYSF", "THBS1", "ENSG00000279821", "SERPINE2", "IGSF3", "NKG2A",
"BPAGM", "LMCD1", "CSF1R", "SPOCD1", "KLF7", "SLC26A2", "ULK1", "SMOX", "MANF", "EDN1", "FAM214B",
"MIR22HG", "RNF121", "PXD1", "B4GALT1", "NDST1", "TARS3", "VMP1", "ABLIM3", "CLN8", "CGB8", "SOX4", "PPP1R14B",
"OCIAD2", "FBLIM1", "ENSG00000270504", "PPP1R3B", "KIRREL1", "ANXA8L1", "SSR3", "PFKFB3", "COL4A2",
"COL6A3", "UCK2", "NET1", "ARHGEF40", "ZC3H12A", "DT", "LFNG", "SLC35F2", "ARNTL", "DNAJB5", "INAVA", "MBOAT2")

genes_infraexp <- c(
"CITED2", "SPRY1", "PHLPP1", "TBC1D8", "HPSE", "ZNF395", "TGFB3", "MAP3K5", "ARRB2",
"BMP4", "DUSP4", "KCNAB2", "DUSP16", "ANK1", "IL6R", "TRERF1", "GPAT3", "MBP", "KLHL4",
"RIN1", "IFIT3", "THRB", "AVPI1", "PLPP3", "CNKSR3", "MAPRE2", "CCN3", "Clorf226",
"OAS3", "AK5", "TFPI", "PHF19", "ASIC1", "TSPAN4", "JADE2", "ZFP36L2", "MGAT5", "SAMHD1",
"ALDH3A2", "CAT", "GSR", "KITLG", "RBM47", "ATP8B1", "CABLES1", "PEG10", "CDKN2C", "DBP",
"IFIT1", "CTNNA1", "BCAM", "NCEH1", "ADI1", "VSIR", "SSBP2", "ARVCF", "SLITRK5",
"GPRC5A", "NMT2", "AGFG2", "FOSL1", "SH2D5", "TM4SF1", "AOX1", "PPL", "APLN", "MYPN",
"ANKRD33B", "ADORA2B", "CAMK2D", "PARP10", "KIAA1217", "ALDH3B1", "MCCC1", "EPB41L4B",
"BTN3A1", "MAP2", "BTN3A2", "TJP2", "PRECSIT", "TMC04", "TXNRD1", "CYB5A", "LPAR1",
"LRRC20", "KHK", "MYOF", "ZFP36", "SECTM1", "CD14", "SYNE3", "TRNP1", "CXCL16", "CREB5",
"NXN", "DENND3", "CFB", "DEPP1", "FLVCR2", "LIMD1", "PLK2", "MITF", "CEBPD", "PHLDA1",
"TXNIP", "PRICKLE1", "ERF1", "CTDSPL", "MYO18A")
)
genes_firma_unicos <- unique(c(genes_sobreexp, genes_infraexp))
cat("Número de genes sobreexpresados: ", length(genes_sobreexp), "\n")

## Número de genes sobreexpresados: 166

cat("Número de genes infraexpresados:", length(genes_infraexp), "\n")

## Número de genes infraexpresados: 109

```



```

# =====
# 3. CARGA Y PREPARACIÓN DE LA MATRIZ DE EXPRESIÓN
# =====

exp_file <- "matriz_corregida_ComBat.csv"
if (!file.exists(exp_file)) stop("ERROR: No se encuentra el fichero de expresión
'matriz_corregida_ComBat.csv'.")

expresion_total <- read.csv(exp_file, row.names = 1, check.names = FALSE)
# Filtrar por TODOS los genes de la firma (UP + DOWN) y TODAS las muestras tumorales
(374)
genes_comunes <- intersect(genes_firma_unicos, rownames(expresion_total))
muestras_comunes <- intersect(muestras_tumorales, colnames(expresion_total))

expresion_filtrada <- expresion_total[genes_comunes, muestras_comunes, drop = FALSE]

cat("Genes comunes entre firma y matriz:", length(genes_comunes), "\n")

## Genes comunes entre firma y matriz: 268

cat("Muestras tumorales utilizadas (final):", ncol(expresion_filtrada), "\n")

## Muestras tumorales utilizadas (final): 373

# Transponer para que las muestras sean filas
expresion_t <- t(expresion_filtrada)
cat("Dimensión final (Muestras x Genes):", nrow(expresion_t), "x", ncol(expresion_t),
"\n")

## Dimensión final (Muestras x Genes): 373 x 268

# =====
# 4. DEFINICIÓN DE LA FUNCIÓN DE ANÁLISIS KM (Kaplan-Meier)# (Se mantiene la función del
Score Ponderado)
# =====

plot_km_analysis_combined <- function(data_subset, dataset_name) {

  data_subset <- data_subset %>%
    dplyr::filter(!is.na(Tiempo_OS), !is.na(Estado_OS))

  if (length(unique(data_subset$Risk_Group)) < 2) {
    cat("Advertencia: solo un grupo presente en la cohorte para supervivencia. Se
omite el análisis KM.\n")
    return(NULL)
  }

  surv_object <- Surv(time = data_subset$Tiempo_OS, event = data_subset$Estado_OS)
  fit_survival <- surv_fit(surv_object ~ Risk_Group, data = data_subset)

  log_rank_test <- survival::survdiffr(surv_object ~ Risk_Group, data = data_subset)
  p_value <- 1 - pchisq(log_rank_test$chisq, df = length(log_rank_test$n) - 1)

  cox_fit <- coxph(surv_object ~ Risk_Group, data = data_subset)
  hr_data <- summary(cox_fit)

  cat("\n--- Resultados de Supervivencia Log-Rank para", dataset_name, " (Score
PONDERADO UP+DOWN) ---\n")
  cat("P-value Log-Rank:", format.pval(p_value, digits = 3), "\n")
  cat("Hazard Ratio (TGF-B UPREGULATED vs DOWNREGULATED):",
      round(hr_data$conf.int[1], 2),

```

```

" (95% CI:", round(hr_data$conf.int[3], 2),
"-", round(hr_data$conf.int[4], 2), ")\n")

p <- survminer::ggsurvplot(
  fit_survival,
  data = data_subset,
  pval = format.pval(p_value, digits = 3),
  pval.method = FALSE,
  risk.table = TRUE,
  palette = c("blue", "red"),
  title = paste("Supervivencia KM -", dataset_name, " (Firma TGF-β PONDERADA
UP+DOWN)"),
  xlab = "Tiempo (meses)",
  legend.title = "Firma TGF-B",
  legend.labs = c("TGF-B downregulated", "TGF-B upregulated"),
  font.main = c(12, "bold", "black"),
  font.x = c(10, "plain", "black"),
  font.y = c(10, "plain", "black"),
  risk.table.fontsize = 3
)

return(p)
}

# =====
# 5. CÁLCULO DEL SCORE PONDERADO POR COEFICIENTES DE COX (RISK SCORE)
# =====

# VALIDACIÓN DE GENES (Define genes_up_validos y genes_down_validos)
# Se usa La matriz 'expresion_t' que incluye UP+DOWN (268 genes)
genes_up_validos <- intersect(genes_sobreexp, colnames(expresion_t))
genes_down_validos <- intersect(genes_infraexp, colnames(expresion_t))

# La salida ahora debería mostrar Los genes DOWN:
cat("Genes UP válidos:", length(genes_up_validos), " | Genes DOWN válidos:",
length(genes_down_validos), "\n")

## Genes UP válidos: 160 | Genes DOWN válidos: 108

if (length(genes_up_validos) == 0 && length(genes_down_validos) == 0) {
  stop("ERROR: No hay genes válidos para calcular el score combinado.")
}

# Carga de datos clínicos de supervivencia (GSE244807)
metadata_file <- "GSE244807_metadata_filtered.csv"
if (!file.exists(metadata_file)) stop("ERROR: No se encuentra el fichero de metadata
'GSE244807_metadata_filtered.csv'.")
metadata_244807 <- read.csv(metadata_file)

clinical_data_survival <- metadata_244807 %>%
  dplyr::filter(geo_accession %in% GSE244807_samples) %>%
  dplyr::select(Paciente_ID = geo_accession, Tiempo_OS = os, Estado_OS = death) %>%
  dplyr::mutate(
    Tiempo_OS = as.numeric(Tiempo_OS),
    Estado_OS = as.numeric(Estado_OS)
  ) %>%

```

```

dplyr::filter(!is.na(Tiempo_OS) & !is.na(Estado_OS) & Estado_OS %in% c(0, 1))

# Filtrar expresion_t SOLO para Las 241 muestras de GSE244807
expresion_t_gse244807 <- expresion_t[rownames(expresion_t) %in%
clinical_data_survival$Paciente_ID, , drop = FALSE]

expresion_t_gse244807_df <- as.data.frame(expresion_t_gse244807)
expresion_t_gse244807_df$Paciente_ID <- rownames(expresion_t_gse244807_df)

# Unir expresión (solo 241 muestras) con datos clínicos de supervivencia
survival_cox_data <- clinical_data_survival %>%
  dplyr::inner_join(expresion_t_gse244807_df, by = "Paciente_ID") %>%
  dplyr::filter(!is.na(Tiempo_OS), !is.na(Estado_OS))

# Calcular coeficientes beta (Hazard Ratio) univariantes para cada gen
all_valid_genes <- union(genes_up_validos, genes_down_validos)
beta_coefficients <- list()
surv_object_cox <- Surv(time = survival_cox_data$Tiempo_OS, event =
survival_cox_data$Estado_OS)

cat("\nCalculando coeficientes de riesgo (Beta) para cada gen (UP + DOWN)... \n")

##
## Calculando coeficientes de riesgo (Beta) para cada gen (UP + DOWN)...

for (gene in all_valid_genes) {
  # Solo ejecutar Cox si el gen tiene varianza (no es constante)
  if (sd(survival_cox_data[[gene]], na.rm = TRUE) > 1e-6) {
    formula_cox <- as.formula(paste("surv_object_cox ~", gene))
    tryCatch({
      uni_cox <- coxph(formula_cox, data = survival_cox_data)
      beta_coefficients[[gene]] <- coef(uni_cox)[gene]
    }, error = function(e) {
      # Omitir genes problemáticos
    })
  }
}

beta_df <- data.frame(
  Gene = names(beta_coefficients),
  Beta = unlist(beta_coefficients)
)

cat("Genes con coeficientes Beta válidos (final):", nrow(beta_df), "/",
length(all_valid_genes), "\n")

## Genes con coeficientes Beta válidos (final): 266 / 268

# =====
# 6. ORDENAMIENTO DE COEFICIENTES BETA
# =====

# Ordenar Los coeficientes Beta de Cox de MAYOR a MENOR
resultados_beta_ordenados <- beta_df %>%
  dplyr::arrange(dplyr::desc(Beta)) %>%
  dplyr::rename(Gene_ID = Gene, Beta_Cox = Beta) # Formato de salida

# Exportar La tabla ordenada a un nuevo archivo CSV
write_csv(resultados_beta_ordenados, "coeficientes_beta_gse244807_ordenados.csv")

```

```

cat(paste0("El archivo 'coeficientes_beta_gse244807_ordenados.csv' ha sido creado con ",
nrow(resultados_beta_ordenados), " genes, ordenados de mayor a menor Beta.\n"))

## El archivo 'coeficientes_beta_gse244807_ordenados.csv' ha sido creado con 266 genes,
ordenados de mayor a menor Beta.

# Calcular el Score de Riesgo (Risk Score)
# Usamos la expresión SÓLO de las 241 muestras de GSE244807
expresion_risk_genes <- expresion_t_gse244807[, beta_df$Gene, drop = FALSE]
expresion_risk_genes_matrix <- as.matrix(expresion_risk_genes)
beta_vector <- setNames(beta_df$Beta, beta_df$Gene)

# Cálculo del Score Ponderado (matriz de expresión * vector beta)
score_ponderado_gs <- expresion_risk_genes_matrix %>% beta_vector
score_ponderado_gs <- as.vector(score_ponderado_gs)
names(score_ponderado_gs) <- rownames(expresion_risk_genes)

# 4.4. CLASIFICACIÓN DEL RIESGO PONDERADO
global_median_score_ponderado <- median(score_ponderado_gs, na.rm = TRUE)

clasificacion_tgfb_ponderada <- data.frame(
  Paciente_ID = names(score_ponderado_gs),
  TGFB_Score = score_ponderado_gs,
  # Lógica: Score ALTO (>= mediana) es Alto Riesgo / UPREGULATED
  Grupo_TGFB = factor(
    ifelse(score_ponderado_gs >= global_median_score_ponderado, "TGF-B UPREGULATED",
    "TGF-B DOWNREGULATED"),
    levels = c("TGF-B DOWNREGULATED", "TGF-B UPREGULATED")
  )
)

cat("\n--- Resumen de Clasificación Ponderada (n=", nrow(clasificacion_tgfb_ponderada),
") ---\n")

##
## --- Resumen de Clasificación Ponderada (n= 241 ) ---

print(clasificacion_tgfb_ponderada %>% group_by(Grupo_TGFB) %>% summarise(N = n()))

## # A tibble: 2 × 2
##   Grupo_TGFB          N
##   <fct>          <int>
## 1 TGF-B DOWNREGULATED 120
## 2 TGF-B UPREGULATED  121

cat("Punto de corte (Mediana global):", global_median_score_ponderado, "\n")

## Punto de corte (Mediana global): -120.0604

# =====
# 7. INTEGRACIÓN Y ANÁLISIS KM PONDERADO (UP+DOWN)
# =====

clinical_data_final_ponderado <- clinical_data_survival %>%
  dplyr::inner_join(clasificacion_tgfb_ponderada, by = "Paciente_ID") %>%
  dplyr::rename(Risk_Score = TGFB_Score, Risk_Group = Grupo_TGFB)

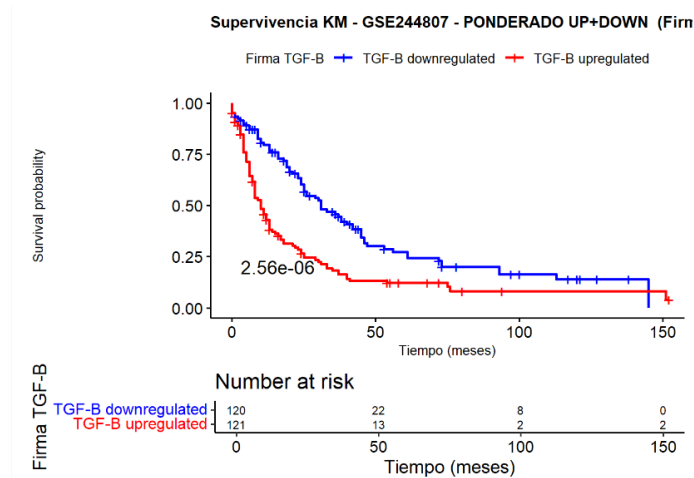
cat("\nMuestras GSE244807 con datos clínicos + score PONDERADO (final):",
nrow(clinical_data_final_ponderado), "\n")
## Muestras GSE244807 con datos clínicos + score PONDERADO (final): 241

```

```
# --- EJECUCIÓN DEL ANÁLISIS KM PONDERADO ---
km_plot_ponderado <- plot_km_analysis_combined(clinical_data_final_ponderado, "GSE244807
- PONDERADO UP+DOWN")

##
## --- Resultados de Supervivencia Log-Rank para GSE244807 - PONDERADO UP+DOWN (Score
PONDERADO UP+DOWN) ---
## P-value Log-Rank: 2.56e-06
## Hazard Ratio (TGF-B UPREGULATED vs DOWNREGULATED): 2.03 (95% CI: 1.5 - 2.75 )

if (!is.null(km_plot_ponderado)) {
  print(km_plot_ponderado)
  cat("\nGráfico de Kaplan-Meier (Score PONDERADO UP+DOWN) generado.\n")
} else {
  cat("El análisis KM ponderado no se pudo realizar.\n")
}
```



8. ANÁLISIS UNIVARIANTE MEDIANTE EL MODELO DE COX- SOLO GENES UP DE LA FIRMA DE TGF-B

```
genes_firma_unicos <- genes_sobreexp # Solo usamos genes UP para el filtrado inicial
cat("Muestras GSE244807 para análisis de supervivencia:", length(genes_firma_unicos),
"\n")

## Muestras GSE244807 para análisis de supervivencia: 166

# =====
# CARGA Y PREPARACIÓN DE LA MATRIZ DE EXPRESIÓN
# =====

exp_file <- "matriz_corregida_ComBat.csv"
if (!file.exists(exp_file)) stop("ERROR: No se encuentra el fichero de expresión
'matriz_corregida_ComBat.csv'.")

expresion_total <- read.csv(exp_file, row.names = 1, check.names = FALSE)
# Filtrar solo por GENES UP y TODAS las muestras tumorales (374)
genes_comunes <- intersect(genes_firma_unicos, rownames(expresion_total))
muestras_comunes <- intersect(muestras_tumorales, colnames(expresion_total))

expresion_filtrada <- expresion_total[genes_comunes, muestras_comunes, drop = FALSE]

cat("Genes comunes entre firma y matriz (SOLO UP):", length(genes_comunes), "\n")

## Genes comunes entre firma y matriz (SOLO UP): 160
```

```

cat("Muestras tumorales utilizadas (final):", ncol(expresion_filtrada), "\n")

## Muestras tumorales utilizadas (final): 373

# Transponer para que las muestras sean filas
expresion_t <- t(expresion_filtrada)
cat("Dimensión final (Muestras x Genes):", nrow(expresion_t), "x", ncol(expresion_t),
"\n")

## Dimensión final (Muestras x Genes): 373 x 160

# =====
# DEFINICIÓN DE LA FUNCIÓN DE ANÁLISIS KM
# =====

plot_km_analysis_up_only <- function(data_subset, dataset_name) {

  data_subset <- data_subset %>%
    dplyr::filter(!is.na(Tiempo_OS), !is.na(Estado_OS))

  if (length(unique(data_subset$Risk_Group)) < 2) {
    cat("Advertencia: solo un grupo presente en la cohorte para supervivencia. Se
omite el análisis KM.\n")
    return(NULL)
  }

  surv_object <- Surv(time = data_subset$Tiempo_OS, event = data_subset$Estado_OS)
  fit_survival <- surv_fit(surv_object ~ Risk_Group, data = data_subset)

  log_rank_test <- survival::survdifff(surv_object ~ Risk_Group, data = data_subset)
  p_value <- 1 - pchisq(log_rank_test$chisq, df = length(log_rank_test$n) - 1)

  cox_fit <- coxph(surv_object ~ Risk_Group, data = data_subset)
  hr_data <- summary(cox_fit)

  cat("\n--- Resultados de Supervivencia Log-Rank para", dataset_name, " (Score
PONDERADO SOLO UP) ---\n")
  cat("P-value Log-Rank:", format.pval(p_value, digits = 3), "\n")
  cat("Hazard Ratio (TGF-B UPREGULATED vs DOWNREGULATED):",
      round(hr_data$conf.int[1], 2),
      " (95% CI:", round(hr_data$conf.int[3], 2),
      "-", round(hr_data$conf.int[4], 2), ")\n")
  p <- survminer::ggsurvplot(
    fit_survival,
    data = data_subset,
    pval = format.pval(p_value, digits = 3),
    pval.method = FALSE,
    risk.table = TRUE,
    palette = c("blue", "red"),
    title = paste("Supervivencia KM -", dataset_name, " (Firma TGF-β PONDERADA SOLO
UP)"),
    xlab = "Tiempo (meses)",
    legend.title = "Firma TGF-B",
    legend.labs = c("TGF-B downregulated", "TGF-B upregulated"),
    font.main = c(12, "bold", "black"),
    font.x = c(10, "plain", "black"),
    font.y = c(10, "plain", "black"),
    risk.table.fontsize = 3
  )

  return(p)
}

```

```

}

# =====
# CÁLCULO DEL SCORE PONDERADO POR COEFICIENTES DE COX
# =====

# Solo los genes UP son válidos para este análisis
genes_up_validos <- intersect(genes_sobreexp, colnames(expresion_t))

# La lista de genes para el modelo Cox es solo UP
all_valid_genes_up_only <- genes_up_validos

cat("Genes UP válidos para Cox:", length(all_valid_genes_up_only), "\n")

## Genes UP válidos para Cox: 160

if (length(all_valid_genes_up_only) == 0) {
  stop("ERROR: No hay genes UP válidos para calcular el score.")
}

# Carga de datos clínicos de supervivencia (GSE244807)
metadata_file <- "GSE244807_metadata_filtered.csv"
if (!file.exists(metadata_file)) stop("ERROR: No se encuentra el fichero de metadata
'GSE244807_metadata_filtered.csv'.")
metadata_244807 <- read.csv(metadata_file)

clinical_data_survival <- metadata_244807 %>%
  dplyr::filter(geo_accession %in% GSE244807_samples) %>%
  dplyr::select(Paciente_ID = geo_accession, Tiempo_OS = os, Estado_OS = death) %>%
  dplyr::mutate(
    Tiempo_OS = as.numeric(Tiempo_OS),
    Estado_OS = as.numeric(Estado_OS)
  ) %>%
  dplyr::filter(!is.na(Tiempo_OS) & !is.na(Estado_OS) & Estado_OS %in% c(0, 1))

# Filtrar expresion_t SOLO para las 241 muestras de GSE244807
# expresion_t ya solo tiene genes UP en sus columnas
expresion_t_gse244807 <- expresion_t[rownames(expresion_t) %in%
clinical_data_survival$Paciente_ID, , drop = FALSE]

expresion_t_gse244807_df <- as.data.frame(expresion_t_gse244807)
expresion_t_gse244807_df$Paciente_ID <- rownames(expresion_t_gse244807_df)

# Unir expresión (solo 241 muestras) con datos clínicos de supervivencia
survival_cox_data <- clinical_data_survival %>%
  dplyr::inner_join(expresion_t_gse244807_df, by = "Paciente_ID") %>%
  dplyr::filter(!is.na(Tiempo_OS), !is.na(Estado_OS))

# Calcular coeficientes beta (Hazard Ratio) univariantes para cada gen
beta_coefficients <- list()
surv_object_cox <- Surv(time = survival_cox_data$Tiempo_OS, event =
survival_cox_data$Estado_OS)

cat("\nCalculando coeficientes de riesgo (Beta) para cada gen (SOLO UP)... \n")

##
## Calculando coeficientes de riesgo (Beta) para cada gen (SOLO UP)...

for (gene in all_valid_genes_up_only) {
  # Solo ejecutar Cox si el gen tiene varianza

```

```

    if (sd(survival_cox_data[[gene]], na.rm = TRUE) > 1e-6) {
      formula_cox <- as.formula(paste("surv_object_cox ~", gene))
      tryCatch({
        uni_cox <- coxph(formula_cox, data = survival_cox_data)
        beta_coefficients[[gene]] <- coef(uni_cox)[gene]
      }, error = function(e) {
        # Omitir genes problemáticos
      })
    }
  }

beta_df_up_only <- data.frame(
  Gene = names(beta_coefficients),
  Beta = unlist(beta_coefficients)
)

cat("Genes UP con coeficientes Beta válidos (final):", nrow(beta_df_up_only), "/",
length(all_valid_genes_up_only), "\n")

## Genes UP con coeficientes Beta válidos (final): 158 / 160

# Calcular el Score de Riesgo (Risk Score)
expresion_risk_genes <- expresion_t_gse244807[, beta_df_up_only$Gene, drop = FALSE]
expresion_risk_genes_matrix <- as.matrix(expresion_risk_genes)
beta_vector <- setNames(beta_df_up_only$Beta, beta_df_up_only$Gene)

# Cálculo del Score Ponderado (matriz de expresión * vector beta)
score_ponderado_gs <- expresion_risk_genes_matrix %*% beta_vector
score_ponderado_gs <- as.vector(score_ponderado_gs)
names(score_ponderado_gs) <- rownames(expresion_risk_genes)

# CLASIFICACIÓN DEL RIESGO PONDERADO
global_median_score_ponderado <- median(score_ponderado_gs, na.rm = TRUE)

clasificacion_tgfb_ponderada <- data.frame(
  Paciente_ID = names(score_ponderado_gs),
  TGFB_Score = score_ponderado_gs,
  # Lógica: Score ALTO (>= mediana) es Alto Riesgo / UPREGULATED
  Grupo_TGFB = factor(
    ifelse(score_ponderado_gs >= global_median_score_ponderado, "TGF-B UPREGULATED",
    "TGF-B DOWNREGULATED"),
    levels = c("TGF-B DOWNREGULATED", "TGF-B UPREGULATED")
  )
)

cat("\n--- Resumen de Clasificación Ponderada (n=", nrow(clasificacion_tgfb_ponderada),
") ---\n")

##
## --- Resumen de Clasificación Ponderada (n= 241 ) ---

print(clasificacion_tgfb_ponderada %>% group_by(Grupo_TGFB) %>% summarise(N = n()))

## # A tibble: 2 × 2
##   Grupo_TGFB      N
##   <fct>         <int>
## 1 TGF-B DOWNREGULATED 120
## 2 TGF-B UPREGULATED 121

```



```

cat("Punto de corte (Mediana global):", global_median_score_ponderado, "\n")

## Punto de corte (Mediana global): -57.49464

# =====
# INTEGRACIÓN Y ANÁLISIS KM PONDERADO (SOLO UP)
# =====

clinical_data_final_ponderado <- clinical_data_survival %>%
  dplyr::inner_join(clasificacion_tgfb_ponderada, by = "Paciente_ID") %>%
  dplyr::rename(Risk_Score = TGFB_Score, Risk_Group = Grupo_TGFB)

cat("\nMuestras GSE244807 con datos clínicos + score PONDERADO SOLO UP (final):",
nrow(clinical_data_final_ponderado), "\n")

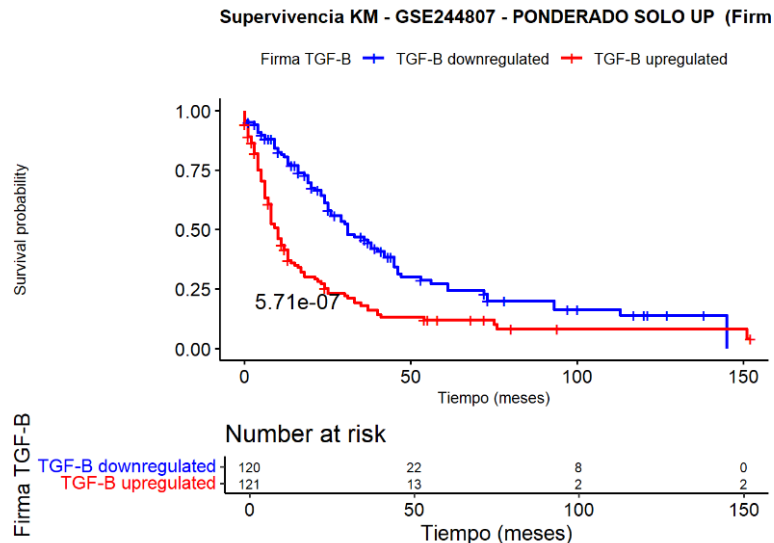
##
## Muestras GSE244807 con datos clínicos + score PONDERADO SOLO UP (final): 241

# --- EJECUCIÓN DEL ANÁLISIS KM PONDERADO SOLO UP ---
km_plot_ponderado <- plot_km_analysis_up_only(clinical_data_final_ponderado, "GSE244807 -
PONDERADO SOLO UP")

##
## --- Resultados de Supervivencia Log-Rank para GSE244807 - PONDERADO SOLO UP (Score
PONDERADO SOLO UP) ---
## P-value Log-Rank: 5.71e-07
## Hazard Ratio (TGF-B UPREGULATED vs DOWNREGULATED): 2.12 (95% CI: 1.57 - 2.86 )

if (!is.null(km_plot_ponderado)) {
  print(km_plot_ponderado)
  cat("\nGráfico de Kaplan-Meier (Score PONDERADO SOLO UP) generado.\n")
} else {
  cat("El análisis KM ponderado (SOLO UP) no se pudo realizar.\n")
}

```



9. ANÁLISIS UNIVARIANTE MEDIANTE EL MODELO DE COX- SOLO GENES DOWN DE LA FIRMA DE TGF-B

```
# =====
# CARGA DE LIBRERÍAS NECESARIAS
# =====
if (!requireNamespace("tidyverse", quietly = TRUE)) install.packages("tidyverse")
if (!requireNamespace("survival", quietly = TRUE)) install.packages("survival")
if (!requireNamespace("survminer", quietly = TRUE)) install.packages("survminer")
library(tidyverse)
library(survival)
library(survminer)

# Asumiendo que 'genes_infraexp', 'muestras_tumorales', 'GSE244807_samples', etc., están
# cargados
genes_firma_unicos <- genes_infraexp # <--- SOLO GENES DOWN

cat("Muestras GSE244807 para análisis de supervivencia:", length(GSE244807_samples),
    "\n")

## Muestras GSE244807 para análisis de supervivencia: 241

cat("Genes en la firma (SOLO DOWN):", length(genes_firma_unicos), "\n")

## Genes en la firma (SOLO DOWN): 109

# =====
# CARGA Y PREPARACIÓN DE LA MATRIZ DE EXPRESIÓN
# =====

exp_file <- "matriz_corregida_ComBat.csv"
if (!file.exists(exp_file)) stop("ERROR: No se encuentra el fichero de expresión
'matriz_corregida_ComBat.csv'.")

expresion_total <- read.csv(exp_file, row.names = 1, check.names = FALSE)
# Filtrar solo por GENES DOWN y muestras tumorales
genes_comunes <- intersect(genes_firma_unicos, rownames(expresion_total))
muestras_comunes <- intersect(muestras_tumorales, colnames(expresion_total))

expresion_filtrada <- expresion_total[genes_comunes, muestras_comunes, drop = FALSE]

cat("Genes comunes entre firma y matriz (SOLO DOWN):", length(genes_comunes), "\n")

## Genes comunes entre firma y matriz (SOLO DOWN): 108

cat("Muestras tumorales utilizadas (final):", ncol(expresion_filtrada), "\n")

## Muestras tumorales utilizadas (final): 373

# Transponer para que las muestras sean filas
expresion_t <- t(expresion_filtrada)
cat("Dimensión final (Muestras x Genes):", nrow(expresion_t), "x", ncol(expresion_t),
    "\n")

## Dimensión final (Muestras x Genes): 373 x 108
```

```

# =====
# DEFINICIÓN DE LA FUNCIÓN DE ANÁLISIS KM
# =====

plot_km_analysis_down_only <- function(data_subset, dataset_name) {

  data_subset <- data_subset %>%
    dplyr::filter(!is.na(Tiempo_OS), !is.na(Estado_OS))

  if (length(unique(data_subset$Risk_Group)) < 2) {
    cat("Advertencia: solo un grupo presente en la cohorte para supervivencia. Se
omite el análisis KM.\n")
    return(NULL)
  }

  surv_object <- Surv(time = data_subset$Tiempo_OS, event = data_subset$Estado_OS)
  fit_survival <- surv_fit(surv_object ~ Risk_Group, data = data_subset)

  log_rank_test <- survival::survdif(fit_survival ~ Risk_Group, data = data_subset)
  p_value <- 1 - pchisq(log_rank_test$chisq, df = length(log_rank_test$n) - 1)

  cox_fit <- coxph(surv_object ~ Risk_Group, data = data_subset)
  hr_data <- summary(cox_fit)

  cat("\n--- Resultados de Supervivencia Log-Rank para", dataset_name, " (Score
PONDERADO SOLO DOWN) ---\n")
  cat("P-value Log-Rank:", format.pval(p_value, digits = 3), "\n")
  cat("Hazard Ratio (Alto Enriquecimiento vs Bajo Enriquecimiento):",
      round(hr_data$conf.int[1], 2),
      " (95% CI:", round(hr_data$conf.int[3], 2),
      "-", round(hr_data$conf.int[4], 2), ") \n")

  # Los colores se asignarán automáticamente al orden del factor (Bajo
  # Enriquecimiento=1er color, Alto Enriquecimiento=2do color)
  p <- survminer::ggsurvplot(
    fit_survival,
    data = data_subset,
    pval = format.pval(p_value, digits = 3),
    pval.method = FALSE,
    risk.table = TRUE,
    palette = c("blue", "red"), # Azul para Bajo Enriquecimiento, Rojo para Alto
    # Enriquecimiento
    title = paste("Supervivencia KM -", dataset_name, " (Firma TGF-β PONDERADA SOLO
    DOWN)"),
    xlab = "Tiempo (meses)",
    legend.title = "Firma TGF-B",
    legend.labs = c("TGF-B downregulated", "TGF-B upregulated"),
    font.main = c(12, "bold", "black"),
    font.x = c(10, "plain", "black"),
    font.y = c(10, "plain", "black"),
    risk.table.fontsize = 3
  )

  return(p)
}

```

```

# =====
# CÁLCULO DEL SCORE PONDERADO POR COEFICIENTES DE COX (RISK SCORE)
# =====

genes_down_validos <- intersect(genes_infraexp, colnames(expresion_t))
all_valid_genes_down_only <- genes_down_validos

cat("Genes DOWN válidos para Cox:", length(all_valid_genes_down_only), "\n")

## Genes DOWN válidos para Cox: 108

if (length(all_valid_genes_down_only) == 0) {
  stop("ERROR: No hay genes DOWN válidos para calcular el score.")
}

# Carga de datos clínicos y unión con expresión (mismo proceso)
metadata_file <- "GSE244807_metadata_filtered.csv"
if (!file.exists(metadata_file)) stop("ERROR: No se encuentra el fichero de metadata
'GSE244807_metadata_filtered.csv'.")
metadata_244807 <- read.csv(metadata_file)

clinical_data_survival <- metadata_244807 %>%
  dplyr::filter(geo_accession %in% GSE244807_samples) %>%
  dplyr::select(Paciente_ID = geo_accession, Tiempo_OS = os, Estado_OS = death) %>%
  dplyr::mutate(
    Tiempo_OS = as.numeric(Tiempo_OS),
    Estado_OS = as.numeric(Estado_OS)
  ) %>%
  dplyr::filter(!is.na(Tiempo_OS) & !is.na(Estado_OS) & Estado_OS %in% c(0, 1))

expresion_t_gse244807 <- expresion_t[rownames(expresion_t) %in%
clinical_data_survival$Paciente_ID, , drop = FALSE]

expresion_t_gse244807_df <- as.data.frame(expresion_t_gse244807)
expresion_t_gse244807_df$Paciente_ID <- rownames(expresion_t_gse244807_df)

survival_cox_data <- clinical_data_survival %>%
  dplyr::inner_join(expresion_t_gse244807_df, by = "Paciente_ID") %>%
  dplyr::filter(!is.na(Tiempo_OS), !is.na(Estado_OS))

# Calcular coeficientes beta (mismo proceso)
beta_coefficients <- list()
surv_object_cox <- Surv(time = survival_cox_data$Tiempo_OS, event =
survival_cox_data$Estado_OS)

cat("\nCalculando coeficientes de riesgo (Beta) para cada gen (SOLO DOWN)...\n")

##
## Calculando coeficientes de riesgo (Beta) para cada gen (SOLO DOWN)...

for (gene in all_valid_genes_down_only) {
  if (sd(survival_cox_data[[gene]], na.rm = TRUE) > 1e-6) {
    formula_cox <- as.formula(paste("surv_object_cox ~", gene))
    tryCatch({
      uni_cox <- coxph(formula_cox, data = survival_cox_data)
      beta_coefficients[[gene]] <- coef(uni_cox)[gene]
    }, error = function(e) {})
  }
}

```

```

beta_df_down_only <- data.frame(
  Gene = names(beta_coefficients),
  Beta = unlist(beta_coefficients)
)

# Calcular el Score de Riesgo (Risk Score) (mismo proceso)
expresion_risk_genes <- expresion_t_gse244807[, beta_df_down_only$Gene, drop = FALSE]
expresion_risk_genes_matrix <- as.matrix(expresion_risk_genes)
beta_vector <- setNames(beta_df_down_only$Beta, beta_df_down_only$Gene)

score_ponderado_gs <- expresion_risk_genes_matrix %*% beta_vector
score_ponderado_gs <- as.vector(score_ponderado_gs)
names(score_ponderado_gs) <- rownames(expresion_risk_genes)

# CLASIFICACIÓN DEL RIESGO PONDERADO
global_median_score_ponderado <- median(score_ponderado_gs, na.rm = TRUE)

clasificacion_tgfb_ponderada <- data.frame(
  Paciente_ID = names(score_ponderado_gs),
  TGFB_Score = score_ponderado_gs,
  # Asignar a "Alto Enriquecimiento" si el score es >= mediana
  Risk_Group = factor(
    ifelse(score_ponderado_gs >= global_median_score_ponderado, "Alto
Enriquecimiento", "Bajo Enriquecimiento"),
    # IMPORTANTE: Bajo Enriquecimiento DEBE ser el primer nivel (referencia) para HR
    > 1
    levels = c("Bajo Enriquecimiento", "Alto Enriquecimiento")
  )
)

cat("\n--- Resumen de Clasificación Ponderada (SOLO DOWN, n=",
nrow(clasificacion_tgfb_ponderada), ") ---\n")

print(clasificacion_tgfb_ponderada %>% group_by(Risk_Group) %>% summarise(N = n()))

## # A tibble: 2 × 2
##   Risk_Group      N
##   <fct>         <int>
## 1 Bajo Enriquecimiento 120
## 2 Alto Enriquecimiento 121

cat("Punto de corte (Mediana global):", global_median_score_ponderado, "\n")

## Punto de corte (Mediana global): -62.47159

# =====
# INTEGRACIÓN Y ANÁLISIS KM PONDERADO (SOLO DOWN)
# =====

clinical_data_final_ponderado <- clinical_data_survival %>%
  dplyr::inner_join(clasificacion_tgfb_ponderada, by = "Paciente_ID") %>%
  dplyr::rename(Risk_Score = TGFB_Score)

cat("\nMuestras GSE244807 con datos clínicos + score PONDERADO SOLO DOWN (final):",
nrow(clinical_data_final_ponderado), "\n")

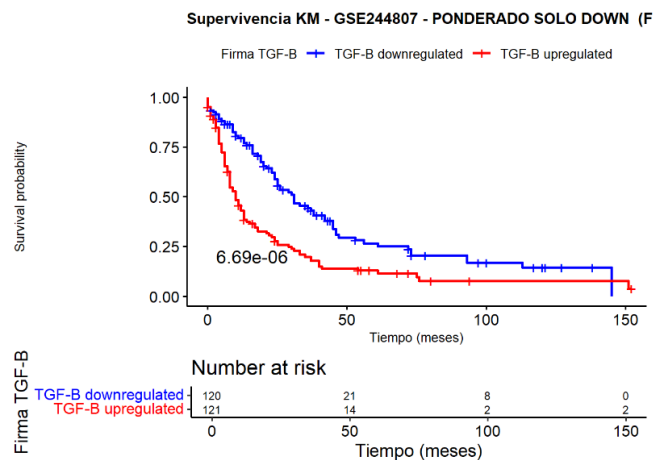
##
## Muestras GSE244807 con datos clínicos + score PONDERADO SOLO DOWN (final): 241

```

```
# --- EJECUCIÓN DEL ANÁLISIS KM PONDERADO SOLO DOWN ---
km_plot_ponderado <- plot_km_analysis_down_only(clinical_data_final_ponderado, "GSE244807
- PONDERADO SOLO DOWN")

##
## --- Resultados de Supervivencia Log-Rank para GSE244807 - PONDERADO SOLO DOWN (Score
PONDERADO SOLO DOWN) ---
## P-value Log-Rank: 6.69e-06
## Hazard Ratio (Alto Enriquecimiento vs Bajo Enriquecimiento): 1.97 (95% CI: 1.46 -
2.66 )

if (!is.null(km_plot_ponderado)) {
  print(km_plot_ponderado)
  cat("\nGráfico de Kaplan-Meier (Score PONDERADO SOLO DOWN) generado.\n")
} else {
  cat("El análisis KM ponderado (SOLO DOWN) no se pudo realizar.\n")
}
```



10. CURVAS ROC

```
# =====
# INSTALACIÓN Y CARGA DE LIBRERÍAS
# =====
if (!requireNamespace("timeROC", quietly = TRUE)) install.packages("timeROC")
if (!requireNamespace("tidyverse", quietly = TRUE)) install.packages("tidyverse")
library(tidyverse)
library(survival)
library(timeROC)

# =====
# DEFINICIÓN Y PREPARACIÓN DE DATOS
# =====

if (!exists("clinical_data_final_ponderado")) {
  stop("ERROR: 'clinical_data_final_ponderado' no está cargado. Asegúrese de que el
dataframe esté en el entorno.")
}

data_roc <- clinical_data_final_ponderado %>%
  # Aseguramos los nombres de columna y tipos de datos
  dplyr::select(Tiempo_OS, Estado_OS, Risk_Score) %>%
  dplyr::filter(!is.na(Tiempo_OS), !is.na(Estado_OS), !is.na(Risk_Score)) %>%
  dplyr::mutate(
    Tiempo_OS = as.numeric(Tiempo_OS),
    Estado_OS = as.numeric(Estado_OS)
  )
```

```

cat("Muestras válidas para ROC:", nrow(data_roc), "\n")

## Muestras válidas para ROC: 241

# =====
# ANÁLISIS ROC DEPENDIENTE DEL TIEMPO (Múltiples Tiempos)
# =====

# Definir Los puntos de tiempo de interés (1, 3 y 5 años)
time_points <- c(12, 36, 60)

# Inicializar un dataframe para almacenar Los resultados
results_table <- data.frame(
  Tiempo_Meses = integer(),
  AUC = numeric(),
  IC_95_Lower = numeric(),
  IC_95_Upper = numeric()
)

paste(time_points, collapse = ", "), "meses ---\n"))

# Bucle para calcular La AUC en cada punto de tiempo
for (t in time_points) {
  # Ejecutar La función timeROC
  roc_fit <- timeROC(
    T = data_roc$Tiempo_OS,
    delta = data_roc$Estado_OS,
    marker = data_roc$Risk_Score,
    cause = 1,
    weighting = "marginal",
    times = t,
    iid = TRUE
  )

  # Extraer el AUC y el IC
  auc_value <- round(roc_fit$AUC[[2]], 3)

  # Calcular IC 95%
  auc_ci <- confint(roc_fit, level = 0.95)$CI_AUC[1, ]
  auc_ci_lower <- round(auc_ci[1], 3)
  auc_ci_upper <- round(auc_ci[2], 3)

  # Agregar resultados a La tabla
  results_table <- results_table %>%
    add_row(
      Tiempo_Meses = t,
      AUC = auc_value,
      IC_95_Lower = auc_ci_lower,
      IC_95_Upper = auc_ci_upper
    )
}

```

```

# =====
# REPORTE DE RESULTADOS Y VISUALIZACIÓN
# =====

print(results_table)

##   Tiempo_Meses   AUC IC_95_Lower IC_95_Upper
## 1           12 0.734         66.83       79.95
## 2           36 0.721         63.72       80.50
## 3           60 0.691         57.08       81.14

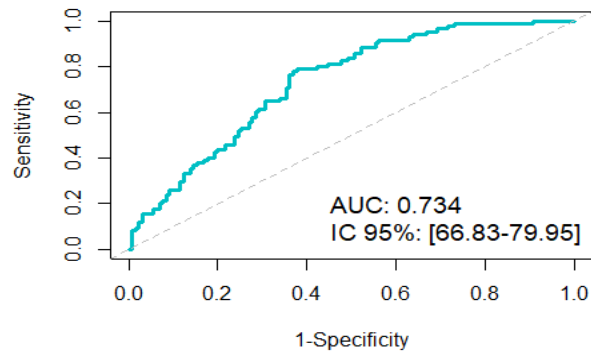
# --- Visualización de La Curva ROC para 12 meses ---
roc_fit_12 <- timeROC(
  T = data_roc$Tiempo_OS,
  delta = data_roc$Estado_OS,
  marker = data_roc$Risk_Score,
  cause = 1,
  weighting = "marginal",
  times = 12,
  iid = TRUE
)

# Obtener AUC y CI de 12 meses para La Leyenda
auc_12 <- results_table %>% filter(Tiempo_Meses == 12)
auc_val_12 <- auc_12$AUC
ci_low_12 <- auc_12$IC_95_Lower
ci_up_12 <- auc_12$IC_95_Upper
plot(roc_fit_12,
     time = 12,
     col = "#00BFC4",
     title = FALSE,
     lwd = 3,
     main = paste("Curva ROC dependiente del tiempo a 36 meses"),
     cex.lab = 1.1,
     cex.axis = 1.1
)

# Línea diagonal de referencia (azar)
abline(0, 1, col = "grey", lty = 2)

# Añadir La etiqueta del AUC y el IC
legend("bottomright",
      legend = c(paste0("AUC: ", auc_val_12),
                 paste0("IC 95%: [", ci_low_12, "-", ci_up_12, "]")),
      col = "black",
      lty = 0,
      bty = "n",
      cex = 1.2)

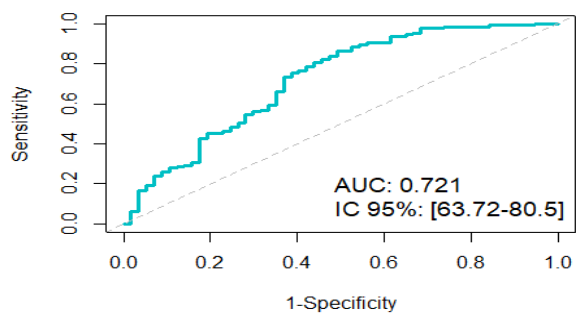
```

```
# Recalcular roc_fit para 36 meses para asegurar la visualización
roc_fit_36 <- timeROC(
  T = data_roc$Tiempo_OS,
  delta = data_roc$Estado_OS,
  marker = data_roc$Risk_Score,
  cause = 1,
  weighting = "marginal",
  times = 36,
  iid = TRUE
)
# Obtener AUC y CI de 36 meses para La Leyenda
auc_36 <- results_table %>% filter(Tiempo_Meses == 36)
auc_val_36 <- auc_36$AUC
ci_low_36 <- auc_36$IC_95_Lower
ci_up_36 <- auc_36$IC_95_Upper

plot(roc_fit_36,
  time = 36,
  col = "#00BFC4",
  title = FALSE,
  lwd = 3,
  main = paste("Curva ROC dependiente del tiempo a 36 meses"),
  cex.lab = 1.1,
  cex.axis = 1.1
)
# Línea diagonal de referencia (azar)
abline(0, 1, col = "grey", lty = 2)

# Añadir la etiqueta del AUC y el IC
legend("bottomright",
  legend = c(paste0("AUC: ", auc_val_36),
    paste0("IC 95%: [", ci_low_36, "-", ci_up_36, "]")),
  col = "black",
  lty = 0,
  bty = "n",
  cex = 1.2)
```



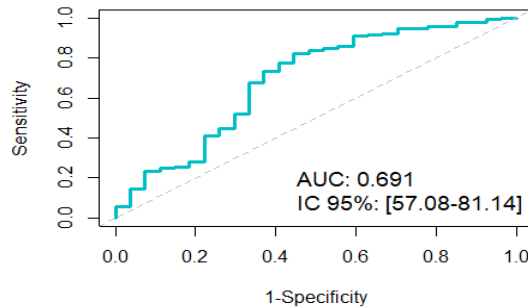
```
# Recalcular roc_fit para 60 meses para asegurar la visualización
roc_fit_60 <- timeROC(
  T = data_roc$Tiempo_OS,
  delta = data_roc$Estado_OS,
  marker = data_roc$Risk_Score,
  cause = 1,
  weighting = "marginal",
  times = 60,
  iid = TRUE
)

# Obtener AUC y CI de 60 meses para la leyenda
auc_60 <- results_table %>% filter(Tiempo_Meses == 60)
auc_val_60 <- auc_60$AUC
ci_low_60 <- auc_60$IC_95_Lower
ci_up_60 <- auc_60$IC_95_Upper

plot(roc_fit_60,
  time = 60,
  col = "#00BFC4",
  title = FALSE,
  lwd = 3,
  main = paste("Curva ROC dependiente del tiempo a 36 meses"),
  cex.lab = 1.1,
  cex.axis = 1.1
)

# Línea diagonal de referencia (azar)
abline(0, 1, col = "grey", lty = 2)

# Añadir la etiqueta del AUC y el IC
legend("bottomright",
  legend = c(paste0("AUC: ", auc_val_60),
    paste0("IC 95%: [", ci_low_60, "-", ci_up_60, "]")),
  col = "black",
  lty = 0,
  bty = "n",
  cex = 1.2)
```



11. ANÁLISIS DE ENRIQUECIMIENTO FUNCIONAL DE LOS GENES DE LA FIRMA DE TGF-B

```
# 0. LIBRERÍAS Y CONFIGURACIÓN INICIAL
# =====
suppressPackageStartupMessages({
  library(tidyverse)
  library(clusterProfiler)
  library(org.Hs.eg.db)
  library(enrichplot)
  library(ggplot2)
  library(DOSE)
  library(stringr)
  library(ReactomePA)
})

# --- Parámetros de Análisis ---
N_PLOT <- 20 # Número de categorías a mostrar en los gráficos
CATEGORIES <- c("GO", "KEGG", "Reactome")
output_dir_csv <- "resultados_enriquecimiento_detallado"
output_dir_plots_dot <- "resultados_graficos_dot_plots_individuales_FE_solo"
output_dir_plots_dot_comp <- "resultados_graficos_comparativos_dot_plots_FE_en_X"

# Crear directorios de salida
if (!dir.exists(output_dir_csv)) { dir.create(output_dir_csv) }
if (!dir.exists(output_dir_plots_dot)) { dir.create(output_dir_plots_dot) }
if (!dir.exists(output_dir_plots_dot_comp)) { dir.create(output_dir_plots_dot_comp) }

# --- Muestras (241) para calcular Log2FC ---
GSE244807_samples <- c("GSM7829922", "GSM7829924", "GSM7829925", "GSM7829926",
"GSM7829927", "GSM7829928", "GSM7829929", "GSM7829930", "GSM7829931", "GSM7829932",
"GSM7829933", "GSM7829934", "GSM7829935", "GSM7829936", "GSM7829937", "GSM7829938",
"GSM7829939", "GSM7829940", "GSM7829941", "GSM7829942", "GSM7829943", "GSM7829944",
"GSM7829945", "GSM7829947", "GSM7829948", "GSM7829949", "GSM7829950", "GSM7829951",
"GSM7829952", "GSM7829954", "GSM7829955", "GSM7829956", "GSM7829957", "GSM7829958",
"GSM7829959", "GSM7829960", "GSM7829961", "GSM7829962", "GSM7829963", "GSM7829964",
"GSM7829965", "GSM7829966", "GSM7829967", "GSM7829968", "GSM7829969", "GSM7829970",
"GSM7829971", "GSM7829972", "GSM7829973", "GSM7829974", "GSM7829975", "GSM7829976",
"GSM7829977", "GSM7829978", "GSM7829979", "GSM7829980", "GSM7829981", "GSM7829982",
"GSM7829983", "GSM7829984", "GSM7829985", "GSM7829986", "GSM7829987", "GSM7829988",
"GSM7829989", "GSM7829990", "GSM7829991", "GSM7829992", "GSM7829993", "GSM7829994",
"GSM7829995", "GSM7829996", "GSM7829997", "GSM7829998", "GSM7829999", "GSM7830000",
"GSM7830001", "GSM7830002", "GSM7830003", "GSM7830004", "GSM7830005", "GSM7830006",
"GSM7830007", "GSM7830008", "GSM7830009", "GSM7830010", "GSM7830011", "GSM7830012",
"GSM7830013", "GSM7830014", "GSM7830015", "GSM7830016", "GSM7830017", "GSM7830018",
"GSM7830019", "GSM7830020", "GSM7830021", "GSM7830022", "GSM7830023", "GSM7830024",
"GSM7830025", "GSM7830026", "GSM7830027", "GSM7830028", "GSM7830029", "GSM7830030",
"GSM7830031", "GSM7830032", "GSM7830033", "GSM7830034", "GSM7830035", "GSM7830036",
"GSM7830037", "GSM7830038", "GSM7830039", "GSM7830040", "GSM7830041", "GSM7830042",
```

```

"GSM7830043", "GSM7830044", "GSM7830045", "GSM7830046", "GSM7830047", "GSM7830048",
"GSM7830049", "GSM7830050", "GSM7830051", "GSM7830052", "GSM7830053", "GSM7830054",
"GSM7830055", "GSM7830056", "GSM7830057", "GSM7830058", "GSM7830059", "GSM7830060",
"GSM7830061", "GSM7830062", "GSM7830063", "GSM7830064", "GSM7830065", "GSM7830066",
"GSM7830067", "GSM7830068", "GSM7830069", "GSM7830070", "GSM7830071", "GSM7830072",
"GSM7830073", "GSM7830074", "GSM7830075", "GSM7830076", "GSM7830077", "GSM7830078",
"GSM7830079", "GSM7830080", "GSM7830081", "GSM7830082", "GSM7830083", "GSM7830084",
"GSM7830085", "GSM7830087", "GSM7830088", "GSM7830089", "GSM7830090", "GSM7830091",
"GSM7830092", "GSM7830093", "GSM7830094", "GSM7830095", "GSM7830096", "GSM7830097",
"GSM7830098", "GSM7830099", "GSM7830100", "GSM7830101", "GSM7830102", "GSM7830103",
"GSM7830104", "GSM7830105", "GSM7830106", "GSM7830107", "GSM7830108", "GSM7830109",
"GSM7830110", "GSM7830111", "GSM7830112", "GSM7830113", "GSM7830114", "GSM7830115",
"GSM7830116", "GSM7830117", "GSM7830118", "GSM7830119", "GSM7830120", "GSM7830121",
"GSM7830122", "GSM7830123", "GSM7830124", "GSM7830125", "GSM7830126", "GSM7830127",
"GSM7830128", "GSM7830129", "GSM7830131", "GSM7830132", "GSM7830133", "GSM7830134",
"GSM7830135", "GSM7830136", "GSM7830137", "GSM7830138", "GSM7830139", "GSM7830140",
"GSM7830141", "GSM7830142", "GSM7830143", "GSM7830144", "GSM7830145", "GSM7830146",
"GSM7830147", "GSM7830148", "GSM7830149", "GSM7830150", "GSM7830151", "GSM7830152",
"GSM7830153", "GSM7830154", "GSM7830155", "GSM7830156", "GSM7830157", "GSM7830158",
"GSM7830159", "GSM7830160", "GSM7830161", "GSM7830162", "GSM7830163", "GSM7830164",
"GSM7830165", "GSM7830166", "GSM7830167")
columnas_control <- head(GSE244807_samples, 120)
columnas_caso <- tail(GSE244807_samples, 121)

# --- Firma Génica TGF-β Pro-Tumorigénica ---
genes_sobreexp <-
c("PMEPA1", "BHLHE40", "ADAM19", "SERPINE1", "EPHB2", "TGFB1", "JUNB", "LTBP2", "SEMA7A", "COL7A1",
"AMIGO2", "SH3KBP1", "SMAD7", "SPHK1", "NUAK1", "LIPG", "VDR", "ZSWIM4", "SKIL", "MLLT11", "DCBLD1",
"FOXP1", "COL1A1", "PDLIM7", "PNMA1", "ITGA5", "CDH2", "NAV1", "SLC19A2", "DSE", "SNAI1", "LPXN",
"CSRNP1", "PORCN", "SGK1", "TUFT1", "ANKLE2", "ZNF697", "BMP1", "MYORG", "PTPRK", "ENSG00000290569",
"GRB10", "ENSG00000274265", "MLXIP", "PODXL", "TPM1", "IL11", "TMCC1", "PID1", "NEDD9", "COL16A1",
"ENSG00000275993", "PCDH1", "MMP14", "GALNT10", "DLC1", "CHST11", "NOL4L", "PTPRE", "ENSG00000285517",
"CCN2", "CLTCL1", "SPSB1", "GADD45B", "MIR193BHG", "PHLDB1", "ITGB5", "SPECC1", "MICAL2", "ETV6",
"SIK1", "RALA", "VEGFA", "FHL3", "PPARD", "NT5DC2", "MARCKSL1", "PFKFB4", "ITGA2", "HIVEP1",
"INHBA", "ADAMTS6", "SRRD", "PACS1", "MT1X", "CDC42SE1", "TSPAN14", "PGRMC2", "MSANTD3", "PSMD2",
POMT2", "LIF", "ANKRD37", "SLC29A1", "PLEK2", "HAPLN3", "ADAM12", "ABCA1", "FSTL3", "MRC2", "UBASH3",
"PLPP4", "IGF1R", "HHAT", "GLI2", "LIMS1", "SGPL1", "METRNL", "IFFO2", "MRAS", "KDM5B", "MARCHF4",
"KRT7", "KCTD11", "P4HA2", "HSF2BP", "DYSF", "THBS1", "ENSG00000279821", "SERPINE2", "IGSF3", "N",
KILA", "BPGM", "LMCD1", "CSF1R", "SPOCD1", "KLF7", "SLC26A2", "ULK1", "SMOX", "MANF", "EDN1", "FAM21",
4B", "MIR22HG", "RNF121", "PXDC1", "B4GALT1", "NDST1", "TARS3", "VMP1", "ABLM3", "CLN8", "CGB8", "S",
OX4", "PPP1R14BAS1", "OCIAD2", "FBLIM1", "ENSG00000270504", "PPP1R3B", "KIRREL1", "ANXA8L1", "SSR",
3", "PFKFB3", "COL4A2", "COL6A3", "UCK2", "NET1", "ARHGEF40", "ZC3H12ADT", "LFNG", "SLC35F2", "ARNT",
L", "DNAJB5", "INAVA", "MBOAT2")
)
genes_infraexp <- c("CITED2", "SPRY1", "PHLPP1", "TBC1D8", "HPSE", "ZNF395", "TGFB3",
"MAP3K5", "ARRB2", "BMP4", "DUSP4", "KCNAB2", "DUSP16", "ANK1", "IL6R", "TRERF1",
"GPAT3", "MBP", "KLHL4", "RIN1", "IFIT3", "THRB", "AVPI1", "PLPP3", "CNKSR3", "MAPRE2",
"CCN3", "C1orf226", "OAS3", "AK5", "TFPI", "PHF19", "ASIC1", "TSPAN4", "JADE2",
"ZFP36L2", "MGAT5", "SAMHD1", "ALDH3A2", "CAT", "GSR", "KITLG", "RBM47", "ATP8B1",
"CABLES1", "PEG10", "CDKN2C", "DBP", "IFIT1", "CTNNAL1", "BCAM", "NCEH1", "ADI1", "VSIR",
"SSBP2", "ARVCF", "SLITRK5", "GPRC5A", "NMT2", "AGFG2", "FOSL1", "SH2D5", "TM4SF1",
"AOX1", "PPL", "APLN", "MYPN", "ANKRD33B", "ADORA2B", "CAMK2D", "PARP10", "KIAA1217",
"ALDH3B1", "MCCC1", "EPB41L4B", "BTN3A1", "MAP2", "BTN3A2", "TJP2", "PRECSIT", "TMC04",
"TXNRD1", "CYB5A", "LPAR1", "LRRC20", "KHK", "MYOF", "ZFP36", "SECTM1", "CD14", "SYNE3",
"TRNP1", "CXCL16", "CREB5", "NXN", "DENND3", "CFB", "DEPP1", "FLVCR2", "LIMD1", "PLK2",
"MITF", "CEBPD", "PHLDA1", "TXNIP", "PRICKLE1", "ERRFI1", "CTDSPL", "MYO18A")

```

```

# =====
# 1. CÁLCULO DEL LOG2FC y MAPEO
#=====
tryCatch({
  matriz_exp_full <- read.csv("matriz_corregida_ComBat.csv", row.names = 1, check.names =
FALSE)
}, error = function(e) {
  stop("Error: No se puede leer 'matriz_corregida_ComBat.csv'. Asegúrate de que el
archivo existe en el directorio de trabajo.")
})

# 1. Cálculo de Log2FC a nivel de Símbolo Génico
de_results <- matriz_exp_full %>%
  dplyr::select(all_of(GSE244807_samples)) %>%
  rownames_to_column(var = "Symbol") %>%
  dplyr::rowwise() %>%
  dplyr::mutate(
    media_control = mean(c_across(all_of(columnas_control)), na.rm = TRUE),
    media_caso = mean(c_across(all_of(columnas_caso)), na.rm = TRUE),
    # Se añade un pequeño pseudoconteo para evitar log2(0)
    log2FC = log2((media_caso + 1e-5) / (media_control + 1e-5))
  ) %>%
  dplyr::select(Symbol, log2FC) %>%
  dplyr::ungroup()

# 2. Mapeo a ENTREZID y cálculo del logFC promedio si hay duplicados
mapped_full_list <- bitr(de_results$Symbol, fromType = "SYMBOL", toType = "ENTREZID",
OrgDb = org.Hs.eg.db, drop = TRUE)

mapped_full_list <- mapped_full_list %>%
  dplyr::inner_join(de_results, by = c("SYMBOL" = "Symbol")) %>%
  dplyr::group_by(ENTREZID) %>%
  dplyr::summarise(log2FC = mean(log2FC), .groups = 'drop') %>%
  dplyr::ungroup()

# Lista final de LogFCs (Entrez ID como nombres) - MANTENIDA PERO NO USADA DESPUÉS
full_log2fc_list <- mapped_full_list$log2FC
names(full_log2fc_list) <- mapped_full_list$ENTREZID

# 3. Mapeo de Las Listas UP y DOWN (Subconjunto)
entrez_ids_up <- bitr(genes_sobreexp, fromType = "SYMBOL", toType =

genes_up_entrez <- entrez_ids_up$ENTREZID

entrez_ids_down <- bitr(genes_infraexp, fromType = "SYMBOL", toType = "ENTREZID", OrgDb =
org.Hs.eg.db, drop = TRUE)

genes_down_entrez <- entrez_ids_down$ENTREZID

genes_total_entrez <- unique(c(genes_up_entrez, genes_down_entrez))

# =====
# 2. FUNCIÓN CENTRAL DE ENRIQUECIMIENTO (Solo añade FE)
# =====
run_enrichment_analysis <- function(gene_list, category, title_prefix, N_plot,
logFCs_completo) {

  if (length(gene_list) < 10) {
    cat(paste("ADVERTENCIA: Menos de 10 genes en la lista", title_prefix, ". Se salta el
análisis", category, ".\n"))
  }
}

```

```

    return(NULL)
}

# Parámetros de enriquecimiento condicionales
p_cutoff_standard <- 0.05
q_cutoff_standard <- 0.2
p_cutoff_pathway <- 0.5
q_cutoff_pathway <- 0.5

current_p_cutoff <- if (category %in% c("KEGG", "Reactome")) p_cutoff_pathway else
p_cutoff_standard
current_q_cutoff <- if (category %in% c("KEGG", "Reactome")) q_cutoff_pathway else
q_cutoff_standard

if (category %in% c("KEGG", "Reactome")) {
  cat(paste("NOTA: Aplicando umbrales", category, "menos estrictos (p <",
current_p_cutoff, ", q <", current_q_cutoff, ").\n"))
}

# Ejecución del enriquecimiento
res <- NULL
if (category == "GO") {
  res <- enrichGO(gene = gene_list, OrgDb = org.Hs.eg.db,
                  ont = "ALL", pAdjustMethod = "BH", pvalueCutoff = current_p_cutoff,
                  qvalueCutoff = current_q_cutoff, readable = TRUE)
  color_opt <- "A"
} else if (category == "KEGG") {
  res <- enrichKEGG(gene = gene_list, organism = 'hsa',
                    pAdjustMethod = "BH", pvalueCutoff = current_p_cutoff, qvalueCutoff
= current_q_cutoff)
  color_opt <- "D"
} else if (category == "Reactome") {
  res <- ReactomePA::enrichPathway(gene = gene_list, organism = "human",
                                   pAdjustMethod = "BH", pvalueCutoff =
current_p_cutoff, qvalueCutoff = current_q_cutoff)
  color_opt <- "E"
} else {
  stop("Categoría no válida. Usa 'GO', 'KEGG' o 'Reactome'.")
}

# Post-procesamiento
if (!is.null(res) && nrow(as.data.frame(res)) > 0) {

  # Hacer legible el resultado de KEGG/Reactome
  if (category %in% c("KEGG", "Reactome")) {
    res <- setReadable(res, OrgDb = org.Hs.eg.db, keyType = "ENTREZID")
  }

  # 1. Guardar TODOS los términos enriquecidos
  df_all_enriched <- as.data.frame(res)

  # ==> CÁLCULO CRUCIAL DE MÉTRICA ADICIONAL: FoId Enrichment (FE) <==
  df_all_enriched <- df_all_enriched %>%
    # 1. Calcular Fold Enrichment (FE)
    rowwise() %>%
    dplyr::mutate(
      # Convertir GeneRatio y BgRatio a valores numéricos
      GeneRatio_Term = as.numeric(str_split(GeneRatio, "/")[[1]][1]) /
as.numeric(str_split(GeneRatio, "/")[[1]][2]),

```

```

        BgRatio_Term = as.numeric(str_split(BgRatio, "/")[[1]][1]) /
as.numeric(str_split(BgRatio, "/")[[1]][2]),
        FoldEnrichment = GeneRatio_Term / BgRatio_Term
    ) %>%
    # 2. Limpiar columnas temporales/intermedias
    dplyr::select(-GeneRatio_Term, -BgRatio_Term) %>%
    ungroup()

    cat(paste(" Guardados", nrow(df_all_enriched), "términos", category, title_prefix,
"en un dataframe ('_all') con FE.\n"))

    # 3. Visualización: Dotplot estándar de clusterProfiler (usa p.adjust en color)
    p <- dotplot(res, showCategory = N_plot,
        title = paste(category, "Enriquecido (Top", N_plot, ") -",
title_prefix)) +
        scale_y_discrete(labels = function(x) str_wrap(x, width = 45)) +
        scale_color_viridis_c(option = color_opt) +
        theme(plot.title = element_text(hjust = 0.5, face = "bold"),
            axis.text.y = element_text(size = 8))
    print(p)
    cat(paste("✅ Dot Plot", category, title_prefix, "generado mostrando", N_PLOT,
"categorías (Color: p.adjust).\n"))

    # Crear un nuevo objeto de enriquecimiento con la métrica FE añadida en el resultado.
    res_with_metrics <- res
    res_with_metrics@result <- df_all_enriched

    return(list(result_full = res_with_metrics, df_all_enriched = df_all_enriched))
} else {
    cat(paste("ADVERTENCIA: No se encontraron términos", category, "significativamente
enriquecidos", title_prefix, ".\n"))
    return(NULL)
}
}

# =====
# 3. EJECUCIÓN DEL ANÁLISIS
# =====
cat("\n--- 3. Ejecución del Análisis de Enriquecimiento (GO, KEGG, Reactome) ---\n")

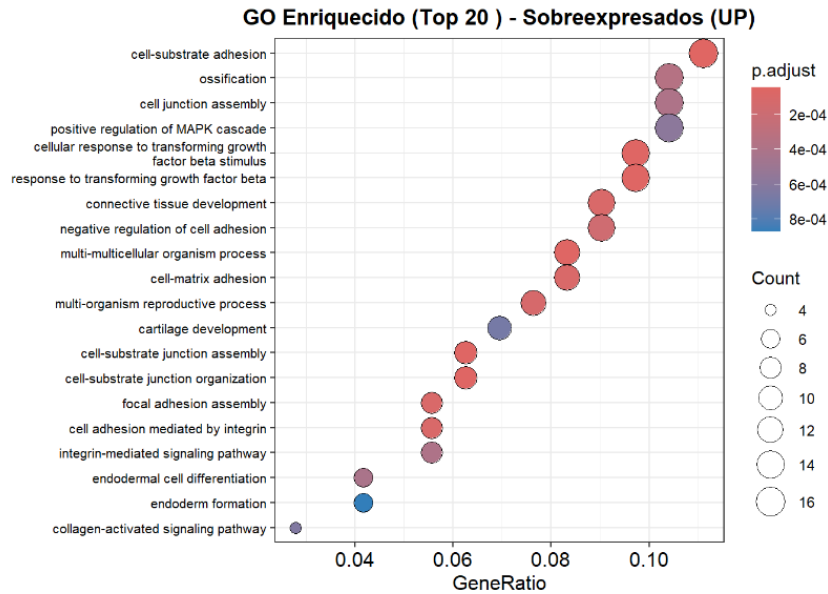
##
## --- 3. Ejecución del Análisis de Enriquecimiento (GO, KEGG, Reactome) ---

# NOTA: Se sigue pasando full_log2fc_list, pero la función lo ignora.
results_up <- list()
results_down <- list()
results_total <- list()

for(cat in CATEGORIES) {
    results_up[[cat]] <- run_enrichment_analysis(genes_up_entrez, cat, "Sobreexpresados
(UP)", N_PLOT, full_log2fc_list)
    results_down[[cat]] <- run_enrichment_analysis(genes_down_entrez, cat, "Infraexpresados
(DOWN)", N_PLOT, full_log2fc_list)
    results_total[[cat]] <- run_enrichment_analysis(genes_total_entrez, cat, "Totales
(UP+DOWN)", N_PLOT, full_log2fc_list)
}

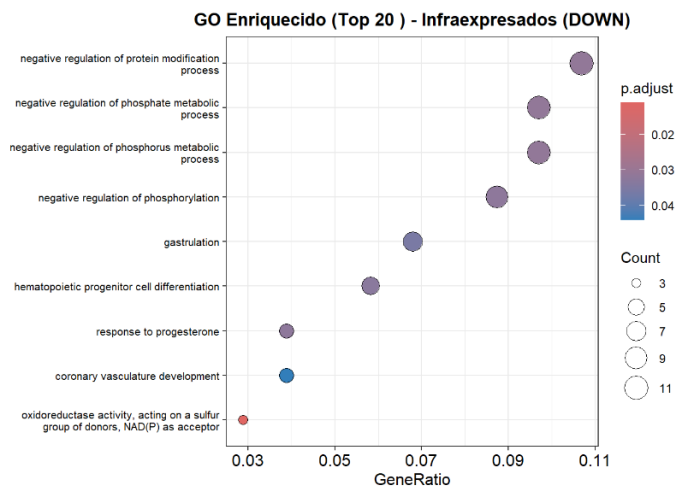
## ✅ Guardados 236 términos GO Sobreexpresados (UP) en un dataframe

```



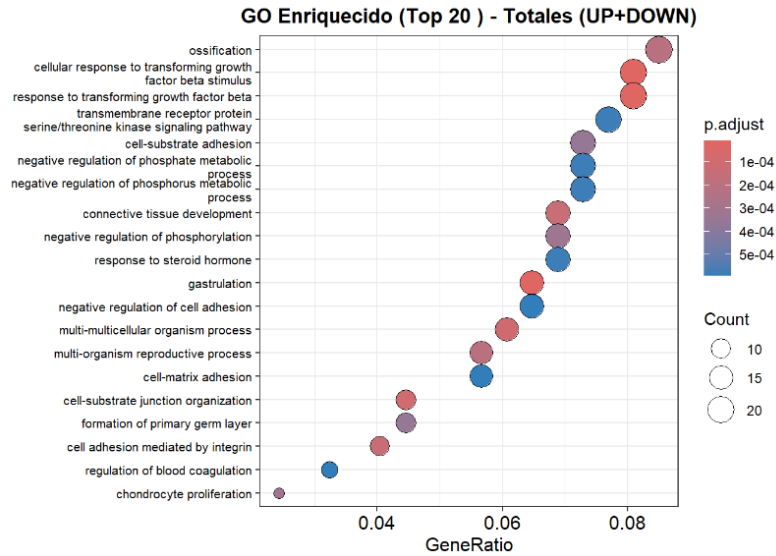
☒ Dot Plot GO Sobreexpresados (UP) generado mostrando 20 categorías

☒ Guardados 9 términos GO Infraexpresados (DOWN) en un dataframe



☒ Dot Plot GO Infraexpresados (DOWN) generado mostrando 20 categorías

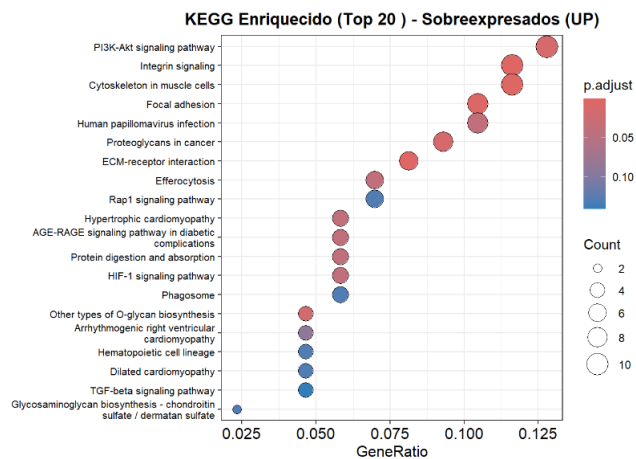
☒ Guardados 337 términos GO Totales (UP+DOWN) en un dataframe



☒ Dot Plot GO Totales (UP+DOWN) generado mostrando 20 categorías

NOTA: Aplicando umbrales KEGG menos estrictos ($p < 0.5$, $q < 0.5$).

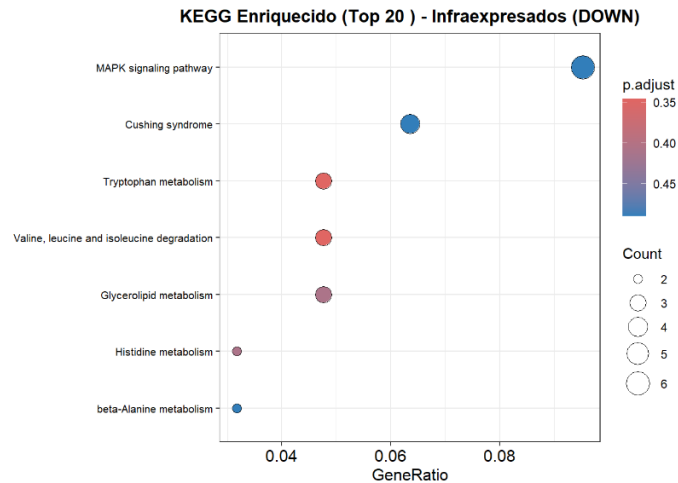
☒ Guardados 59 términos KEGG Sobreexpresados (UP) en un dataframe



☒ Dot Plot KEGG Sobreexpresados (UP) generado mostrando 20

NOTA: Aplicando umbrales KEGG menos estrictos ($p < 0.5$, $q < 0.5$).

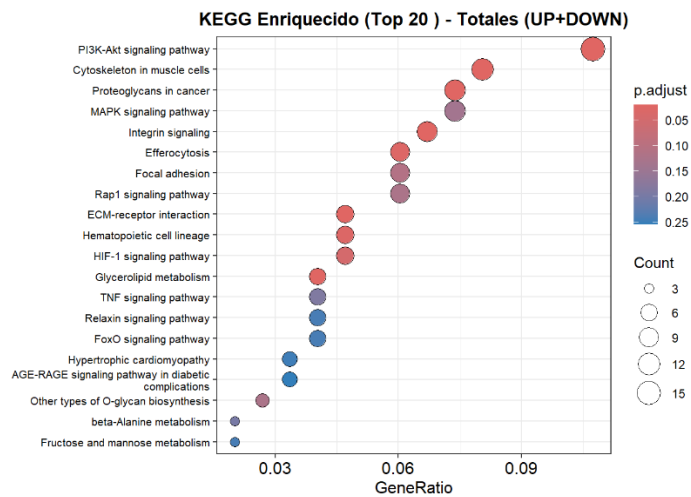
☒ Guardados 7 términos KEGG Infraexpresados (DOWN) en un dataframe



☒ Dot Plot KEGG Infraexpresados (DOWN) generado mostrando 20 categorías

NOTA: Aplicando umbrales KEGG menos estrictos ($p < 0.5$, $q < 0.5$).

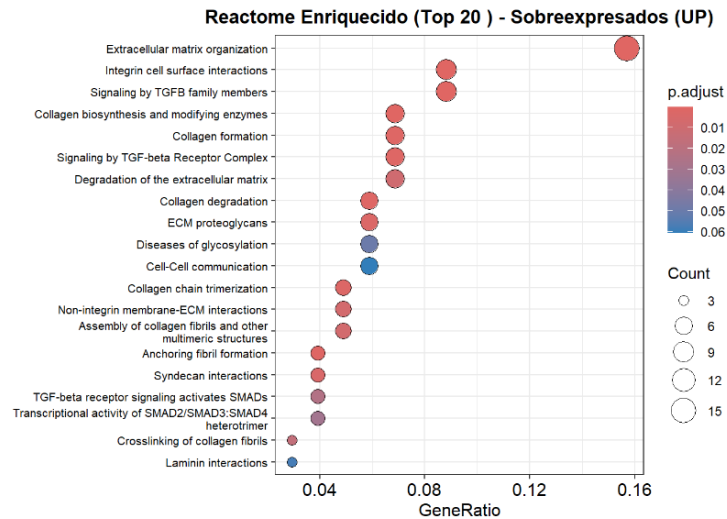
☒ Guardados 58 términos KEGG Totales (UP+DOWN) en un dataframe



☒ Dot Plot KEGG Totales (UP+DOWN) generado mostrando 20 categorías

NOTA: Aplicando umbrales Reactome menos estrictos ($p < 0.5$, $q < 0.5$)

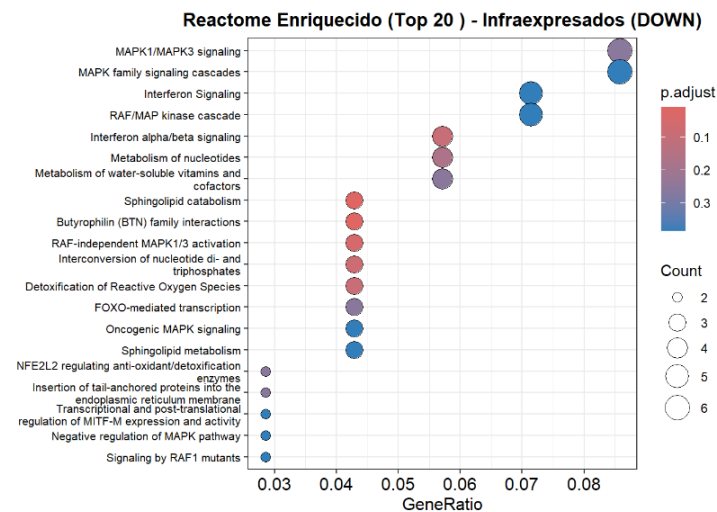
☒ Guardados 124 términos Reactome Sobreexpresados (UP) en un dataframe



☒ Dot Plot Reactome Sobreexpresados (UP) generado mostrando 20 categorías

NOTA: Aplicando umbrales Reactome menos estrictos ($p < 0.5$, $q < 0.5$)

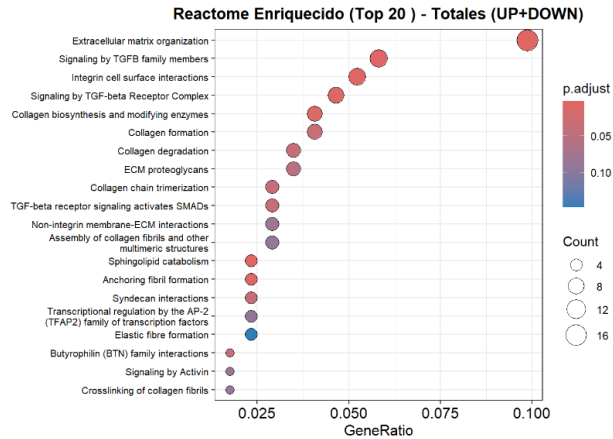
☒ Guardados 233 términos Reactome Infraexpresados (DOWN) en un dataframe



☒ Dot Plot Reactome Infraexpresados (DOWN) generado mostrando 20

NOTA: Aplicando umbrales Reactome menos estrictos ($p < 0.5$, $q < 0.5$)

☒ Guardados 74 términos Reactome Totales (UP+DOWN) en un dataframe



☒ Dot Plot Reactome Totales (UP+DOWN) generado mostrando 20 categorías

Extraer Los dataframes para el guardado y Los gráficos

```
df_go_up_all <- if (!is.null(results_up[["GO"]])) results_up[["GO"]]df_all_enriched else
NULL
```

```
df_go_down_all <- if (!is.null(results_down[["GO"]]))
```

```
results_down[["GO"]]df_all_enriched else NULL
```

```
df_kegg_up_all <- if (!is.null(results_up[["KEGG"]]))
```

```
results_up[["KEGG"]]df_all_enriched else NULL
```

```
df_kegg_down_all <- if (!is.null(results_down[["KEGG"]]))
```

```
results_down[["KEGG"]]df_all_enriched else NULL
```

```
df_reactome_up_all <- if (!is.null(results_up[["Reactome"]]))
```

```
results_up[["Reactome"]]df_all_enriched else NULL
```

```
df_reactome_down_all <- if (!is.null(results_down[["Reactome"]]))
```

```
results_down[["Reactome"]]df_all_enriched else NULL
```

```
df_go_total_all <- if (!is.null(results_total[["GO"]]))
```

```
results_total[["GO"]]df_all_enriched else NULL
```

```
df_kegg_total_all <- if (!is.null(results_total[["KEGG"]]))
```

```
results_total[["KEGG"]]df_all_enriched else NULL
```

```
df_reactome_total_all <- if (!is.null(results_total[["Reactome"]]))
```

```
results_total[["Reactome"]]df_all_enriched else NULL
```

```
# =====
```

```
# 4. GUARDAR RESULTADOS EN CSV (DOBLE ORDENACIÓN)
```

```
# =====
```

```
df_list_map <- list(
```

```
  "GO_UP" = df_go_up_all, "KEGG_UP" = df_kegg_up_all, "REACTOME_UP" = df_reactome_up_all,
```

```
  "GO_DOWN" = df_go_down_all, "KEGG_DOWN" = df_kegg_down_all, "REACTOME_DOWN" =
```

```
df_reactome_down_all,
```

```
  "GO_TOTAL" = df_go_total_all, "KEGG_TOTAL" = df_kegg_total_all, "REACTOME_TOTAL" =
```

```
df_reactome_total_all
```

```
)
```

```
for (name in names(df_list_map)) {
```

```
  df_to_save <- df_list_map[[name]]
```

```
  if (!is.null(df_to_save) && nrow(df_to_save) > 0) {
```

```
    # 1. ORDENACIÓN POR FOLD ENRICHMENT (Descendente)
```

```
    if ("FoldEnrichment" %in% names(df_to_save)) {
```

```
      df_fe_sorted <- df_to_save %>%
```

```
        dplyr::arrange(desc(FoldEnrichment))
```

```
      file_path_fe <- file.path(output_dir_csv, paste0("1_FE_ORDENADO_", name, ".csv"))
```

```
      write.csv(df_fe_sorted, file = file_path_fe, row.names = FALSE)
```

```

    cat(paste0("✅ CSV Guardado (Fold Enrichment): ", file_path_fe, "\n"))
  }

  # 2. ORDENACIÓN POR P.ADJUST (Ascendente)
  if ("p.adjust" %in% names(df_to_save)) {
    df_p_sorted <- df_to_save %>%
      dplyr::arrange(p.adjust)
    file_path_p <- file.path(output_dir_csv, paste0("2_PVALOR_ORDENADO_", name,
".csv"))
    write.csv(df_p_sorted, file = file_path_p, row.names = FALSE)
    cat(paste0("✅ CSV Guardado (P.Adjust): ", file_path_p, "\n"))
  }

  } else {
    cat(paste0("ADVERTENCIA: [", name, "] - El dataframe está vacío o no existe. No se
generó CSV.\n"))
  }
}

## ✅ CSV Guardado (Fold Enrichment):
resultados_enriquecimiento_detallado/1_FE_ORDENADO_GO_UP.csv
## ✅ CSV Guardado (P.Adjust):
resultados_enriquecimiento_detallado/2_PVALOR_ORDENADO_GO_UP.csv
## ✅ CSV Guardado (Fold Enrichment):
resultados_enriquecimiento_detallado/1_FE_ORDENADO_KEGG_UP.csv
## ✅ CSV Guardado (P.Adjust):
resultados_enriquecimiento_detallado/2_PVALOR_ORDENADO_KEGG_UP.csv
## ✅ CSV Guardado (Fold Enrichment):
resultados_enriquecimiento_detallado/1_FE_ORDENADO_REACTOME_UP.csv
## ✅ CSV Guardado (P.Adjust):
resultados_enriquecimiento_detallado/2_PVALOR_ORDENADO_REACTOME_UP.csv
## ✅ CSV Guardado (Fold Enrichment):
resultados_enriquecimiento_detallado/1_FE_ORDENADO_GO_DOWN.csv
## ✅ CSV Guardado (P.Adjust):
resultados_enriquecimiento_detallado/2_PVALOR_ORDENADO_GO_DOWN.csv
## ✅ CSV Guardado (Fold Enrichment):
resultados_enriquecimiento_detallado/1_FE_ORDENADO_KEGG_DOWN.csv
## ✅ CSV Guardado (P.Adjust):
resultados_enriquecimiento_detallado/2_PVALOR_ORDENADO_KEGG_DOWN.csv
## ✅ CSV Guardado (Fold Enrichment):
resultados_enriquecimiento_detallado/1_FE_ORDENADO_REACTOME_DOWN.csv
## ✅ CSV Guardado (P.Adjust):
resultados_enriquecimiento_detallado/2_PVALOR_ORDENADO_REACTOME_DOWN.csv
## ✅ CSV Guardado (Fold Enrichment):
resultados_enriquecimiento_detallado/1_FE_ORDENADO_GO_TOTAL.csv
## ✅ CSV Guardado (P.Adjust):
resultados_enriquecimiento_detallado/2_PVALOR_ORDENADO_GO_TOTAL.csv
## ✅ CSV Guardado (Fold Enrichment):
resultados_enriquecimiento_detallado/1_FE_ORDENADO_KEGG_TOTAL.csv
## ✅ CSV Guardado (P.Adjust):
resultados_enriquecimiento_detallado/2_PVALOR_ORDENADO_KEGG_TOTAL.csv
## ✅ CSV Guardado (Fold Enrichment):
resultados_enriquecimiento_detallado/1_FE_ORDENADO_REACTOME_TOTAL.csv
## ✅ CSV Guardado (P.Adjust):
resultados_enriquecimiento_detallado/2_PVALOR_ORDENADO_REACTOME_TOTAL.csv

```

12. ANÁLISIS DE ENRIQUECIMIENTO DE GENES ENTRE LOS GRUPOS TGF-B UPREGULATED VS TGF-B DOWNREGULATED MEDIANTE GSEA

```
# 0. LIBRERÍAS Y CONFIGURACIÓN INICIAL
suppressPackageStartupMessages({
  library(tidyverse)
  library(clusterProfiler)
  library(org.Hs.eg.db)
  library(enrichplot)
  library(ggplot2)
  library(DOSE)
  library(stringr)
  library(ReactomePA)
  library(ggribes)
})

# --- Configuración CRAN y Parámetros ---
options(repos = c(CRAN = "https://cran.rstudio.com/"))

N_PLOT <- 20
N_PLOT_CURVE <- 8 # Número de curvas para gseaplot2
CATEGORIES <- c("GO", "KEGG", "Reactome")
INPUT_FILE <- "matriz_corregida_ComBat.csv"

# >>> DETALLE CLAVE: DEFINICIÓN EXPLÍCITA DEL ANÁLISIS <<<
ANALYSIS_NAME_FULL <- "TGFB_ESTIMULADO_VS_CONTROL"

# --- Rutas de Directorio ---
output_dir_csv_full <- paste0("resultados_gsea_detallado_", ANALYSIS_NAME_FULL)
output_dir_plots_dot_full <- paste0("resultados_graficos_dot_plots_gsea_individual_",
ANALYSIS_NAME_FULL)
output_dir_plots_gsea_clasico_full <- paste0("resultados_graficos_gsea_clasico_",
ANALYSIS_NAME_FULL)

# --- CREACIÓN ROBUSTA DE DIRECTORIOS ---
output_dirs_full <- c(output_dir_csv_full, output_dir_plots_dot_full,
output_dir_plots_gsea_clasico_full)
for (dir_path in output_dirs_full) {
  if (!dir.exists(dir_path)) {
    success <- dir.create(dir_path, recursive = TRUE)
    if (!success) {
      stop(paste0("ERROR FATAL: No se pudo crear el directorio de salida: ",
dir_path, ".\nVerifica los permisos de escritura. "))
    } else {
      cat(paste0("Directorio creado: ", dir_path, "\n"))
    }
  } else {
    cat(paste0("[i] Directorio existente: ", dir_path, "\n"))
  }
}

## [i] Directorio existente: resultados_gsea_detallado_TGFB_ESTIMULADO_VS_CONTROL
## [i] Directorio existente:
resultados_graficos_dot_plots_gsea_individual_TGFB_ESTIMULADO_VS_CONTROL
## [i] Directorio existente: resultados_graficos_gsea_clasico_TGFB_ESTIMULADO_VS_CONTROL
```

```

# --- Definición de Muestras ---
# Lista completa de muestras (241 muestras)
GSE244807_samples <- c("GSM7829922", "GSM7829924", "GSM7829925", "GSM7829926",
"GSM7829927", "GSM7829928", "GSM7829929", "GSM7829930", "GSM7829931", "GSM7829932",
"GSM7829933", "GSM7829934", "GSM7829935", "GSM7829936", "GSM7829937", "GSM7829938",
"GSM7829939", "GSM7829940", "GSM7829941", "GSM7829942", "GSM7829943", "GSM7829944",
"GSM7829945", "GSM7829947", "GSM7829948", "GSM7829949", "GSM7829950", "GSM7829951",
"GSM7829952", "GSM7829954", "GSM7829955", "GSM7829956", "GSM7829957", "GSM7829958",
"GSM7829959", "GSM7829960", "GSM7829961", "GSM7829962", "GSM7829963", "GSM7829964",
"GSM7829965", "GSM7829966", "GSM7829967", "GSM7829968", "GSM7829969", "GSM7829970",
"GSM7830007", "GSM7830008", "GSM7830009", "GSM7830010", "GSM7830011", "GSM7830012",
"GSM7830013", "GSM7830014", "GSM7830015", "GSM7830016", "GSM7830017", "GSM7830018",
"GSM7830019", "GSM7830020", "GSM7830021", "GSM7830022", "GSM7830023", "GSM7830024",
"GSM7830025", "GSM7830026", "GSM7830027", "GSM7830028", "GSM7830029", "GSM7830030",
"GSM7830031", "GSM7830032", "GSM7830033", "GSM7830034", "GSM7830035", "GSM7830036",
"GSM7830037", "GSM7830038", "GSM7830039", "GSM7830040", "GSM7830041", "GSM7830042",
"GSM7830043", "GSM7830044", "GSM7830045", "GSM7830046", "GSM7830047", "GSM7830048",
"GSM7830049", "GSM7830050", "GSM7830051", "GSM7830052", "GSM7830053", "GSM7830054",
"GSM7830055", "GSM7830056", "GSM7830057", "GSM7830058", "GSM7830059", "GSM7830060",
"GSM7830061", "GSM7830062", "GSM7830063", "GSM7830064", "GSM7830065", "GSM7830066",
"GSM7830067", "GSM7830068", "GSM7830069", "GSM7830070", "GSM7830071", "GSM7830072",
"GSM7830073", "GSM7830074", "GSM7830075", "GSM7830076", "GSM7830077", "GSM7830078",
"GSM7830079", "GSM7830080", "GSM7830081", "GSM7830082", "GSM7830083", "GSM7830084",
"GSM7830085", "GSM7830087", "GSM7830088", "GSM7830089", "GSM7830090", "GSM7830091",
"GSM7830092", "GSM7830093", "GSM7830094", "GSM7830095", "GSM7830096", "GSM7830097",
"GSM7830098", "GSM7830099", "GSM7830100", "GSM7830101", "GSM7830102", "GSM7830103",
"GSM7830104", "GSM7830105", "GSM7830106", "GSM7830107", "GSM7830108", "GSM7830109",
"GSM7830110", "GSM7830111", "GSM7830112", "GSM7830113", "GSM7830114", "GSM7830115",
"GSM7830116", "GSM7830117", "GSM7830118", "GSM7830119", "GSM7830120", "GSM7830121",
"GSM7830122", "GSM7830123", "GSM7830124", "GSM7830125", "GSM7830126", "GSM7830127",
"GSM7830128", "GSM7830129", "GSM7830131", "GSM7830132", "GSM7830133", "GSM7830134",
"GSM7830135", "GSM7830136", "GSM7830137", "GSM7830138", "GSM7830139", "GSM7830140",
"GSM7830141", "GSM7830142", "GSM7830143", "GSM7830144", "GSM7830145", "GSM7830146",
"GSM7830147", "GSM7830148", "GSM7830149", "GSM7830150", "GSM7830151", "GSM7830152",
"GSM7830153", "GSM7830154", "GSM7830155", "GSM7830156", "GSM7830157", "GSM7830158",
"GSM7830159", "GSM7830160", "GSM7830161", "GSM7830162", "GSM7830163", "GSM7830164",
"GSM7830165", "GSM7830166", "GSM7830167")

# Separación en grupos (120 Control, 121 Caso)
columnas_control <- head(GSE244807_samples, 120)
columnas_caso <- tail(GSE244807_samples, 121)

# =====
# 1. CÁLCULO DEL T-SCORE DE WELCH Y LISTA RANQUEADA
# =====
cat(paste0("\n--- 1. Cálculo del T-Score de Welch y lista ranqueada (TODOS LOS GENES) ---\n"))

##
## --- 1. Cálculo del T-Score de Welch y lista ranqueada (TODOS LOS GENES) ---

tryCatch({
  matriz_exp_full <- read.csv(INPUT_FILE, row.names = 1, check.names = FALSE)
}, error = function(e) {
  stop("Error: No se puede leer 'matriz_corregida_ComBat.csv'. Asegúrate de que el
archivo existe en el directorio de trabajo.")
})

# 1.1. FILTRO DE EXPRESIÓN MEDIA Y CÁLCULO DEL T-SCORE DE WELCH
matriz_exp_full_filtered <- matriz_exp_full %>%

```

```

    dplyr::mutate(Mean_Expression = rowMeans(dplyr::select(., all_of(GSE244807_samples)),
na.rm = TRUE)) %>%
    dplyr::filter(Mean_Expression >= 1) %>%
    dplyr::select(-Mean_Expression)
cat(paste("Genes después del filtro de expresión media (>= 1):",
nrow(matriz_exp_full_filtered), "\n"))

## Genes después del filtro de expresión media (>= 1): 16742

de_results_robust <- matriz_exp_full_filtered %>%
    dplyr::select(all_of(GSE244807_samples)) %>%
    rownames_to_column(var = "Symbol") %>%
    dplyr::rowwise() %>%
    dplyr::mutate(
        mean_control = mean(c_across(all_of(columnas_control)), na.rm = TRUE),
        sd_control = sd(c_across(all_of(columnas_control)), na.rm = TRUE),
        n_control = sum(!is.na(c_across(all_of(columnas_control))))),
        mean_caso = mean(c_across(all_of(columnas_caso)), na.rm = TRUE),
        sd_caso = sd(c_across(all_of(columnas_caso)), na.rm = TRUE),
        n_caso = sum(!is.na(c_across(all_of(columnas_caso))))),

        # Error estándar de Welch (no asume varianzas iguales)
        SE_welch = sqrt((sd_control^2 / n_control) + (sd_caso^2 / n_caso)),

        # Cálculo del T-Score (Welch). Se añade 1e-5 al denominador para evitar
divisiones por cero.
        T_Score = (mean_caso - mean_control) / (SE_welch + 1e-5)
    ) %>%
    dplyr::select(Symbol, T_Score) %>%
    dplyr::ungroup()

# 1.2. Mapeo a ENTREZID y manejo de duplicados
mapped_full_list <- bitr(de_results_robust$Symbol, fromType = "SYMBOL", toType =
"ENTREZID", OrgDb = org.Hs.eg.db, drop = TRUE)

mapped_full_list <- mapped_full_list %>%
    dplyr::inner_join(de_results_robust, by = c("SYMBOL" = "Symbol")) %>%
    # Usar la media del T-Score para símbolos duplicados mapeados al mismo ENTREZID
    dplyr::group_by(ENTREZID) %>%
    dplyr::summarise(T_Score = mean(T_Score), .groups = 'drop') %>%
    dplyr::ungroup()

# 1.3. CREAR LA LISTA RANQUEADA FINAL
full_t_score_list <- mapped_full_list$T_Score
names(full_t_score_list) <- mapped_full_list$ENTREZID

# --- CORRECCIÓN: ROMPER EMPATES (Añadir ruido para GSEA) ---
set.seed(42)
random_noise <- runif(length(full_t_score_list), -1e-8, 1e-8)
full_t_score_list_robust <- full_t_score_list + random_noise

# ¡Ordenar DESCENDENTEMENTE! (Los genes más upregulated tienen el T-Score más alto y van
primero)
geneList_Full <- sort(full_t_score_list_robust, decreasing = TRUE)
cat(paste("Lista de genes ranqueada (T-Score de Welch + ruido) lista para GSEA.
Longitud:", length(geneList_Full), "\n"))

## Lista de genes ranqueada (T-Score de Welch + ruido) lista para GSEA. Longitud: 14906

```

1


```

# =====
# 2. FUNCIÓN CENTRAL DE ENRIQUECIMIENTO (GSEA - Direccional y Robusta)
# =====
run_gsea_analysis <- function(gene_rank_list, category, N_plot) {
  p_cutoff_gsea <- 0.25

  if (length(gene_rank_list) < 500) {
    cat(paste("ADVERTENCIA [", category, "]: La lista de rango de genes es pequeña
(", length(gene_rank_list), " genes). Se recomienda >500 genes.\n", sep=""))
  }

  gsea_res_raw <- NULL # Objeto GSEA sin modificar
  min_size <- ifelse(length(gene_rank_list) < 500, 5, 10)

  # 2.1. Ejecución del GSEA
  if (category == "GO") {
    gsea_res_raw <- gseGO(geneList = gene_rank_list, OrgDb = org.Hs.eg.db,
      ont = "ALL", pAdjustMethod = "BH", pvalueCutoff =
p_cutoff_gsea,
      minGSSize = min_size, maxGSSize = 500, verbose = FALSE)
  } else if (category == "KEGG") {
    gsea_res_raw <- gseKEGG(geneList = gene_rank_list, organism = 'hsa',
      pAdjustMethod = "BH", pvalueCutoff = p_cutoff_gsea,
      minGSSize = min_size, maxGSSize = 500, verbose = FALSE)
  } else if (category == "Reactome") {
    gsea_res_raw <- ReactomePA::gsePathway(geneList = gene_rank_list, organism =
"human",
      pAdjustMethod = "BH", pvalueCutoff =
p_cutoff_gsea,
      minGSSize = min_size, maxGSSize = 500,
      verbose = FALSE)
  } else {
    stop("Categoría no válida. Usa 'GO', 'KEGG' o 'Reactome'.")
  }

  # 2.2. Procesamiento del resultado (Manejo robusto para setReadable)
  if (!is.null(gsea_res_raw) && nrow(as.data.frame(gsea_res_raw)) > 0) {

    # Copia del objeto para hacerlo legible (necesario para CSV y Dot Plot)
    gsea_res_readable <- gsea_res_raw
    if (category %in% c("KEGG", "Reactome")) {
      gsea_res_readable <- setReadable(gsea_res_readable, OrgDb = org.Hs.eg.db,
keyType = "ENTREZID")
    }

    # Crear DataFrame final con dirección del enriquecimiento
    df_all_enriched <- as.data.frame(gsea_res_readable) %>%
      dplyr::mutate(Direction = ifelse(NES > 0, "ACTIVADA (Up-Regulated por Caso)",
"INHIBIDA (Down-Regulated por Caso)"))

    cat(paste("GSEA completado. Enriquecidos:", nrow(df_all_enriched), "términos",
category, "(NES calculado).\n"))

    # Devolver el resultado RAW (para gseaplot2) y el dataframe legible (para
CSV/plots)
    return(list(result_full_raw = gsea_res_raw, df_all_enriched = df_all_enriched))
  } else {
    cat(paste("ADVERTENCIA: No se encontraron términos", category,
"significativamente enriquecidos (GSEA).\n"))
    return(NULL)
  }
}

```

```

}

# =====
# 3. EJECUCIÓN DEL ANÁLISIS GSEA (TODOS LOS GENES)
# =====

results_gsea_full <- list()
for(cat in CATEGORIES) {
  results_gsea_full[[cat]] <- run_gsea_analysis(geneList_Full, cat, N_PLOT)
}

# Extraer dataframes para guardado
df_go_all_full <- if (!is.null(results_gsea_full[["GO"]]))
results_gsea_full[["GO"]]$df_all_enriched else NULL
df_kegg_all_full <- if (!is.null(results_gsea_full[["KEGG"]]))
results_gsea_full[["KEGG"]]$df_all_enriched else NULL
df_reactome_all_full <- if (!is.null(results_gsea_full[["Reactome"]]))
results_gsea_full[["Reactome"]]$df_all_enriched else NULL
# =====
# 4. GUARDAR RESULTADOS EN CSV (TODOS LOS GENES)
# =====

df_list_map_gsea_full <- list(
  "GO_GSEA" = df_go_all_full, "KEGG_GSEA" = df_kegg_all_full, "REACTOME_GSEA" =
df_reactome_all_full
)
for (name in names(df_list_map_gsea_full)) {
  df_to_save <- df_list_map_gsea_full[[name]]
  if (!is.null(df_to_save) && nrow(df_to_save) > 0) {

    if ("NES" %in% names(df_to_save)) {
      df_nes_sorted <- df_to_save %>% dplyr::arrange(desc(NES))
      file_path_nes <- file.path(output_dir_csv_full, paste0("1_NES_ORDENADO_",
ANALYSIS_NAME_FULL, "_", name, ".csv"))
      tryCatch({
        write.csv(df_nes_sorted, file = file_path_nes, row.names = FALSE)
        cat(paste0("CSV Guardado (NES): ", file_path_nes, "\n"))
      }, error = function(e) {
        cat(paste0("ERROR CRÍTICO al guardar CSV (NES) para ", name, ": ",
e$message, ". Cierra el archivo si está abierto.\n"))
      })
    }

    if ("p.adjust" %in% names(df_to_save)) {
      df_p_sorted <- df_to_save %>% dplyr::arrange(p.adjust)
      file_path_p <- file.path(output_dir_csv_full, paste0("2_PVALOR_ORDENADO_",
ANALYSIS_NAME_FULL, "_", name, ".csv"))
      tryCatch({
        write.csv(df_p_sorted, file = file_path_p, row.names = FALSE)
        cat(paste0("CSV Guardado (P.Adjust): ", file_path_p, "\n"))
      }, error = function(e) {
        cat(paste0("ERROR CRÍTICO al guardar CSV (P.Adjust) para ", name, ": ",
e$message, ". Cierra el archivo si está abierto.\n"))
      })
    }
  } else {
    cat(paste0("ADVERTENCIA: [", name, "] - El dataframe está vacío. No se generó
CSV.\n"))
  }
}
}

```

```

## CSV Guardado (NES):
resultados_gsea_detallado_TGFB_ESTIMULADO_VS_CONTROL/1_NES_ORDENADO_TGFB_ESTIMULADO_VS_CO
NTROL_GO_GSEA.csv
## cSV Guardado (P.Adjust):
resultados_gsea_detallado_TGFB_ESTIMULADO_VS_CONTROL/2_PVALOR_ORDENADO_TGFB_ESTIMULADO_VS
_CONTROL_GO_GSEA.csv
## CSV Guardado (NES):
resultados_gsea_detallado_TGFB_ESTIMULADO_VS_CONTROL/1_NES_ORDENADO_TGFB_ESTIMULADO_VS_CO
NTROL_KEGG_GSEA.csv
## cSV Guardado (P.Adjust):
resultados_gsea_detallado_TGFB_ESTIMULADO_VS_CONTROL/2_PVALOR_ORDENADO_TGFB_ESTIMULADO_VS
_CONTROL_KEGG_GSEA.csv
## CSV Guardado (NES):
resultados_gsea_detallado_TGFB_ESTIMULADO_VS_CONTROL/1_NES_ORDENADO_TGFB_ESTIMULADO_VS_CO
NTROL_REACTOME_GSEA.csv
## cSV Guardado (P.Adjust):
resultados_gsea_detallado_TGFB_ESTIMULADO_VS_CONTROL/2_PVALOR_ORDENADO_TGFB_ESTIMULADO_VS
_CONTROL_REACTOME_GSEA.csv

# =====
# 5. FUNCIÓN DE PLOTEO (Dot Plot)
# =====
# Nota: Usa el dataframe Legible que contiene la columna 'Description'
generate_gsea_dot_plot <- function(df_enriched, category, N_plot, output_dir,
analysis_name) {
  if (is.null(df_enriched) || nrow(df_enriched) == 0) {
    cat(paste("ADVERTENCIA: No se encontraron resultados enriquecidos para",
category, ".\n"))
    return(NULL)
  }

  # Usamos el DF Legible
  df_plot_data <- df_enriched %>% dplyr::arrange(p.adjust) %>% head(N_plot)

  df_plot_data <- df_plot_data %>%
    dplyr::mutate(Direction_Color = ifelse(NES > 0, "Activada (NES > 0) por Caso",
"Inhibida (NES < 0) por Caso"))

  plot_title <- paste(category, "GSEA Direccional (Top", nrow(df_plot_data), "vías)")
  p <- ggplot(df_plot_data, aes(x = NES, y = reorder(Description, NES))) +
    geom_point(aes(size = -log10(p.adjust), color = Direction_Color)) +
    scale_color_manual(values = c("Activada (NES > 0) por Caso" = "#E41A1C",
"Inhibida (NES < 0) por Caso" = "#377EB8"), name = "Dirección") +
    geom_vline(xintercept = 0, linetype = "dashed", color = "gray50") +
    labs(title = plot_title, subtitle = paste("Análisis:", analysis_name), x =
"Normalized Enrichment Score (NES)", y = NULL, size = "-log10(p.adjust)") +
    scale_y_discrete(labels = function(x) str_wrap(x, width = 45)) +
    theme_bw() +
    theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 12),
plot.subtitle = element_text(hjust = 0.5, size = 10),
axis.text.y = element_text(size = 8), legend.position = "bottom",
plot.margin = unit(c(0.5, 1, 0.5, 1), "cm"))

  plot_height <- max(6, nrow(df_plot_data) * 0.4)
  plot_filename <- file.path(output_dir, paste0("DotPlot_GSEA_", analysis_name, "_",
category, ".png"))

  tryCatch({
    ggsave(plot_filename, plot = p, width = 10, height = plot_height)
    if(interactive()) print(p)
    cat(paste0(" Dot Plot Direccional (GSEA) guardado para ", category, ": ",

```

```

plot_filename, "\n"))
  }, error = function(e) {
    cat(paste0(" ERROR CRÍTICO al guardar el Dot Plot para ", category, ": ",
e$message, ". Cierra el archivo .png si está abierto.\n"))
  })
  return(p)
}

# =====
# 6. FUNCIÓN DE PLOTEO (GSEA Plot Clásico)
# =====
# Nota: Usa el objeto RAW sin modificar para evitar fallos de gseaplot2
generate_gsea_enrichment_plot <- function(res_raw, category, N_plot_curve, output_dir,
analysis_name) {
  if (is.null(res_raw) || nrow(as.data.frame(res_raw)) == 0) {
    cat(paste("ADVERTENCIA: No hay resultados enriquecidos para generar la curva de
enriquecimiento para", category, ".\n"))
    return(NULL)
  }

  df_temp <- as.data.frame(res_raw) %>% dplyr::arrange(p.adjust)

  # Selecciona Los Top N (puede ser menos si hay menos enriquecidos)
  pathway_ids <- head(df_temp, N_plot_curve)$ID
  num_paths <- length(pathway_ids)
  if (num_paths == 0) return(NULL)

  # Definición de colores (hasta 8)
  plot_colors <- c("#E41A1C", "#377EB8", "#4DAF4A", "#984EA3", "#FF7F00", "#FFFF33",
"#A65628", "#F781BF")[1:num_paths]

  plot_title <- paste("Curva de Enriquecimiento GSEA (Top", num_paths, "vías) -",
category)

  p_gsea <- tryCatch({
    # CRÍTICO: Usar el objeto RAW sin modificar aquí
    p <- gseaplot2(x = res_raw, geneSetID = pathway_ids, title = plot_title, color =
plot_colors, pvalue_table = TRUE, subplots = 1:2)
    return(p)
  }, error = function(e) {
    cat(paste0("ERROR en gseaplot2 para ", category, ": ", e$message, "\n"))
    return(NULL)
  })

  if (!is.null(p_gsea) && "ggplot" %in% class(p_gsea)) {
    p_gsea <- p_gsea + labs(subtitle = paste("Análisis:", analysis_name)) +
theme(plot.subtitle = element_text(hjust = 0.5, size = 10))
  } else {
    cat(paste0("ADVERTENCIA: No se pudo generar el plot gseaplot2 para ", category,
".\n"))
    return(NULL)
  }

  plot_filename <- file.path(output_dir, paste0("GSEA_EnrichmentPlot_Top", num_paths,
"_", analysis_name, "_", category, ".png"))

  tryCatch({
    ggsave(plot_filename, plot = p_gsea, width = 12, height = 8)
    cat(paste0("GSEA Enrichment Plot (gseaplot2) guardado para ", category, ": ",
plot_filename, "\n"))
  }, error = function(e) {
    cat(paste0("ERROR al guardar el GSEA Enrichment Plot (gseaplot2) para ", category, ": ",
e$message, "\n"))
    return(NULL)
  })
}

```

```

        if(interactive()) print(p_gsea)

    }, error = function(e) {
        cat(paste0("ERROR CRÍTICO al guardar el plot de enriquecimiento para ", category,
": ", e$message, ". Cierra el archivo .png si está abierto.\n"))
    })
}
# -----
# 6.5. FUNCIÓN DE PLOTEO (NES Ridge Plot)
# -----
generate_gsea_ridge_plot <- function(df_enriched, category, output_dir, analysis_name) {
    if (is.null(df_enriched) || nrow(df_enriched) == 0) {
        cat(paste("ADVERTENCIA: No hay datos enriquecidos para generar el Ridge Plot
para", category, ".\n"))
        return(NULL)
    }

    p <- ggplot(df_enriched, aes(x = NES, y = "Vías Enriquecidas", fill = Direction)) +
        ggribes::geom_density_ridges(scale = 2, alpha = 0.7) +
        geom_vline(xintercept = 0, linetype = "dashed", color = "black") +
        scale_fill_manual(values = c("ACTIVADA (Up-Regulated por Caso)" = "#E41A1C",
"INHIBIDA (Down-Regulated por Caso)" = "#377EB8"),
name = "Dirección") +
        labs(title = paste("Distribución del NES (Ridge Plot) -", category),
            subtitle = paste("Análisis:", analysis_name),
            x = "Normalized Enrichment Score (NES)", y = NULL, fill = "Dirección") +
        theme_minimal() +
        theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 14),
            legend.position = "bottom")

    plot_filename <- file.path(output_dir, paste0("RidgePlot_NES_", analysis_name, "_",
category, ".png"))

    tryCatch({
        ggsave(plot_filename, plot = p, width = 10, height = 5)
        cat(paste0("Ridge Plot (NES) guardado para ", category, ": ", plot_filename,
"\n"))

        if(interactive()) print(p)

    }, error = function(e) {
        cat(paste0("ERROR CRÍTICO al guardar el Ridge Plot para ", category, ": ",
e$message, "\n"))
    })
}
# =====
# 7. EJECUCIÓN DE PLOTS GSEA (TODOS LOS GENES)
# =====
for (cat in CATEGORIES) {
    if (!is.null(results_gsea_full[[cat]])) {
        res_raw <- results_gsea_full[[cat]]$result_full_raw
        df_enriched_full <- results_gsea_full[[cat]]$df_all_enriched

        # Comprobar que hay resultados enriquecidos
        if (nrow(as.data.frame(res_raw)) > 0) {

            # 1. Dot Plot (Usa el dataframe Legible)
            generate_gsea_dot_plot(df_enriched_full, cat, N_PLOT,
output_dir_plots_dot_full, ANALYSIS_NAME_FULL)

            # 2. GSEA Plot Clásico (Usa el objeto RAW)

```

```

        generate_gsea_enrichment_plot(res_raw, cat, N_PLOT_CURVE,
output_dir_plots_gsea_clasico_full, ANALYSIS_NAME_FULL)

        # 3. Ridge Plot (Usa el dataframe legible)
        generate_gsea_ridge_plot(df_enriched_full, cat, output_dir_plots_dot_full,
ANALYSIS_NAME_FULL)
    } else {
        cat(paste("ADVERTENCIA: No hay vías enriquecidas significativas para", cat,
". Omitiendo plots.\n"))
    }
}
}

## Dot Plot Direccional (GSEA) guardado para GO:
resultados_graficos_dot_plots_gsea_individual_TGFB_ESTIMULADO_VS_CONTROL/DotPlot_GSEA_TGF
B_ESTIMULADO_VS_CONTROL_GO.png

## Picking joint bandwidth of 0.086

## Ridge Plot (NES) guardado para GO:
resultados_graficos_dot_plots_gsea_individual_TGFB_ESTIMULADO_VS_CONTROL/RidgePlot_NES_TG
FB_ESTIMULADO_VS_CONTROL_GO.png

## Dot Plot Direccional (GSEA) guardado para KEGG:
resultados_graficos_dot_plots_gsea_individual_TGFB_ESTIMULADO_VS_CONTROL/DotPlot_GSEA_TGF
B_ESTIMULADO_VS_CONTROL_KEGG.png

## Picking joint bandwidth of 0.25

## Ridge Plot (NES) guardado para KEGG:
resultados_graficos_dot_plots_gsea_individual_TGFB_ESTIMULADO_VS_CONTROL/RidgePlot_NES_TG
FB_ESTIMULADO_VS_CONTROL_KEGG.png

## Dot Plot Direccional (GSEA) guardado para Reactome:
resultados_graficos_dot_plots_gsea_individual_TGFB_ESTIMULADO_VS_CONTROL/DotPlot_GSEA_TGF
B_ESTIMULADO_VS_CONTROL_Reactome.png

## Picking joint bandwidth of 0.172

## Ridge Plot (NES) guardado para Reactome:
resultados_graficos_dot_plots_gsea_individual_TGFB_ESTIMULADO_VS_CONTROL/RidgePlot_NES_TG
FB_ESTIMULADO_VS_CONTROL_Reactome.png

```

13. ANALISIS DE SUPERVIVENCIA ESTRATIFICADO POR SEXO y EDAD

```

# =====
# 1. CARGA DE LIBRERÍAS Y CONFIGURACIÓN ESTÉTICA
# =====
library(survival)
library(survminer)
library(dplyr)
library(ggplot2)

# Colores profesionales
paleta_riesgo <- c("#2E9FDF", "#E7B800")
paleta_edad <- "jco"

```

```

# =====
# 2. PREPARACIÓN DE DATOS Y CÁLCULO DE SCORE
# =====
exp_file <- "matriz_corregida_ComBat.csv"
metadata_file <- "GSE244807_metadata_filtered.csv"

if (!file.exists(exp_file) | !file.exists(metadata_file)) stop("Faltan archivos CSV.")

expresion_total <- read.csv(exp_file, row.names = 1, check.names = FALSE)
metadata_244807 <- read.csv(metadata_file)

# --- Filtrado de genes y muestras (Firma TGF-B) ---
genes_comunes <- intersect(genes_firma_unicos, rownames(expresion_total))
muestras_gse244807 <- intersect(colnames(expresion_total), metadata_244807$geo_accession)

expresion_t <- as.data.frame(t(expresion_total[genes_comunes, muestras_gse244807]))
expresion_t$Paciente_ID <- rownames(expresion_t)

# --- Procesamiento clínico ---
clinical_data <- metadata_244807 %>%
  dplyr::filter(geo_accession %in% muestras_gse244807) %>%
  dplyr::select(Paciente_ID = geo_accession, os, death, gender, age =
age_at_the_time_of_sampling) %>%
  dplyr::mutate(os = as.numeric(os), death = as.numeric(death), age = as.numeric(age))
  %>%
  dplyr::filter(!is.na(os), !is.na(death))

survival_cox_data <- clinical_data %>% inner_join(expresion_t, by = "Paciente_ID")

# --- Cálculo de Coeficientes Beta y Risk Score ---
beta_coefficients <- list()
surv_obj <- Surv(survival_cox_data$os, survival_cox_data$death)

for (gene in genes_comunes) {
  if (sd(survival_cox_data[[gene]], na.rm = TRUE) > 1e-6) {
    formula_cox <- as.formula(paste("surv_obj ~ `", gene, "`", sep=""))
    tryCatch({
      fit_uni <- coxph(formula_cox, data = survival_cox_data)
      beta_coefficients[[gene]] <- coef(fit_uni)[1]
    }, error = function(e) {})
  }
}

genes_finales <- names(beta_coefficients)
beta_vector <- unlist(beta_coefficients)
risk_scores <- as.matrix(survival_cox_data[, genes_finales]) %%% beta_vector

# --- Creación de Grupos Finales ---
clinical_data_final <- survival_cox_data %>%
  dplyr::mutate(
    Risk_Score = as.vector(risk_scores),
    Risk_Group = factor(ifelse(Risk_Score >= median(Risk_Score), "High Risk", "Low
Risk"),
                        levels = c("Low Risk", "High Risk")),
    age_group = factor(case_when(age < 60 ~ "<60", age >= 60 & age < 70 ~ "60-69",
age >= 70 ~ ">70"),
                        levels = c("<60", "60-69", ">70"))
  )

```

```
# =====
# 3. CÁLCULO DE LOS MODELOS DE SUPERVIVENCIA (LOS "FIT")
# =====
# 1. Fit para Hombres
data_hombres <- clinical_data_final %>% filter(gender == "Male")
fit_hombres <- survfit(Surv(os, death) ~ Risk_Group, data = data_hombres)

# 2. Fit para Mujeres
data_mujeres <- clinical_data_final %>% filter(gender == "Female")
fit_mujeres <- survfit(Surv(os, death) ~ Risk_Group, data = data_mujeres)

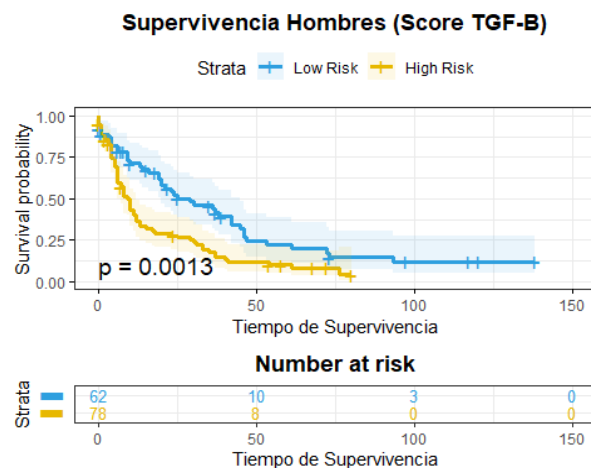
# 3. Fit para Edad
fit_age <- survfit(Surv(os, death) ~ age_group, data = clinical_data_final)

# =====
# 4. GENERACIÓN DE PLOTS
# =====

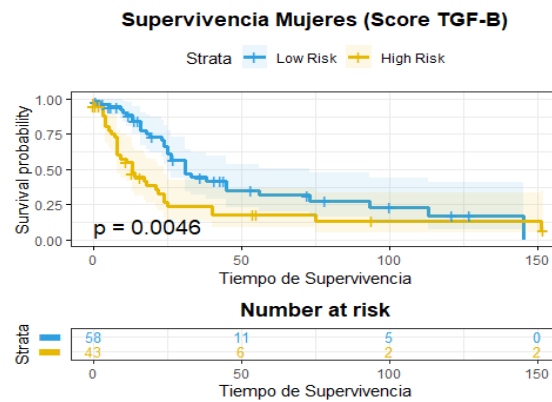
plot_km_premium <- function(fit, data, title, palette, legend_labs) {
  ggsurvplot(
    fit, data = data, title = title, palette = palette,
    size = 1.2, conf.int = TRUE, conf.int.alpha = 0.1,
    pval = TRUE, pval.coord = c(0, 0.1), risk.table = TRUE,
    risk.table.height = 0.28, risk.table.fontsize = 3.5,
    risk.table.y.text = FALSE, risk.table.col = "strata",
    legend.labs = legend_labs, xlab = "Tiempo de Supervivencia",
    ggtheme = theme_bw() + theme(plot.title = element_text(hjust = 0.5, face = "bold"))
  )
}

# Generar Los objetos de imagen
p1 <- plot_km_premium(fit_hombres, data_hombres, "Supervivencia Hombres (Score TGF-B)",
  paleta_riesgo, c("Low Risk", "High Risk"))
p2 <- plot_km_premium(fit_mujeres, data_mujeres, "Supervivencia Mujeres (Score TGF-B)",
  paleta_riesgo, c("Low Risk", "High Risk"))
p3 <- plot_km_premium(fit_age, clinical_data_final, "Supervivencia por Grupos de Edad",
  paleta_edad, c("<60", "60-69", "≥70"))

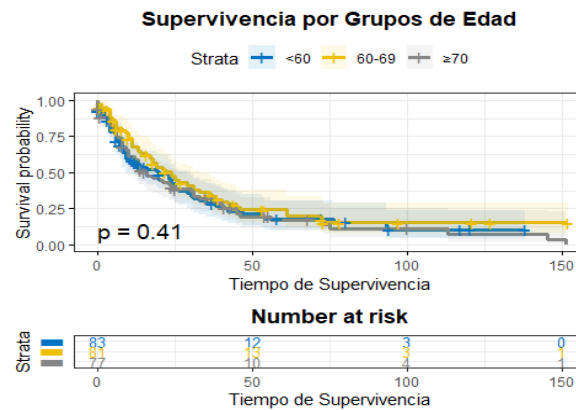
# Imprimir en pantalla
print(p1)
```




```
print(p2)
```



```
print(p3)
```



14. ANÁLISIS DE FIRMAS INMUNOLÓGICAS EN LOS PACIENTES DE LA COHORTE GSE244807

```
# 1. Instalación y Carga de Paquetes (Ajustado)
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager", repos = "https://cran.rstudio.com/")

# 2. Instalar paquetes CRAN si no están instalados
cran_packages <- c("pheatmap", "dplyr")
new_cran_packages <- cran_packages[!(cran_packages %in%
  installed.packages()[,"Package"])]
if (length(new_cran_packages) > 0) {
  install.packages(
    new_cran_packages,
    repos = "https://cran.rstudio.com/",
    Ncpus = 4
  )
}
```

```

# 3. Cargar paquetes
library(pheatmap)
library(dplyr)
library(stats)
# 4. Definición de Firmas de Genes

T_CD8 <-
c("ANXA1", "AREG", "BIRC3", "BTG2", "CCR7", "CD55", "CD69", "CREM", "CXCR4", "DNAJA1", "DNAJB1", "DUSP1", "DUSP2", "EIF3E", "FAM177A1", "FOS", "FOSB", "GADD45B", "GPR183", "GZMK", "IER2", "IL7R", "JUN", "JUNB", "KLF6", "LEPROTL1", "LMNA", "LTB", "MYADM", "NFKBIA", "NR4A2", "PIK3IP1", "PIK3R1", "PPP1R15A", "RGCC", "RGS1", "RGS2", "SELL", "SLC2A3", "SNHG8", "TMEM123", "TNFAIP3", "TOB1", "TSC22D3", "TUBA4A", "TUBB4B", "TXNIP", "YPEL5", "ZFP36", "ZNF331")

CAF <- c(
"ACTA2", "AEBP1", "ASPN", "CCDC80", "COL10A1", "COL11A1", "COL12A1", "COL1A1",
"COL1A2", "COL3A1", "COL5A1", "COL5A2", "COL6A3", "COL8A1", "COMP", "CCN2",
"CTHRC1", "CTSK", "CXCL14", "DCN", "DPT", "ELN", "FBLN1", "FBN1", "FN1", "FNDC1",
"IGFBP3", "IGFBP5", "INHBA", "ISLR", "ITGBL1", "LUM", "MFAP2", "MFAP5", "MMP11",
"MMP2", "MXRA5", "PALLD", "PLAU", "POSTN", "PRSS23", "SDC1", "SFRP2", "SFRP4",
"SPARC", "SULF1", "TAGLN", "THBS2", "TIMP3", "VCAN")

TGFB_EMT_UP <- c(
"CDH2", "AKT3", "CDH11", "DHRS2", "FSTL3", "DLC1", "MYL9", "SEMA3C", "POSTN", "MAGED2", "FERMT2", "GLIPR1", "KDELR3", "HSF2BP", "PTPN21", "ADAMTS6", "DUSP10", "COL1A1", "COL3A1", "COL4A1", "COL4A2", "COL5A1", "COL5A2", "COL6A3", "COL7A1", "MBOAT2", "VCAN", "CCN2", "JAG1", "SLC26A2", "ABCA1", "ELK3", "EML1", "EPHB2", "ETS2", "FBN1", "PXDC1", "ZNF365", "FOXD1", "DAAM1", "PALLD", "TMCC1", "ANKLE2", "RFTN1", "FN1", "ALOX5AP", "SACS", "PLEK2", "GREM1", "GNG11", "GRB10", "DSE", "LMCD1", "HMOX1", "APBB2", "HRH1", "AMIGO2", "IGFBP5", "IGFBP7", "IL11", "INHBA", "ITGA5", "ITGB3", "JARID2", "JUN", "JUNB", "KCNJ15", "KCNMA1", "VGLL3", "LAMC2", "LOX", "SRRD", "BMAL1", "LUM", "SMAD7", "MAF", "MATN3", "MFA2", "MMP1", "MMP2", "MMP9", "MMP10", "MN1", "GADD45B", "MYO10", "NCF2", "NEDD9", "NKX31", "NT5E", "CHST11", "SERPINE1", "GAL", "ANGPTL4", "GASK1B", "DACT1", "PDGFA", "CDK14", "SERPINE2", "PIK3CD", "PLAUR", "PODXL", "SPDL1", "PID1", "LARP6", "GALNT10", "CHRNA9", "ARHGEF40", "ARFGAP1", "PDGFC", "HTRA1", "NRIP3", "PMEPA1", "ACKR3", "PSMD2", "PTHLH", "PTPRK", "RALA", "RGS4", "XYLT1", "SCG5", "SMURF2", "BMP1", "SKIL", "BMP2", "SLC02A1", "SLC22A4", "SLN", "BMPR2", "SNAI2", "SPARC", "BPGM", "SPOCK1", "STC1", "TAGLN", "TCF4", "TBX3", "TGFB1", "TGFB1I1", "TGFB1", "TGM2", "THBS1", "TIMP2", "TNFAIP6", "TNS1", "TPM1", "TPM4", "TUBA4A", "TUFT1", "VEGFC", "VIM", "WNT5A", "TUBA1A", "SCG2", "C3orf52", "TFPI2", "PRR5L", "CALD1", "FHOD3", "ADAM12", "HMGA2", "WNT5B", "LBH", "MAP1LC3B", "SRPX", "TAGLN2", "TPST2", "TPST1", "DIXDC1", "BHLHE40", "RUNX2", "KLF7", "PEA15", "ACTN1", "ADAM19", "INPP4B", "SPHK1", "AP1S2", "CRLF1", "PDLIM7", "CYTH1", "FAM114A1", "NREP", "TP53I3", "MICAL2", "CD59", "DOCK4", "NUAK1", "MRC2", "GFPT2", "HS3ST3A1", "HS3ST3A1")

TGFB_EMT_DOWN <-
c("TSPAN1", "MPZL2", "SPRY1", "CITED2", "VAV3", "CEBPD", "AGR2", "SLC27A2", "TBC1D8", "PLAAT3", "CP", "ADORA2B", "CXADR", "CYB5A", "CYP1B1", "TMEM30B", "DEFB1", "SERPINB1", "ELF3", "EMP1", "EPAS1", "ERBB3", "EREG", "ALDH1A3", "ALDH3A2", "PEG10", "GSE1", "SYNE2", "NUP210", "RAB38", "RAB26", "TMT1A", "EHF", "SMPDL3B", "GLDC", "SLC04A1", "ANK3", "CFH", "FOXA2", "HPGD", "BIRC3", "AQP3", "IMPA2", "INHB", "JAG2", "AREG", "KRT15", "KRT19", "LAMA5", "LCN2", "ABLM1", "LY6E", "EPCAM", "MBP", "KITLG", "MITF", "MMP7", "MUC1", "CEACAM6", "HOOK1", "GULP1", "LSR", "PDK4", "ATP8B1", "PKP2", "PLS1", "RBM47", "EPB41L4B", "MANSC1", "ESRP1", "PPL", "PPP1R9A", "SYBU", "GPRC5C", "MYO5C", "RAB25", "MTUS1", "SQOR", "PLAAT4", "S100P", "CFB", "SCNN1A", "DEPTOR", "SLPI", "SORD", "SOX2", "DST", "SULT1A1", "TFAP2A", "NR2F2", "TNFAIP2", "TPD52L1", "C1orf116", "ALDH5A1", "FA2H", "C1orf115", "GRTP1", "ERMP1", "CAVIN2", "MAP7", "SLC16A7", "CD9", "ARHGAP29", "TJP2", "GDF15", "HS3ST1", "FGFBP1", "CDH1")

monocyte <- c("KIF17", "BTRC", "MAML1", "GRIP2", "HDAC7", "CDC42BPA", "HDAC5", "NPHP4", "FLNB", "PIP5K1C", "ITGB3", "CACNB4", "CA9", "ATN1", "ITGA9", "MAPK8", "SH3GL2", "TGM2", "LIMS1", "PFN1", "NSF", "NEBL", "WASF3", "NRG1", "SPEN", "SEMA4C", "TEAD1")

T_CD4 <- c("ALOX5AP", "ANXA1", "AREG", "BTG2", "CCL5", "CCR7", "CD48", "CD55", "CD69", "CXCR4", "DNAJB1", "DUSP1", "DUSP2", "FAM177A1", "FOS", "FOSB", "FOXP1", "GIMAP4", "GIMAP7", "GPR171", "GPR183", "GZMA", "IER2", "IL7R", "JUN", "JUNB", "KLF2", "KLRB1", "LEPROTL1", "LMNA", "NOSIP", "NR4A2", "PLAC8", "PPP2R5C", "PTGER4", "PLAAT4", "RGCC", "SELL", "SLC2A3", "SNHG8", "SOCS3", "TCF7", "TMEM123", "TOB1", "TRAT1", "TXNIP", "YPEL5", "ZFP36", "ZFP36L2", "ZNF331")

```

```

# Lista de todas las firmas

gene_signatures_list <- list(
  "T_CD8_Signature" = T_CD8,
  "T_CD4_Signature" = T_CD4,
  "Monocyte_Signature" = monocyte,
  "CAF_Signature" = CAF,
  "TGFB_EMT_UP_Signature" = TGFB_EMT_UP,
  "TGFB_EMT_DOWN_Signature" = TGFB_EMT_DOWN
)

# --- Muestras de Interés (GSE244807) ---
GSE244807_samples <- c("GSM7829922", "GSM7829924", "GSM7829925", "GSM7829926",
"GSM7829927", "GSM7829928", "GSM7829929", "GSM7829930", "GSM7829931", "GSM7829932",
"GSM7829933", "GSM7829934", "GSM7829935", "GSM7829936", "GSM7829937", "GSM7829938",
"GSM7829939", "GSM7829940", "GSM7829941", "GSM7829942", "GSM7829943", "GSM7829944",
"GSM7829945", "GSM7829947", "GSM7829948", "GSM7829949", "GSM7829950", "GSM7829951",
"GSM7829952", "GSM7829954", "GSM7829955", "GSM7829956", "GSM7829957", "GSM7829958",
"GSM7829959", "GSM7829960", "GSM7829961", "GSM7829962", "GSM7829963", "GSM7829964",
"GSM7829965", "GSM7829966", "GSM7829967", "GSM7829968", "GSM7829969", "GSM7829970",
"GSM7829971", "GSM7829972", "GSM7829973", "GSM7829974", "GSM7829975", "GSM7829976",
"GSM7829977", "GSM7829978", "GSM7829979", "GSM7829980", "GSM7829981", "GSM7829982",
"GSM7829983", "GSM7829984", "GSM7829985", "GSM7829986", "GSM7829987", "GSM7829988",
"GSM7829989", "GSM7829990", "GSM7829991", "GSM7829992", "GSM7829993", "GSM7829994",
"GSM7829995", "GSM7829996", "GSM7829997", "GSM7829998", "GSM7829999", "GSM7830000",
"GSM7830001", "GSM7830002", "GSM7830003", "GSM7830004", "GSM7830005", "GSM7830006",
"GSM7830007", "GSM7830008", "GSM7830009", "GSM7830010", "GSM7830011", "GSM7830012",
"GSM7830013", "GSM7830014", "GSM7830015", "GSM7830016", "GSM7830017", "GSM7830018",
"GSM7830019", "GSM7830020", "GSM7830021", "GSM7830022", "GSM7830023", "GSM7830024",
"GSM7830025", "GSM7830026", "GSM7830027", "GSM7830028", "GSM7830029", "GSM7830030",
"GSM7830031", "GSM7830032", "GSM7830033", "GSM7830034", "GSM7830035", "GSM7830036",
"GSM7830037", "GSM7830038", "GSM7830039", "GSM7830040", "GSM7830041", "GSM7830042",
"GSM7830043", "GSM7830044", "GSM7830045", "GSM7830046", "GSM7830047", "GSM7830048",
"GSM7830049", "GSM7830050", "GSM7830051", "GSM7830052", "GSM7830053", "GSM7830054",
"GSM7830055", "GSM7830056", "GSM7830057", "GSM7830058", "GSM7830059", "GSM7830060",
"GSM7830061", "GSM7830062", "GSM7830063", "GSM7830064", "GSM7830065", "GSM7830066",
"GSM7830067", "GSM7830068", "GSM7830069", "GSM7830070", "GSM7830071", "GSM7830072",
"GSM7830073", "GSM7830074", "GSM7830075", "GSM7830076", "GSM7830077", "GSM7830078",
"GSM7830079", "GSM7830080", "GSM7830081", "GSM7830082", "GSM7830083", "GSM7830084",
"GSM7830085", "GSM7830087", "GSM7830088", "GSM7830089", "GSM7830090", "GSM7830091",
"GSM7830092", "GSM7830093", "GSM7830094", "GSM7830095", "GSM7830096", "GSM7830097",
"GSM7830098", "GSM7830099", "GSM7830100", "GSM7830101", "GSM7830102", "GSM7830103",
"GSM7830104", "GSM7830105", "GSM7830106", "GSM7830107", "GSM7830108", "GSM7830109",
"GSM7830110", "GSM7830111", "GSM7830112", "GSM7830113", "GSM7830114", "GSM7830115",
"GSM7830116", "GSM7830117", "GSM7830118", "GSM7830119", "GSM7830120", "GSM7830121",
"GSM7830122", "GSM7830123", "GSM7830124", "GSM7830125", "GSM7830126", "GSM7830127",
"GSM7830128", "GSM7830129", "GSM7830131", "GSM7830132", "GSM7830133", "GSM7830134",
"GSM7830135", "GSM7830136", "GSM7830137", "GSM7830138", "GSM7830139", "GSM7830140",
"GSM7830141", "GSM7830142", "GSM7830143", "GSM7830144", "GSM7830145", "GSM7830146",
"GSM7830147", "GSM7830148", "GSM7830149", "GSM7830150", "GSM7830151", "GSM7830152",
"GSM7830153", "GSM7830154", "GSM7830155", "GSM7830156", "GSM7830157", "GSM7830158",
"GSM7830159", "GSM7830160", "GSM7830161", "GSM7830162", "GSM7830163", "GSM7830164",
"GSM7830165", "GSM7830166", "GSM7830167")
cat("Muestras GSE244807 para análisis de supervivencia:", length(GSE244807_samples),
"\n")

## Muestras GSE244807 para análisis de supervivencia: 241

# --- 2.1. Cargar la Matriz de Expresión ---
print("Cargando la matriz de expresión génica desde 'matriz_corregida_ComBat.csv'...")

```

```

## [1] "Cargando la matriz de expresión génica desde 'matriz_corregida_ComBat.csv'..."

exp_data <- read.csv("matriz_corregida_ComBat.csv", row.names = 1, stringsAsFactors =
FALSE)
exp_matrix_num <- as.matrix(exp_data)

# --- 2.2. Filtrado de Muestras ---
valid_samples <- intersect(GSE244807_samples, colnames(exp_matrix_num))

if (length(valid_samples) == 0) {
  stop("¡Error! No se encontró ninguna de las muestras de GSE244807 en la matriz cargada.
Asegúrate de que los nombres de columna coincidan con los IDs GSM y que el archivo se
cargó correctamente.")
} else if (length(valid_samples) < length(GSE244807_samples)) {
  warning(paste("Advertencia: Solo", length(valid_samples), "de las",
length(GSE244807_samples), "muestras fueron encontradas y se utilizarán para el
análisis."))
}

exp_matrix_filtered <- exp_matrix_num[, valid_samples]

print(paste("Matriz filtrada lista. Dimensiones (Genes x Muestras):",
paste(dim(exp_matrix_filtered), collapse = " x ")))

## [1] "Matriz filtrada lista. Dimensiones (Genes x Muestras): 24144 x 241"

# --- 3.1. Ejecutar el cálculo de puntajes de firmas (Alternativa a GSVA) ---
print("Ejecutando el cálculo de puntajes de firmas usando el Método de Expresión
Promedio...")

## [1] "Ejecutando el cálculo de puntajes de firmas usando el Método de Expresión
Promedio..."

# Crear una matriz vacía para guardar los puntajes (Firmas x Muestras)
mean_scores_matrix <- matrix(
  NA,
  nrow = length(gene_signatures_list),
  ncol = ncol(exp_matrix_filtered),
  dimnames = list(names(gene_signatures_list), colnames(exp_matrix_filtered))
)

# Iterar sobre cada firma y calcular el puntaje promedio por muestra
for (signature_name in names(gene_signatures_list)) {
  genes <- gene_signatures_list[[signature_name]]

  # Intersección de genes en la matriz de expresión y la firma
  common_genes <- intersect(genes, rownames(exp_matrix_filtered))

  if (length(common_genes) == 0) {
    warning(paste("Advertencia: Ningún gen de la firma", signature_name, "se encontró en
la matriz de expresión."))
    next
  }

  # Extraer la submatriz de genes de la firma
  sub_matrix <- exp_matrix_filtered[common_genes, , drop = FALSE]

  # Calcular la media de expresión de los genes de la firma por columna (por muestra)
  # Usamos log2(expresión + 1) en este tipo de análisis, por lo que la media es
  apropiada.
  mean_scores_matrix[signature_name, ] <- colMeans(sub_matrix)

```

```

}

# La matriz final de puntajes (sin escalar todavía)
signature_scores <- mean_scores_matrix[!is.na(mean_scores_matrix[,1]), ]

print("Cálculo de puntajes promedio completado.")

## [1] "Cálculo de puntajes promedio completado."

print(signature_scores[1:4, 1:4])

##           GSM7829922 GSM7829924 GSM7829925 GSM7829926
## T_CD8_Signature    5.080365  5.601284  5.703204  5.380020
## T_CD4_Signature    4.179903  4.667863  4.991679  4.436264
## Monocyte_Signature 5.111158  4.976204  4.285576  4.641466
## CAF_Signature      6.475318  5.752682  5.553044  6.441361

# --- 4.1. Escalar las puntuaciones para visualización (ESTANDARIZACIÓN) ---
gsva_scaled <- t(scale(t(signature_scores)))

# =====
# 5. AGRUPACIÓN Y ANOTACIÓN BASADA EN TGF-B (CLAVE PARA EL HEATMAP)
# =====
# 5.1. y 5.2. Definir el criterio de grupo y crear el Dataframe de Anotación
up_scores <- signature_scores["TGFB_EMT_UP_Signature", ]
down_scores <- signature_scores["TGFB_EMT_DOWN_Signature", ]
group_assignment <- ifelse(up_scores > down_scores, "TGFB_UP", "TGFB_DOWN")

annotation_col_df <- data.frame(
  TGFB_Group = group_assignment,
  row.names = colnames(gsva_scaled)
)

# 5.3. Definir Colores para la Anotación (¡Amarillo para UP, Verde para DOWN!)
annotation_colors_list <- list(
  TGFB_Group = c(
    "TGFB_UP" = "#FFD700", # Amarillo (Dorado) para Upregulated
    "TGFB_DOWN" = "#3CB371" # Verde para Downregulated
  )
)

# 5.4. Calcular la Matriz de Distancia Específica (Basada solo en firmas TGF-B)
clustering_signatures <- signature_scores[c("TGFB_EMT_UP_Signature",
"TGFB_EMT_DOWN_Signature"), ]
dist_matrix_cols <- dist(t(clustering_signatures), method = "euclidean")
cluster_cols_specific <- hclust(dist_matrix_cols, method = "ward.D2")

# =====
# 6. GENERACIÓN DEL HEATMAP (AJUSTADO)
# =====

heatmap_title_custom <- paste("Firmas Inmunológicas")

pheatmap(
  gsva_scaled,
  main = heatmap_title_custom,
  color = colorRampPalette(c("blue", "white", "red"))(50),

```

```

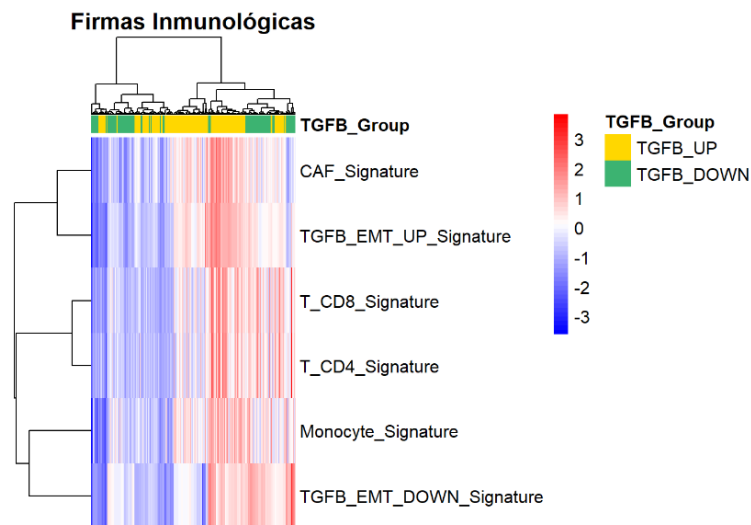
# Clustering y Distancias
clustering_distance_rows = "euclidean",
clustering_method = "ward.D2",

# Parámetros CLAVE para la agrupación por columna:
clustering_distance_cols = dist_matrix_cols,
cluster_cols = cluster_cols_specific,

# Anotación de Columnas (Muestras)
annotation_col = annotation_col_df,
annotation_colors = annotation_colors_list,

# Visualización
show_colnames = FALSE,
show_rownames = TRUE,
border_color = NA,
fontsize = 12
)

```



```

# =====
# 7. GENERACIÓN DE BOX PLOTS COMPARATIVOS (Por Firma vs Grupo TGF-B)
# =====
cran_packages_plot <- c("ggplot2", "ggpubr")
new_cran_packages_plot <- cran_packages_plot[!(cran_packages_plot %in%
installed.packages()[,"Package"])]
if (length(new_cran_packages_plot) > 0) {
  install.packages(
    new_cran_packages_plot,
    repos = "https://cran.rstudio.com/",
    Ncpus = 4
  )
}
library(ggplot2)
library(ggpubr)
library(tidyr)

# Asegúrate de que el directorio de salida para plots exista
output_dir_boxplots <- "output_plots/boxplots"
if (!dir.exists(output_dir_boxplots)) {
  dir.create(output_dir_boxplots, recursive = TRUE)
}

```

```

}

# --- 7.2. Preparación de Datos ---
df_scores_long <- as.data.frame(t(signature_scores)) %>%
  # Añadir la asignación de grupo TGF-B
  dplyr::mutate(TGFB_Group = group_assignment) %>%
  # Convertir a formato largo para tener todas las firmas en una sola columna
  'Signature'
  tidyr::pivot_longer(
    cols = -TGFB_Group,
    names_to = "Signature",
    values_to = "Score"
  )

print("Dataframe para Box Plots preparado.")

## [1] "Dataframe para Box Plots preparado."

print(head(df_scores_long))

## # A tibble: 6 × 3
##   TGFB_Group Signature      Score
##   <chr>      <chr>      <dbl>
## 1 TGFB_UP    T_CD8_Signature    5.08
## 2 TGFB_UP    T_CD4_Signature    4.18
## 3 TGFB_UP    Monocyte_Signature 5.11
## 4 TGFB_UP    CAF_Signature      6.48
## 5 TGFB_UP    TGFB_EMT_UP_Signature 5.08
## 6 TGFB_UP    TGFB_EMT_DOWN_Signature 5.00

# --- 7.3. Iteración y Ploteo ---

# Definir los colores de grupo que usaste en el heatmap (Amarillo/Verde)
group_colors <- c(
  "TGFB_UP" = "#FFD700", # Amarillo para Upregulated
  "TGFB_DOWN" = "#3CB371" # Verde para Downregulated
)

# Lista de todas las firmas a plotear
all_signatures <- names(gene_signatures_list)

cat("\n--- Generando Box Plots comparativos (Signature Score vs Grupo TGF-B) ---\n")

##
## --- Generando Box Plots comparativos (Signature Score vs Grupo TGF-B) ---

for (sig in all_signatures) {

  # 1. Filtrar los datos para la firma actual
  df_plot <- df_scores_long %>%
    dplyr::filter(Signature == sig)

  # 2. Generar el Box Plot
  p <- ggplot(df_plot, aes(x = TGFB_Group, y = Score, fill = TGFB_Group)) +

    # Añadir Boxplots
    geom_boxplot(width = 0.5, alpha = 0.7, outlier.shape = NA) +

```

```

# Añadir Los puntos individuales (jitter) para visualizar la distribución real de las
muestras
geom_jitter(width = 0.15, size = 1, alpha = 0.6) +

# Aplicar los colores definidos
scale_fill_manual(values = group_colors) +

# Títulos y Etiquetas
labs(
  title = paste("Puntuación de la Firma:", gsub("_", " ", sig)),
  x = "Grupo de Clasificación TGF-β",
  y = paste(sig, "Puntuación (Expresión Promedio)",
  fill = "Grupo TGF-β"
) +

# Tema Limpio
theme_bw() +
theme(
  plot.title = element_text(hjust = 0.5, face = "bold"),
  legend.position = "none"
)

# 3. Añadir el Test Estadístico (Wilcoxon Rank Sum Test)
p <- p + stat_compare_means(
  method = "wilcox.test",
  comparisons = list(c("TGFB_UP", "TGFB_DOWN")),
  label = "p.format",
  tip.length = 0.01,
  label.y.npc = "top", # Coloca el p-valor en la parte superior
  size = 4
)

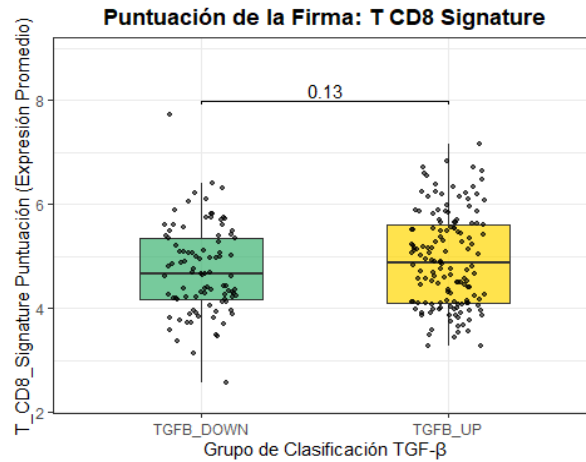
# 4. Ajustar el eje Y para hacer espacio para el p-valor
max_score <- max(df_plot$Score)
p <- p + coord_cartesian(ylim = c(min(df_plot$Score) * 0.9, max_score * 1.15))

# 5. Imprimir y guardar el plot
print(p)

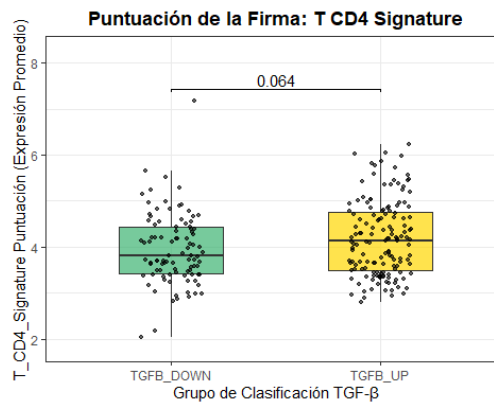
plot_filename <- file.path(output_dir_boxplots, paste0("BoxPlot_", sig,
"_vs_TGFBGroup.png"))
ggsave(plot_filename, plot = p, width = 6, height = 5)

cat(paste0(" Box Plot guardado para ", sig, ": ", plot_filename, "\n"))
}

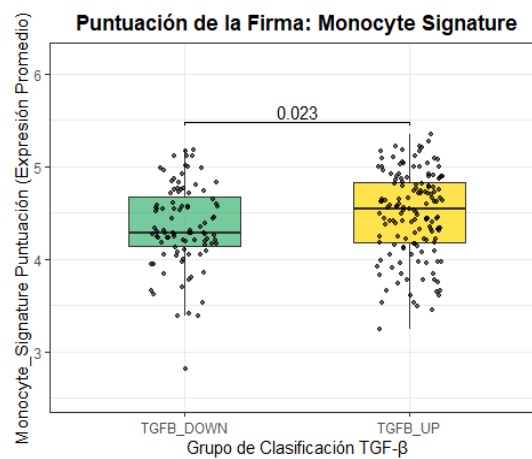
```

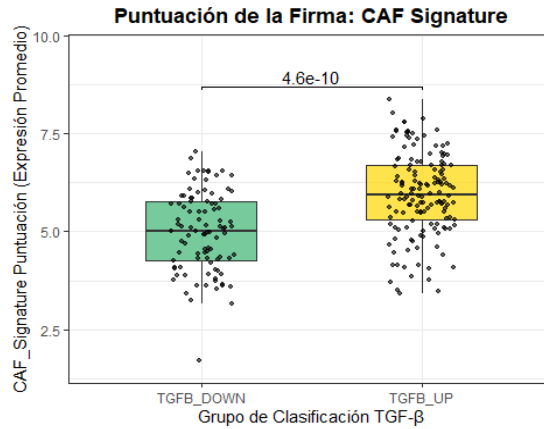
☒ Box Plot guardado para T_CD8_Signature:
 output_plots/boxplots/BoxPlot_T_CD8_Signature_vs_TGFBGroup.png



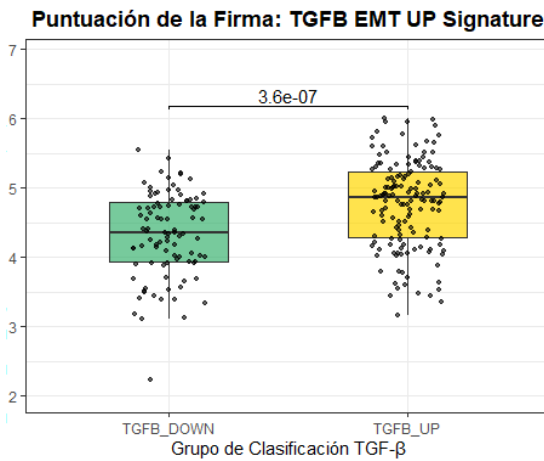
☒ Box Plot guardado para T_CD4_Signature:
 output_plots/boxplots/BoxPlot_T_CD4_Signature_vs_TGFBGroup.png



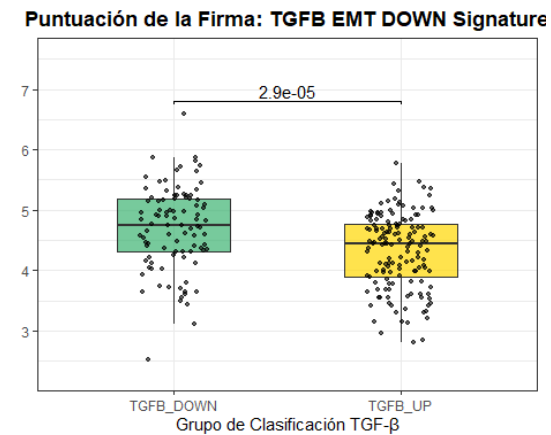
☒ Box Plot guardado para Monocyte_Signature:
 output_plots/boxplots/BoxPlot_Monocyte_Signature_vs_TGFBGroup.png



☒ Box Plot guardado para CAF_Signature:
 output_plots/boxplots/BoxPlot_CAF_Signature_vs_TGFβGroup.png



☒ Box Plot guardado para TGFβ_EMT_UP_Signature:
 output_plots/boxplots/BoxPlot_TGFβ_EMT_UP_Signature_vs_TGFβGroup.png



☒ Box Plot guardado para TGFβ_EMT_DOWN_Signature:
 output_plots/boxplots/BoxPlot_TGFβ_EMT_DOWN_Signature_vs_TGFβGroup.png